

Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

September 21, 2026

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Problem Statement	5
1.3	Contributions	5
1.4	Thesis Outline	6
2	Regular Expressions	7
2.1	Syntax	7
2.2	Semantics	8
2.3	Rust Implementation	9
3	Derivatives and Partial Derivatives	10
3.1	Naive Matching	10
3.2	Brzozowski Derivatives	12
3.3	Antimirov Partial Derivatives	16
4	Matching versus Parsing	20
4.1	The Matching Problem	20
4.2	Parse Trees as Matching Evidence	21
4.3	Disambiguation Policies	24
4.3.1	Greedy Leftmost Matching	25
4.3.2	The Infinite Empty-Match Chain Problem	27
4.3.3	POSIX Longest-Leftmost Matching	28
5	Derivative-Based Parser	32
5.1	Reading the Implementation's Traces	32
5.2	Core Algorithm	33
5.3	Bit-Coded Optimization (Theory)	36
5.4	Simplification Rules	39
5.5	Rust Implementation	42
5.5.1	The Bit-Coded Parser	43
5.6	Comparing the Two Constructions	43
6	Partial-Derivative-Based Parser	46
6.1	Extending Derivatives to Partial Derivatives	46
6.2	Bit-Coded Variant	48
6.3	Rust Implementation: The <code>pderiv_bc</code> Parser	49
6.4	Rust Implementation: The <code>pderiv_std</code> Parser	51

6.5	Frontier Growth	54
6.6	Comparing the Two Policies	55
7	External versus Internal Regex	58
7.1	Two ASTs, One Lowering Step	58
7.2	Parsing Surface Syntax	59
7.3	Translating <code>ExtPat</code> into <code>Regex</code>	61
7.4	The Cost of Translation	63
8	Evaluation	66
8.1	Compared Engines	66
8.1.1	Rust's <code>regex</code> Crate	66
8.1.2	RE2	67
8.2	Testing Strategy	67
8.3	Dataset Preparation	67
8.3.1	Substring Search: How the Corpus Is Run	69
8.4	Correctness Verification	70
8.4.1	Paper Examples Reproduced	70
8.4.2	Cross-Implementation Equivalence	71
8.5	Performance Analysis	72
8.5.1	Stack Depth: Recursive vs. Iterative Construction	72
8.5.2	Heap Allocation: <code>deriv_std</code> vs. <code>deriv_bc</code>	72
8.5.3	Derivative-Based vs. Partial-Derivative-Based	72
8.5.4	Pathological Inputs	72
8.5.5	Benchmark Results	72
8.6	Comparison with Existing Engines	73
8.6.1	Performance	73
8.6.2	Standardized Benchmarking via <code>rebar</code>	73
8.7	Optimization Challenges and Insights	73
8.8	Discussion of Results	74
9	Related Work and Conclusion	75
9.1	Related Work	75
9.2	Conclusion	75
A	Code	78
A.1	Core Types	78
A.2	Shared Algorithms	79
A.3	Matchers	81
A.4	The <code>deriv_std</code> Parser	81
A.5	The <code>deriv_bc</code> Parser	82
A.6	The <code>pderiv_bc</code> Parser	83
A.7	The <code>pderiv_std</code> Parser	85
A.8	Frontend	87
B	Tests	90
B.1	Shared Fixtures	90
B.2	Paper Examples Reproduced	91
B.3	Cross-Implementation Equivalence	92

B.4	Frontend	93
C	Diagnostics	94
C.1	deriv_std_rec, match	94
C.2	deriv_bc, match	96
C.3	pderiv_std, match	97
C.4	pderiv_bc, match	99
C.5	deriv_std_rec, no match	100
C.6	deriv_bc, no match	101
C.7	pderiv_std, no match	102
C.8	pderiv_bc, no match	104

1 Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the problem is regular expression matching. Given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression $(ab|a)^*$ matches "aba", as it decomposes into "ab" followed by "a", but does not match "abb", since no sequence of subexpressions can produce it.

For many applications, a Boolean answer is enough. An input validator may only need to accept or reject a password or URL. Others require more. For instance, a lexer must determine how input decomposes into tokens. Capturing groups, schema validation, and information extraction require identifying which input parts correspond to parts of the expression. Recognition alone is not enough; the matcher must also recover the *structure*.

Efficiently solving these problems remains an active research area. The choice of algorithm influences performance, memory consumption, and security. Once matching becomes parsing, the way in which match structure is recovered also influences correctness and raises the question of how to handle ambiguity when multiple matches are possible.

Existing approaches include backtracking and finite-automata-based techniques, such as nondeterministic (NFA) and deterministic (DFA) automata. These approaches have different trade-offs in implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely used, but may exhibit exponential running time on constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require preprocessing or suffer from state-space explosion.

Derivative-based algorithms provide an alternative by transforming the regular expression as input symbols are consumed. Each derivative represents the language remaining after an input symbol has been consumed. By maintaining a correspondence between an expression and its successive residuals, derivative-based methods provide an elegant foundation for regular expression matching and, with suitable extensions, structured parsing.

However, derivative-based techniques are less common in practical regular expression tooling. Their use for practical matching, parsing, and diagnostic tools motivates this thesis: "From Theory to Tools".

1.2 Problem Statement

Derivative-based approaches provide a principled foundation for regular expression matching and parsing. Brzozowski introduced regular expression derivatives as a way of transforming a regular expression after consuming an input symbol [Brzozowski, 1964]. Antimirov later introduced partial derivatives, which represent sets of residual expressions and are closely related to nondeterministic automata [Antimirov, 1996].

Building on Brzozowski’s derivatives, Sulzmann and Lu developed a derivative-based algorithm for POSIX regular expression parsing [Sulzmann and Lu, 2014]. Their work also presents a bit-coded variant that represents parse information as a compact bit sequence, which can be decoded after a successful match rather than maintaining an explicit parse tree throughout the computation. A Haskell implementation of the POSIX parser is available in Sulzmann’s *Programming Paradigms* teaching material [Sulzmann, 2026].

Sulzmann and Lu also studied partial derivatives in the context of regular expression sub-matching [Sulzmann and Lu, 2012]. While sub-matching is not the focus of this thesis, their work provides relevant background for the partial-derivative approach. The *Programming Paradigms* teaching material also presents a bit-coded partial-derivative-based parser in Haskell.

These constructions have primarily been presented in a theoretical and functional programming setting, leaving open practical questions about execution time, memory consumption, and data representation.

This thesis investigates these questions through implementations in Rust. As a systems programming language, Rust provides fine-grained control over memory allocation and data representation, allowing the derivative-based and partial-derivative-based parsers, including their bit-coded variants, to be evaluated in practice.

1.3 Contributions

The following contributions are being made:

- **Rust implementation of derivative-based parsers.** The thesis implements derivative-based and partial-derivative-based parsing constructions in Rust, including direct and bit-coded variants. The implementations are based on the constructions described in the literature and the corresponding Haskell material, resulting in four parser variants in total.
- **Empirical evaluation of the parser variants.** The four implementations are evaluated with respect to execution time and memory consumption using synthetic benchmarks and a corpus of real-world regular expressions from Snort/Suricata, SpamAssassin, and RegexLib, following the source categories investigated by Mamouras et al. [Mamouras et al., 2024]. They are also compared with Rust’s `regex` crate and Google’s RE2.
- **Derivative-tracing diagnostics.** The implementation provides traced versions of the parsing algorithms that record intermediate derivative computations during matching. These traces make the transformations performed by the algorithms explicit and provide a basis for explaining successful and unsuccessful matches.

1.4 Thesis Outline

The remainder of this thesis follows the parsing constructions from their formal concepts through their Rust implementation to their evaluation. Theory and implementation are presented together.

- **Chapter 2 - Regular Expressions.** This chapter introduces the syntax and semantics of regular expressions and the Rust representation used throughout the implementation.
- **Chapter 3 - Derivatives.** This chapter introduces Brzozowski’s derivatives and Antimirov’s partial derivatives, together with their basic matching algorithms and Rust implementations.
- **Chapter 4 - Matching vs. Parsing.** This chapter distinguishes regular expression matching from parsing and introduces the disambiguation problem.
- **Chapter 5 - Derivative Parser.** This chapter presents Sulzmann and Lu’s derivative-based POSIX parser and its Rust implementation in direct and bit-coded variants.
- **Chapter 6 - Partial-Derivative Parser.** This chapter presents the partial-derivative-based parser and its Rust implementation in direct and bit-coded variants. It also discusses the differences between the two derivative-based approaches.
- **Chapter 7 - External versus Internal Syntax.** This chapter describes the parsing of regular expression strings into an external AST and their translation into the internal representation used by the parsers.
- **Chapter 8 - Evaluation.** This chapter evaluates the four parser variants with respect to execution time, memory consumption, and correctness. The evaluation uses synthetic benchmarks and a corpus of real-world regular expressions, and compares the implementations with Rust’s `regex` crate and Google’s RE2. It also examines optimization challenges and the insights gained from the evaluation, including the derivative traces produced by the implementations.
- **Chapter 9 - Conclusion.** Finally, this chapter summarizes the findings of the thesis and discusses the practical implications of the results and directions for future work.

2 Regular Expressions

2.1 Syntax

Let Σ be an alphabet. Regular expressions over Σ are defined inductively by the grammar

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \quad (x \in \Sigma). \quad (2.1)$$

Here, x denotes a literal symbol, ε the expression matching only the empty word, and $\emptyset$ the expression matching no word. The operators $+$, $\cdot$, and $*$ denote alternation, concatenation, and Kleene star, respectively. Concatenation is often written implicitly, so rs denotes $r \cdot s$.

Unless parentheses are shown explicitly, Kleene star binds most tightly, followed by concatenation, followed by alternation. Thus, for example, ab^*c is parsed as $(a \cdot (b^*)) \cdot c$, while $a + bc^*$ is parsed as $a + (b \cdot (c^*))$.

At the language level, $+$ and $\cdot$ are associative: different groupings denote the same language. For example, $L((r + s) + t) = L(r + (s + t))$ and $L((r \cdot s) \cdot t) = L(r \cdot (s \cdot t))$. The grouping determines the *syntax tree* on which the algorithms operate. We therefore use left association for both operators: $r + s + t$ means $(r + s) + t$, while rst means $(rs)t$.

The grammar omits operators such as one-or-more, optional, and bounded repetition (r^+ , $r^?$, and $r^{\{m,n\}}$). These operators can be expressed using the six core constructs. For example, $r^+ = r \cdot r^*$ and $r^? = r + \varepsilon$. Bounded repetition can be expanded as

$$r^{\{m,n\}} = \underbrace{r \cdot \dots \cdot r}_{m \text{ times}} \cdot \underbrace{(r + \varepsilon) \cdot \dots \cdot (r + \varepsilon)}_{n-m \text{ times}}.$$

Thus, these convenience operators do not add expressive power to the core syntax. In the implementation, expressions using such operators are translated into the six core constructs before they are processed by the parsing algorithms; the translation from external regular expression syntax to this internal representation is described in Chapter 7.

As a running example, consider the regular expression for binary strings ending in "01". Over $\Sigma = \{0, 1\}$, let

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1. \quad (2.2)$$

The expression consists of a Kleene star applied to the alternation $0 + 1$, followed by the literals 0 and 1. By the precedence and left-association rules, its syntax tree is $((0 + 1)^* \cdot 0) \cdot 1$. The algorithms operate on this syntax tree, with each node corresponding to one of the constructs in the grammar.

2.2 Semantics

The syntax above describes the *shape* of a regular expression, while its semantics determines the language it denotes. A *word* over Σ is a finite sequence of symbols from Σ . The set of all words over Σ , including the empty word ε , is denoted by Σ^* .

For a regular expression r , write $L(r)$ for the language denoted by r , where $L(r) \subseteq \Sigma^*$. The semantics is defined by structural recursion on r :

$$\begin{aligned}
 L(x) &= \{x\}, \\
 L(\varepsilon) &= \{\varepsilon\}, \\
 L(\emptyset) &= \emptyset, \\
 L(r + s) &= L(r) \cup L(s), \\
 L(r \cdot s) &= \{vw \mid v \in L(r), w \in L(s)\}, \\
 L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid n \geq 1, w_1, \dots, w_n \in L(r)\}.
 \end{aligned} \tag{2.3}$$

Some simple examples are shown in Table 2.1.

Expression	Language
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\emptyset$
$a + b$	$\{a, b\}$
ab	$\{ab\}$
a^*	$\{\varepsilon, a, aa, aaa, \dots\}$

Table 2.1: Examples of regular expressions and their denoted languages.

A word w *matches* a regular expression r when $w \in L(r)$. Determining whether this holds is the *membership problem*.

For the running example, $L(0 + 1) = \{0, 1\}$, so $L((0 + 1)^*) = \{0, 1\}^*$. Concatenating this language with $L(0) = \{0\}$ and $L(1) = \{1\}$ gives

$$L(r_{01}) = \{w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^*\}. \tag{2.4}$$

Thus, $L(r_{01})$ is exactly the set of binary strings ending in "01", including "01", "001", and "1101". The empty string is not in $L(r_{01})$, since every word in the language ends with the two symbols 0 and 1.

An expression therefore has both a syntactic structure and a meaning: the syntax determines how the expression is constructed, while its semantics determines the language it denotes. Different syntax trees may denote the same language. This distinction is important for the algorithms developed later: the algorithms manipulate regular expressions and their syntax trees, while $L(r)$ gives the set of words represented by the expression.

2.3 Rust Implementation

The grammar above is represented directly as a Rust enum [The Rust Project Developers, 2026c,h], with one variant for each syntactic construct:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi,
    Eps,
    Lit(char),
    Seq(Box<Regex>, Box<Regex>),
    Alt(Box<Regex>, Box<Regex>),
    Star(Box<Regex>),
}
```

The variants correspond directly to the grammar: `Phi`, `Eps`, and `Lit` represent $\emptyset$, ε , and literals, while `Seq`, `Alt`, and `Star` represent concatenation, alternation, and Kleene star.

For example, the running example $r_{01} = (0+1)^* \cdot 0 \cdot 1$ is represented by an expression tree whose root is the concatenation of $(0+1)^* \cdot 0$ and 1 , following the left-associated grouping from Section 2.1. Its structure is:

$$\text{Seq}(\text{Seq}(\text{Star}(\text{Alt}(0, 1)), 0), 1). \quad (2.5)$$

The same structure is represented in Rust by nested enum values. The outermost `Seq` represents the final concatenation with 1 , and its left child represents $(0+1)^* \cdot 0$. Thus, the Rust value stores the parsed structure of the expression directly, rather than only its textual notation such as $(0+1)^* \cdot 0 \cdot 1$. Since a regular expression can contain other regular expressions, these nested values allow functions on `Regex` to be defined recursively as well.

The recursive variants use `Box<Regex>` because Rust requires recursive types to have a finite size. A definition such as `Seq(Regex, Regex)` would have no finite size, since each `Regex` could contain another recursively. `Box<Regex>` provides the required indirection by storing the recursive value separately [The Rust Project Developers, 2026a,j].

This explicit indirection differs from the Haskell specification on which the implementation is based. Haskell allows recursive algebraic data types without an explicit `Box`-like constructor, whereas Rust requires the indirection because enum sizes must be known at compile time. The Rust representation therefore makes the heap-based structure of recursive expressions explicit, which is relevant to memory behaviour.

The Rust representation also derives several standard traits. `Debug` supports inspection, `Clone` allows duplication, and `Eq` and `Hash` allow `Regex` to be used as a `HashSet` key. The latter are required by the partial-derivative implementation, which stores sets of residual expressions [The Rust Project Developers, 2026b,f,e].

Rust’s `match` expressions are checked for exhaustiveness by the compiler [The Rust Project Developers, 2026d,i]. Thus, a function defined by matching on `Regex` must account for every syntactic construct of the grammar. This makes the Rust implementation closely follow the mathematical definition: each case of the grammar has a corresponding case in the implementation.

3 Derivatives and Partial Derivatives

Chapter 2 introduced the syntax and semantics of regular expressions. This chapter introduces three algorithms for deciding whether $w \in L(r)$.

The first is a naive recursive matcher that follows the structure of the regular-expression semantics. The second uses Brzozowski derivatives to transform a regular expression as input symbols are consumed. The third uses Antimirov’s partial derivatives, representing the possible residual expressions as a set rather than combining them into a single expression. All three approaches are implemented in Rust.

All three matchers `match_naive`, `match_deriv`, and `match_pderiv` share the signature `fn(&str, &Regex) -> bool`, so the same expression and input can be processed by any of them.

3.1 Naive Matching

A direct way to decide whether $w \in L(r)$ is to interpret the regular-expression semantics as recursive matching rules. The resulting naive matcher follows the structure of the expression and searches for possible decompositions of the input.

Let w be given as an indexable sequence of characters, and let $w[i : j]$ denote the substring beginning at position i and ending immediately before position j . A recursive procedure `match(w, i, j, r)` determines whether $w[i : j] \in L(r)$ by following the structure of r :

$$\begin{aligned}
 \text{match}(w, i, j, \varepsilon) &\iff i = j, \\
 \text{match}(w, i, j, x) &\iff j = i + 1 \wedge w[i] = x, \\
 \text{match}(w, i, j, r + s) &\iff \text{match}(w, i, j, r) \vee \text{match}(w, i, j, s), \\
 \text{match}(w, i, j, r \cdot s) &\iff \exists k \in [i, j]. \text{match}(w, i, k, r) \wedge \text{match}(w, k, j, s), \\
 \text{match}(w, i, j, r^*) &\iff i = j \vee \exists k \in (i, j]. \text{match}(w, i, k, r) \wedge \text{match}(w, k, j, r^*).
 \end{aligned} \tag{3.1}$$

Membership of the complete word w is then given by `match(w, 0, |w|, r)`. Each rule directly corresponds to the semantics of the respective regular-expression construct.

The direct correspondence with the semantics comes at a cost. In particular, the concatenation rule does not prescribe where the input should be split. It asks whether *some* split point k exists such that r matches $w[i : k]$ and s matches $w[k : j]$. The naive matcher therefore searches the possible values of k .

For example, consider matching $r \cdot s = (a \cdot a) \cdot a$ against $w[0 : 3]$, where $w = \mathbf{aaa}$. Here, $r = a \cdot a$ and $s = a$, so the recursive rule requires the matcher to consider the possible split points $k \in \{0, 1, 2, 3\}$:

$$\begin{aligned}
& \underbrace{\text{match}(w, 0, 0, a \cdot a)}_{a \cdot a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 3, a)}_{a \text{ against } w[0:3]=\mathbf{aaa}} \\
\vee & \underbrace{\text{match}(w, 0, 1, a \cdot a)}_{a \cdot a \text{ against } w[0:1]=\mathbf{a}} \wedge \underbrace{\text{match}(w, 1, 3, a)}_{a \text{ against } w[1:3]=\mathbf{aa}} \\
\vee & \underbrace{\text{match}(w, 0, 2, a \cdot a)}_{a \cdot a \text{ against } w[0:2]=\mathbf{aa}} \wedge \underbrace{\text{match}(w, 2, 3, a)}_{a \text{ against } w[2:3]=\mathbf{a}} \\
\vee & \underbrace{\text{match}(w, 0, 3, a \cdot a)}_{a \cdot a \text{ against } w[0:3]=\mathbf{aaa}} \wedge \underbrace{\text{match}(w, 3, 3, a)}_{a \text{ against } w[3:3]=\epsilon}.
\end{aligned}$$

The first alternative fails because $a \cdot a$ cannot match the empty substring $w[0 : 0]$. The second fails because $a \cdot a$ cannot match the one-character substring $w[0 : 1]$. The third alternative succeeds: $a \cdot a$ matches $w[0 : 2] = \mathbf{aa}$ and the final a matches $w[2 : 3] = \mathbf{a}$. Thus, the matcher finds the successful split $k = 2$ only after considering the preceding split points.

The successful alternative produces two recursive calls:

$$\underbrace{\text{match}(w, 0, 2, a \cdot a)}_{a \cdot a \text{ against } w[0:2]=\mathbf{aa}} \quad \text{and} \quad \underbrace{\text{match}(w, 2, 3, a)}_{a \text{ against } w[2:3]=\mathbf{a}}.$$

The first call is itself a concatenation match and therefore considers the split points $k \in \{0, 1, 2\}$:

$$\begin{aligned}
& \underbrace{\text{match}(w, 0, 0, a)}_{a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\mathbf{aa}} \\
\vee & \underbrace{\text{match}(w, 0, 1, a)}_{a \text{ against } w[0:1]=\mathbf{a}} \wedge \underbrace{\text{match}(w, 1, 2, a)}_{a \text{ against } w[1:2]=\mathbf{a}} \\
\vee & \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\mathbf{aa}} \wedge \underbrace{\text{match}(w, 2, 2, a)}_{a \text{ against } w[2:2]=\epsilon}.
\end{aligned}$$

The first alternative fails because the first a cannot match the empty substring $w[0 : 0]$. The second succeeds: the first a matches $w[0 : 1] = \mathbf{a}$ and the second a matches $w[1 : 2] = \mathbf{a}$. The second recursive call then checks a against $w[2 : 3] = \mathbf{a}$ and succeeds.

`src/matchers/match_naive.rs` implements this procedure directly. Although inefficient, it is useful as a reference implementation because it follows the regular-expression semantics without any intermediate transformation.

The derivative approach takes a different view of the same problem. Rather than searching for a decomposition of w while keeping r fixed, it transforms r as each character of w is consumed. The resulting expression describes the language that can still match the remaining suffix.

3.2 Brzowski Derivatives

Brzowski's derivative of r with respect to a symbol x , written $r \backslash x$, is the expression describing exactly what remains of r 's language after consuming the leading symbol x of a word $w \in L(r)$ [Brzowski, 1964]:

$$L(r \backslash x) = \{ w \mid xw \in L(r) \}. \quad (3.2)$$

For $w = x_1 x_2 \cdots x_n$, consume one symbol at a time by repeatedly taking derivatives:

$$r, \quad r \backslash x_1, \quad (r \backslash x_1) \backslash x_2, \quad \dots$$

After all symbols have been consumed, the remaining input is ε . Thus, the resulting expression must accept ε for the original word to belong to $L(r)$. This is the question of whether the expression is *nullable*. Let $n(r)$ be true exactly when $\varepsilon \in L(r)$:

$$\begin{array}{lll} n(\emptyset) = \text{false} & (\mathbf{N}_\emptyset) & n(r + s) = n(r) \vee n(s) \quad (\mathbf{N}_+) \\ n(\varepsilon) = \text{true} & (\mathbf{N}_\varepsilon) & n(r \cdot s) = n(r) \wedge n(s) \quad (\mathbf{N}_{\text{seq}}) \\ n(x) = \text{false} & (\mathbf{N}_{\text{lit}}) & n(r^*) = \text{true} \quad (\mathbf{N}_*) \end{array} \quad (3.3)$$

The empty language $\emptyset$ is not nullable because it contains no words, including ε , while a literal cannot match the empty word. ε and r^* always match ε . An alternation is nullable when either branch is nullable, and a concatenation when both components are.

The derivative rules specify the expression that remains after consuming x :

$$\begin{array}{ll} \emptyset \backslash x = \emptyset & (\mathbf{D}_\emptyset) \\ \varepsilon \backslash x = \emptyset & (\mathbf{D}_\varepsilon) \\ y \backslash x = \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases} & (\mathbf{D}_{\text{lit}}) \\ (r + s) \backslash x = (r \backslash x) + (s \backslash x) & (\mathbf{D}_+) \\ r^* \backslash x = (r \backslash x) \cdot r^* & (\mathbf{D}_*) \\ (r \cdot s) \backslash x = \begin{cases} (r \backslash x) \cdot s + s \backslash x & n(r) = \text{true} \\ (r \backslash x) \cdot s & n(r) = \text{false} \end{cases} & \begin{array}{l} (\mathbf{D}_{\text{seq}}^n) \\ (\mathbf{D}_{\text{seq}}) \end{array} \end{array} \quad (3.4)$$

The empty language contains no words, so it cannot match a word beginning with x and $\emptyset \backslash x = \emptyset$. Likewise, ε contains no symbol to consume, so $\varepsilon \backslash x = \emptyset$. For a literal y , consuming x leaves ε when $x = y$ and no possible match otherwise. For $r + s$, the symbol may come from either alternative, so both derivatives are preserved. For r^* , the symbol is consumed by one occurrence of r , after which r^* may still match the remainder.

Concatenation is the only derivative rule that depends on nullability. If r is not nullable, r must consume the leading x , leaving $r \backslash x$ as the remainder of r while s remains untouched. If r is nullable, it may either consume x , as in the non-nullable case, giving $(r \backslash x) \cdot s$, or r matches ε and is finished, so s consumes x and leaves $s \backslash x$. Hence the nullable case produces the alternation $(r \backslash x) \cdot s + s \backslash x$.

Why the case split. Brzozowski's original definition wrote the concatenation case as

$$x^{-1}(r \cdot t) = (x^{-1}r) \cdot t + o(r) \cdot x^{-1}t, \quad (3.5)$$

in the notation Antimirov uses, where $x^{-1}r$ is the derivative of r by x and $o(r)$ is the constant part of r , equal to ε when r is nullable and $\emptyset$ otherwise. Antimirov observed that deriving $(x + y)^*$ by x , then xx , then xxx , and so on under this rule gives “an infinite sequence of regular terms non-equivalent modulo $\mathcal{E}(ACI)$ ” [Antimirov, 1996, Remark 1.3, p. 298], and to restore finiteness, replaces it with his following [Antimirov, 1996, Definition 1.1, Eq. (21), p. 297]:

$$x^{-1}(r \cdot t) = \text{if } o(r) = 0 \text{ then } (x^{-1}r) \cdot t \text{ else } (x^{-1}r) \cdot t + x^{-1}t \text{ fi} \quad (3.6)$$

This is the same rule as (3.4): $x^{-1}r$ is written $r \setminus x$ there and t is written s , while $o(r) = 0$ is the non-nullable case $n(r) = \text{false}$. Its two branches are therefore D_{seq} and D_{seq}^n .

Worked example: single-symbol matching. Consider x^* against the word $\mathbf{xx}$. Writing $r \xrightarrow{x} r \setminus x$ for one derivative step,

Step 1. The expression is a star, so D_* applies with body $r = x$:

$$\begin{aligned} x^* \setminus \mathbf{x} &= (x \setminus \mathbf{x}) \cdot x^* && D_*, r = x \\ &= \varepsilon \cdot x^* && D_{\text{lit}}, \text{ since } \mathbf{x} = x \end{aligned} \quad (3.7)$$

Step 2. The expression is now the concatenation $\varepsilon \cdot x^*$, and deriving it consumes the second $\mathbf{x}$. The result is an alternation because D_{seq}^n applies with $r = \varepsilon$ and $s = x^*$, its nullable case being selected since $n(r) = n(\varepsilon) = \text{true}$ by N_ε :

$$\begin{aligned} (\varepsilon \cdot x^*) \setminus \mathbf{x} &= (\varepsilon \setminus \mathbf{x}) \cdot x^* + x^* \setminus \mathbf{x} && D_{\text{seq}}^n, r = \varepsilon, s = x^* \\ &= \emptyset \cdot x^* + x^* \setminus \mathbf{x} && D_\varepsilon \\ &= \emptyset \cdot x^* + (x \setminus \mathbf{x}) \cdot x^* && D_*, r = x \\ &= \emptyset \cdot x^* + \varepsilon \cdot x^* && D_{\text{lit}} \end{aligned} \quad (3.8)$$

After both symbols are consumed, we check whether the result is nullable:

$$\begin{aligned} n(\emptyset \cdot x^* + \varepsilon \cdot x^*) &= n(\emptyset \cdot x^*) \vee n(\varepsilon \cdot x^*) && N_+, r = \emptyset \cdot x^*, s = \varepsilon \cdot x^* \\ &= (n(\emptyset) \wedge n(x^*)) \vee (n(\varepsilon) \wedge n(x^*)) && N_{\text{seq}}, \text{ twice} \\ &= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) && N_\emptyset, N_*, N_\varepsilon \\ &= \text{true}. \end{aligned} \quad (3.9)$$

Thus x^* matches $\mathbf{xx}$.

Simplification. Every nullable concatenation makes D_{seq}^n introduce an alternation, and each alternation carries a copy of a subexpression that the next derivative step will carry further. Nothing in Equation (3.4) bounds this, so expressions are simplified after each derivative step, using identities that preserve the denoted language:

$$\begin{aligned}
\emptyset \cdot r &= \emptyset & (S_1) \\
\varepsilon \cdot r &= r & (S_2) \\
\emptyset + r &= r & (S_3) \\
r + r &= r & (S_4)
\end{aligned} \tag{3.10}$$

S_1 to S_3 remove units $\emptyset$ and ε wherever a derivative step has left one behind, and each holds equally with the unit on the right. S_4 collapses two syntactically identical summands into one, which is what keeps repeated alternations from doubling. `src/regex/simplify.rs` applies all four recursively over the expression tree, matching on both sides, and additionally reduces ε^* and $\emptyset^*$ to ε . Applying them to the result of Equation (3.8) gives:

$$\begin{aligned}
\emptyset \cdot x^* + \varepsilon \cdot x^* &= \emptyset + \varepsilon \cdot x^* & S_1, \emptyset \cdot x^* &= \emptyset \\
&= \varepsilon \cdot x^* & S_3, \emptyset + \varepsilon \cdot x^* &= \varepsilon \cdot x^* \\
&= x^* & S_2, \varepsilon \cdot x^* &= x^*
\end{aligned} \tag{3.11}$$

The case split of Equation (3.6) repaired the finiteness result that Equation (3.5) had broken, but only up to the ACI-equivalence named in the quotation above, which Antimirov [1996, Section 1.1, p. 293] defines. It does not prevent expressions from growing syntactically. This can be seen by repeatedly deriving Antimirov’s example $(x + y)^*$ by the same symbol $\mathbf{x}$, without simplifying between steps:

$$\begin{aligned}
&(x + y)^* && 4 \\
\overset{\mathbf{x}}{\rightarrow} &(\varepsilon + \emptyset) \cdot (x + y)^* && 8 \\
\overset{\mathbf{x}}{\rightarrow} &(\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^* && 17 \\
\overset{\mathbf{x}}{\rightarrow} &(\emptyset + \emptyset) \cdot (x + y)^* + ((\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^*) && 26
\end{aligned} \tag{3.12}$$

The starting expression has four nodes: a star, an alternation and two literals. The first derivative places it behind a concatenation with $(\varepsilon + \emptyset)$, adding a concatenation, an alternation and two units, so $4 + 4 = 8$. The second is an alternation of two eight-node expressions of that shape, so $1 + 8 + 8 = 17$, and the third has the entire second as its right operand, so $1 + 8 + 17 = 26$. Each step from the second onwards adds exactly nine nodes, one alternation and one eight-node summand.

Those nine nodes are dead every time. D_{seq}^n fires at each step because the left factor is nullable, and the summand it adds, $(\emptyset + \emptyset) \cdot (x + y)^*$, can consume nothing, since $\emptyset$ matches no symbol. It derives to itself from then on while a fresh one joins at every step, so only the rightmost summand is ever live.

The identities remove exactly this. S_3 reduces $\varepsilon + \emptyset$ to ε and S_2 reduces $\varepsilon \cdot (x + y)^*$ to $(x + y)^*$, so the first derivative collapses back to the expression it started from. Under “derive by $\mathbf{x}$, then simplify”, $(x + y)^*$ is a fixed point and the size stays at four at every step.

Worked example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$. Recall r_{01} from Section 2.1. It is left-associated as $((0 + 1)^* \cdot 0) \cdot 1$, so at the outer concatenation the rule's s is the trailing literal 1, while r is the whole $(0 + 1)^* \cdot 0$. We derive against $w = 101$ one symbol at a time.

1. **Derive by 1.** At the outer node, $r = (0 + 1)^* \cdot 0$ and $s = 1$. This r is not nullable:

$$n((0 + 1)^* \cdot 0) = \text{true} \wedge \text{false} = \text{false},$$

by N_{seq} , N_* and N_{lit} . Thus D_{seq} applies without a second summand:

$$r_{01} \setminus 1 = (((0 + 1)^* \cdot 0) \setminus 1) \cdot 1.$$

At the middle node, the left operand $(0 + 1)^*$ is nullable, so D_{seq}^n applies. First,

$$\begin{aligned} (0 + 1)^* \setminus 1 &= ((0 + 1) \setminus 1) \cdot (0 + 1)^* && D_* \\ &= ((0 \setminus 1) + (1 \setminus 1)) \cdot (0 + 1)^* && D_+ \\ &= (\emptyset + \varepsilon) \cdot (0 + 1)^* && D_{\text{lit}} \\ &= (0 + 1)^* && S_3, S_2. \end{aligned} \tag{3.13}$$

Hence

$$\begin{aligned} ((0 + 1)^* \cdot 0) \setminus 1 &= (0 + 1)^* \cdot 0 + (0 \setminus 1) && D_{\text{seq}}^n \\ &= (0 + 1)^* \cdot 0 + \emptyset && D_{\text{lit}} \\ &= (0 + 1)^* \cdot 0 && S_3. \end{aligned} \tag{3.14}$$

Substituting back into the outer node gives

$$r_{01} \setminus 1 = ((0 + 1)^* \cdot 0) \cdot 1 = r_{01}. \tag{3.15}$$

Thus a 1 not yet part of the final 01 is absorbed by the leading star.

2. **Derive by 0.** The expression is unchanged. Again, $(0 + 1)^* \setminus 0$ simplifies to $(0 + 1)^*$, but now the second summand at the middle node survives:

$$\begin{aligned} ((0 + 1)^* \cdot 0) \setminus 0 &= (0 + 1)^* \cdot 0 + (0 \setminus 0) && D_{\text{seq}}^n \\ &= (0 + 1)^* \cdot 0 + \varepsilon && D_{\text{lit}}. \end{aligned} \tag{3.16}$$

No simplification applies. The outer node is still non-nullable, so

$$r_{01} \setminus 1 \setminus 0 = ((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1. \tag{3.17}$$

After 10, either the trailing 01 is still owed in full, represented by $(0 + 1)^* \cdot 0$, or the consumed 0 was already its first symbol, represented by ε .

3. **Derive by 1.** The outer binding has now changed. The left operand is $(0 + 1)^* \cdot 0 + \varepsilon$, which is nullable by N_+ and N_ε . So D_{seq}^n applies and contributes the second summand $1 \setminus 1 = \varepsilon$. For the first summand, D_+ applies, reusing the result from step 1:

$$\begin{aligned} ((0 + 1)^* \cdot 0 + \varepsilon) \setminus 1 &= ((0 + 1)^* \cdot 0) \setminus 1 + (\varepsilon \setminus 1) && D_+ \\ &= (0 + 1)^* \cdot 0 + \emptyset && \text{step 1; } D_\varepsilon \\ &= (0 + 1)^* \cdot 0 && S_3. \end{aligned} \tag{3.18}$$

Therefore

$$r_{01} \setminus 1 \setminus 0 \setminus 1 = ((0 + 1)^* \cdot 0) \cdot 1 + \varepsilon = r_{01} + \varepsilon. \tag{3.19}$$

The final expression is nullable by N_+ and N_ε , so $101 \in L(r_{01})$. The ε summand is the witness: it records that the match is complete after the third symbol.

For comparison, consider $w = 10$. The first two steps are the same, so $((0+1)^* \cdot 0 + \varepsilon) \cdot 1$ is the final expression. The first branch $(0+1)^* \cdot 0$ is not nullable, while the second branch $\varepsilon \cdot 1$ still requires the literal 1. Hence the whole expression is not nullable and $10 \notin L(r_{01})$.

`src/regex/nullable.rs` and `src/regex/deriv.rs` implement $n(r)$ and $r \backslash x$ one `Regex` variant at a time, while `match_deriv` folds `deriv` over the input characters, simplifies after each step, and checks `nullable` on the result.

The sequence $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \dots \xrightarrow{x_n} r_n$ can equivalently be read as a path through an automaton built lazily: each r_i is a state and $\xrightarrow{x}$ gives the transition on symbol x . At every step there is exactly one current expression, corresponding to one current state regardless of the size of its syntax tree. This is a DFA-style view of the construction, while the following Section 3.3 contrasts it directly with an NFA-style one.

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 keeps the residual language as a single expression, combining every alternative reached so far with $+$. Antimirov’s partial derivatives take a different route to the same residual language: the alternatives are kept apart as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set,

$$\begin{aligned}
pd(\emptyset, x) &= \{\} & (P_\emptyset) \\
pd(\varepsilon, x) &= \{\} & (P_\varepsilon) \\
pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} & (P_{\text{lit}}) \\
pd(r + s, x) &= pd(r, x) \cup pd(s, x) & (P_+) \\
pd(r^*, x) &= \{ smartC(r', r^*) \mid r' \in pd(r, x) \} & (P_*) \\
pd(r \cdot s, x) &= \begin{cases} \{ smartC(r', s) \mid r' \in pd(r, x) \} \cup pd(s, x) & n(r) = \text{true} \\ \{ smartC(r', s) \mid r' \in pd(r, x) \} & n(r) = \text{false} \end{cases} & \begin{matrix} (P_{\text{seq}}^n) \\ (P_{\text{seq}}) \end{matrix}
\end{aligned} \tag{3.20}$$

Neither $\emptyset$ nor ε has a symbol to consume, so both give the empty set. This is already a difference worth noting: $D_\emptyset$ and D_ε returned the expression $\emptyset$, which denotes the empty language but is still a node in the result, while $P_\emptyset$ and P_ε return no residual at all. For a literal y , consuming x leaves $\{\varepsilon\}$ when $x = y$ and nothing otherwise. For $r + s$ the symbol may come from either alternative, so the two sets are unioned. For r^* the symbol is consumed by one iteration of r , and each residual of that iteration is placed in front of r^* so that the star may continue.

Concatenation depends on nullability for exactly the reason D_{seq} did. If r is not nullable it must consume the x , and each of its residuals is placed in front of an untouched s . If r is nullable it may instead contribute nothing, in which case s consumes the x and $pd(s, x)$ joins the set. The difference from Section 3.2 is that D_{seq}^n joined them with $+$ inside one expression, while P_{seq}^n keeps them as two elements of one set.

The smart constructor. The concatenation in the P_{seq} rules is handled by the constructor smartC , written as **SC**

$$\text{smartC}(r', s) = \begin{cases} s & r' = \varepsilon \\ r' \cdot s & \text{otherwise} \end{cases} \quad (3.21)$$

The constructor accounts for the case where the consumed symbol completes the left-hand expression. If $r' = \varepsilon$, there is nothing left to concatenate with s , so the residual is simply s . Otherwise, the residual r' remains in front of s . Thus smartC expresses the concatenation rule without introducing the redundant expression $\varepsilon \cdot s$.

A set element represents one possible residual rather than one branch of an explicit sum. When several alternatives can consume the same symbol, the alternatives therefore remain as separate elements of the set. Concatenation is then applied to each surviving residual independently. If one of those residuals is ε , that particular strand has finished the left operand and continues directly with s , while the other strands may still retain a non-empty residual in front of s .

Worked example: single-symbol matching. Consider x^* against $\mathbf{xx}$ again, this time under pd . The partial derivative of the literal x with respect to $\mathbf{x}$ is $\{\varepsilon\}$, which the star rule then places in front of x^* using smartC :

$$\begin{aligned} pd(x^*, \mathbf{x}) &= \{ \text{smartC}(r', x^*) \mid r' \in pd(x, \mathbf{x}) \} && P_*, \text{ star body } r = x \\ &= \{ \text{smartC}(r', x^*) \mid r' \in \{\varepsilon\} \} && P_{\text{lit}}, \text{ since } \mathbf{x} = x \\ &= \{ \text{smartC}(\varepsilon, x^*) \} && \\ &= \{x^*\} && \text{SC, } r' = \varepsilon, s = x^*. \end{aligned} \quad (3.22)$$

The result is again $\{x^*\}$, so deriving by the second $\mathbf{x}$ produces the same set:

$$pd(x^*, \mathbf{x}, \mathbf{x}) = \{x^*\}.$$

The remaining residual x^* is nullable, and therefore $\mathbf{xx} \in L(x^*)$. Thus the partial-derivative computation reaches the same result as the Brzozowski derivative computation, while keeping the residual as a set throughout.

Why no simplification is needed. The partial-derivative rules avoid the redundant expressions that required simplification in the Brzozowski computation. In particular, $P_\emptyset$ and P_ε produce no residual at all, while smartC returns s directly when the left residual is ε . Thus the computation never constructs $\emptyset \cdot x^*$ or $\varepsilon \cdot x^*$ in the first place, so the simplification identities of Section 3.2 are not needed after a pd step. The partial-derivative representation therefore removes these redundant cases at the point where residuals are constructed, rather than constructing them first and simplifying them afterwards.

This also illustrates the different roles of simplification in the two constructions. A Brzozowski derivative produces one regular expression that may contain syntactically redundant structure, whereas a partial derivative produces a set whose elements are already the individual residual alternatives. The set itself supplies the separation that an explicit $+$ supplied in the ordinary derivative. The next example shows this distinction in a case where more than one residual survives.

Worked example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$. Derive r_{01} against $w = 101$, tracking the residual set. The concatenations bind as in Section 3.2, with $(r, s) = ((0 + 1)^* \cdot 0, 1)$ at the outer node and $(r, s) = ((0 + 1)^*, 0)$ at the inner node; the star body is $0 + 1$. The derivation proceeds from the inside outward.

1. **Derive by 1.** At the star body, P_+ distributes the derivative over two alternatives:

$$\begin{aligned} pd(0+1, 1) &= pd(0, 1) \cup pd(1, 1) & P_+, r = 0, s = 1 \\ &= \{\} \cup \{\varepsilon\} & P_{lit}, \text{ both branches} \\ &= \{\varepsilon\}. \end{aligned} \quad (3.23)$$

The star rule then applies *smartC* to the resulting residual:

$$\begin{aligned} pd((0 + 1)^*, 1) &= \{ smartC(r', (0 + 1)^*) \mid r' \in \{\varepsilon\} \} & P_*, r = 0 + 1 \\ &= \{(0 + 1)^*\} & SC, r' = \varepsilon. \end{aligned} \quad (3.24)$$

At the middle concatenation, the left operand is nullable by N_* , so the nullable case contributes both the derivatives through the left and right operand:

$$\begin{aligned} pd((0 + 1)^* \cdot 0, 1) &= \{ smartC(r', 0) \mid r' \in pd((0 + 1)^*, 1) \} \cup pd(0, 1) & P_{seq}^n \\ &= \{ smartC((0 + 1)^*, 0) \} \cup \{\} & \text{above; } P_{lit} \\ &= \{(0 + 1)^* \cdot 0\} & SC, r' \neq \varepsilon. \end{aligned} \quad (3.25)$$

At the outer node, the left operand is not nullable, so only one branch contributes:

$$\begin{aligned} pd(r_{01}, 1) &= \{ smartC(r', 1) \mid r' \in \{(0 + 1)^* \cdot 0\} \} & P_{seq}, r = (0 + 1)^* \cdot 0, s = 1 \\ &= \{(0 + 1)^* \cdot 0 \cdot 1\} = \{r_{01}\} & SC. \end{aligned} \quad (3.26)$$

Thus the first symbol leaves a single residual, the r_{01} itself. The computation has returned to the same residual from which it started, representing the possibility that the consumed 1 belonged to the leading star.

2. **Derive by 0.** The current set is still $\{r_{01}\}$. As before,

$$pd((0 + 1)^*, 0) = \{(0 + 1)^*\}.$$

At the middle concatenation, however, the right operand is now the matching literal 0, so P_{lit} contributes $\{\varepsilon\}$:

$$\begin{aligned} pd((0 + 1)^* \cdot 0, 0) &= \{ smartC((0 + 1)^*, 0) \} \cup pd(0, 0) & P_{seq}^n, r = (0 + 1)^*, s = 0 \\ &= \{(0 + 1)^* \cdot 0\} \cup \{\varepsilon\} & P_{lit}, \text{ since } 0 = 0 \\ &= \{(0 + 1)^* \cdot 0, \varepsilon\}. \end{aligned} \quad (3.27)$$

The outer concatenation applies *smartC* separately to both residuals:

$$\begin{aligned} pd(r_{01}, 1, 0) &= \{ smartC(r', 1) \mid r' \in \{(0 + 1)^* \cdot 0, \varepsilon\} \} & P_{seq}, s = 1 \\ &= \{(0 + 1)^* \cdot 0 \cdot 1, 1\} & SC \\ &= \{r_{01}, 1\}. \end{aligned} \quad (3.28)$$

The two residuals represent the possible strands after 10: r_{01} keeps both symbols in the leading star, while 1 means the 0 began the final 01. Since $\text{smartC}(\varepsilon, 1) = 1$, the latter is represented directly as 1.

3. **Derive by 1.** The two residuals are derived independently and their results are unioned:

$$\begin{aligned} pd(r_{01}, 1, 0, 1) &= pd(r_{01}, 1) \cup pd(1, 1) && \text{union over the set} \\ &= \{r_{01}\} \cup \{\varepsilon\} && \text{step 1; } P_{\text{lit}} \\ &= \{r_{01}, \varepsilon\}. \end{aligned} \tag{3.29}$$

The final set contains ε , so one surviving residual is nullable and therefore $101 \in L(r_{01})$. The conclusion matches Equation (3.19).

Also comparing again for $w = 10$ the first two steps are the same, so $\{r_{01}, 1\}$ is the final set. Neither element is nullable: $n(r_{01}) = \text{false}$, and the literal 1 becomes nullable only once it has itself been derived into ε . Hence $10 \notin L(r_{01})$.

Step 2 is where the two constructions are most directly comparable. Its set $\{r_{01}, 1\}$ carries the same information as Equation (3.17)'s $((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1$. Either the trailing 01 is still owed in full, or the consumed 0 was already its first symbol. The derivative folds both readings into one expression with an internal $+$, while pd holds them as two separate residuals with no $+$ at all.

Finiteness. The set of partial derivatives reachable from a fixed r is finite, bounded by $\|r\| + 1$, where $\|r\|$ counts literal occurrences in r [Antimirov, 1996, Theorem 3.4, p. 309]. For r_{01} , $\|r_{01}\| = 4$, so at most five distinct residuals can occur. The example remains well within this bound, reaching only $\{r_{01}\}$, $\{r_{01}, 1\}$, and $\{r_{01}, \varepsilon\}$.

Ordinary derivatives are finite as well, but only after equivalent expressions have been identified [Brzozowski, 1964, Theorem 4.3]. This is the role of the simplifications in Equation (3.10), and the resulting bound is exponential in the structure of r . Antimirov's bound is linear and holds directly at the syntactic level.

The implementation in `match_pderiv` (`src/matchers/match_pderiv.rs`) follows this construction directly. It maintains a `HashSet<Regex>` [The Rust Project Developers, 2026g,e], replaces it at each input symbol by the union of the partial derivatives of its elements, and accepts when the input is exhausted and at least one residual is nullable.

Thus, while the Brzozowski matcher follows one residual expression at a time, the partial-derivative matcher maintains all surviving residuals explicitly. The transition $\{r_{01}\} \rightarrow \{r_{01}, 1\} \rightarrow \{r_{01}, \varepsilon\}$ shows this difference. Each set element represents one possible NFA state, and the set is the collection of states occupied after reading the input. The partial-derivative matcher can therefore be viewed as simulating an NFA directly on the regular expression. Each residual represents a possible NFA state, and the set contains all states that can be active after reading the input.

4 Matching versus Parsing

Chapter 3 established matching with a Boolean answer: whether $w \in L(r)$. A matcher therefore answers only whether the input is accepted by the regular expression. Parsing goes beyond this membership question by identifying the derivation for the match.

This distinction is important because a regular expression can admit more than one way of matching the same word. A parser must therefore represent the choices made during a match. These choices can be represented by *parse trees*, which provide structured witnesses for membership in the language.

4.1 The Matching Problem

Consider the regular expression

$$r = (a + ab)(b + \varepsilon)$$

and the input word

$$w = \mathbf{ab}.$$

The matcher can establish that $w \in L(r)$ using the matching procedures of Chapter 3. There are, however, two ways in which the match can be obtained. In the first, the left subexpression matches **a** and the right subexpression matches **b**:

$$\underbrace{a}_{a+ab} \underbrace{b}_{b+\varepsilon}.$$

In the second, the left subexpression matches the entire input and the right subexpression matches the empty word:

$$\underbrace{\mathbf{ab}}_{a+ab} \underbrace{\varepsilon}_{b+\varepsilon},$$

Both choices therefore establish the same membership result,

$$\mathbf{ab} \in L((a + ab)(b + \varepsilon)),$$

but assign the input differently to the subexpressions of r . The Boolean result of matching does not distinguish these cases.

Parsing makes this additional structure explicit. A parser must represent which alternatives were chosen and which parts of the input were consumed by the corresponding subexpressions. A *parse tree* provides a structured witness for such a derivation, recording the choices made by the expression and the corresponding consumption of the input.

4.2 Parse Trees as Matching Evidence

Sulzmann and Lu view regular expressions as types and parse trees as their inhabitants [Sulzmann and Lu, 2014]. A parse tree v records the structural choices made while matching an expression r .

A judgment $\vdash v : r$ states that v is a valid parse tree for r . The judgment is defined by:

$$\begin{array}{ll}
\vdash () : \varepsilon & (\mathsf{T}_\varepsilon) \\
\frac{l \in \Sigma}{\vdash l : l} & (\mathsf{T}_{\text{lit}}) \\
\frac{\vdash v_1 : r_1 \quad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2} & (\mathsf{T}_{\text{seq}}) \\
\frac{\vdash v_1 : r_1}{\vdash \text{Left } v_1 : r_1 + r_2} & (\mathsf{T}_+^L) \\
\frac{\vdash v_2 : r_2}{\vdash \text{Right } v_2 : r_1 + r_2} & (\mathsf{T}_+^R) \\
\frac{}{\vdash [] : r^*} & (\mathsf{T}_*^0) \\
\frac{\vdash v : r \quad \vdash vs : r^*}{\vdash (v : vs) : r^*} & (\mathsf{T}_*)
\end{array} \tag{4.1}$$

The rules correspond to the constructors of regular expressions. A concatenation uses a pair of parse trees, an alternation records the chosen branch with a tagged injection, and a Kleene star records its iterations as a list.

The typing judgment describes the structure of a match, but a parse tree must also be related to the word that it matches. For this purpose, they define a *flattening* function $|v|$ that maps a parse tree to the word represented by its structure:

$$\begin{aligned}
|()| &= \varepsilon, \\
|l| &= l, \\
|[]| &= \varepsilon, \\
|(v_1, v_2)| &= |v_1||v_2|, \\
|\text{Left } v| &= |v|, \\
|\text{Right } v| &= |v|, \\
|v : vs| &= |v||vs|.
\end{aligned} \tag{4.2}$$

The flattening function connects parse trees to the language of a regular expression:

$$L(r) = \{ |v| \mid \vdash v : r \}. \tag{4.3}$$

Thus, a parse tree v provides a structured *witness* for membership in $L(r)$, recording both the match and the structural choices by which r produces the word.

Unambiguous example. Consider $r = ab$ and $w = \mathbf{ab}$. The sequence rule requires a parse tree for a followed by a parse tree for b . Since each subexpression is a literal, the literal rule gives the only possible derivation:

$$\frac{\frac{a \in \Sigma}{\vdash a : a} (\mathsf{T}_{\text{lit}}) \quad \frac{b \in \Sigma}{\vdash b : b} (\mathsf{T}_{\text{lit}})}{\vdash (a, b) : ab} (\mathsf{T}_{\text{seq}}) \quad (4.4)$$

Thus the unique parse tree is $v = (a, b)$. Its flattening recovers the input word:

$$|(a, b)| = |a| |b| = \mathbf{ab} \quad (4.5)$$

When the structure of the expression permits different choices, a word may have multiple parse trees. The first two examples are adapted from the running examples of [Sulzmann and Lu \[2014\]](#), but are developed here in more detail to establish the derivations used later in the discussion of the parser implementations.

Ambiguous example 1: concatenation. The example from the introduction can now be revisited using parse trees. For $r_1 = (a + ab)(b + \varepsilon)$ and $w = \mathbf{ab}$, there are two possible matches, corresponding to two distinct parse trees.

The first consumes $\mathbf{ab}$ with the second alternative of $a + ab$ and then chooses ε :

$$\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : a + ab} \quad \frac{\vdash () : \varepsilon}{\vdash \text{Right}() : b + \varepsilon} \Rightarrow \vdash (\text{Right}(a, b), \text{Right}()) : (a + ab)(b + \varepsilon). \quad (4.6)$$

Thus,

$$v = (\text{Right}(a, b), \text{Right}()), \quad |v| = \mathbf{ab}. \quad (4.7)$$

The second consumes $\mathbf{a}$ with the first alternative of $a + ab$ and $\mathbf{b}$ with the first alternative of $b + \varepsilon$:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + ab} \quad \frac{\vdash b : b}{\vdash \text{Left } b : b + \varepsilon} \Rightarrow \vdash (\text{Left } a, \text{Left } b) : (a + ab)(b + \varepsilon). \quad (4.8)$$

Thus,

$$v' = (\text{Left } a, \text{Left } b), \quad |v'| = \mathbf{ab}. \quad (4.9)$$

Hence the same expression and word have two distinct parse trees, $v \neq v'$, both of which flatten to $\mathbf{ab}$.

Ambiguous example 2: repetition. The same example from [Sulzmann and Lu \[2014\]](#) also illustrates ambiguity arising from repetition. Consider $r_2 = (a + b + ab)^*$ against $w = \mathbf{ab}$, with the alternation nested as $a + (b + ab)$. There are two possible matches: one iteration can consume the entire word using the ab alternative, or two iterations can consume $\mathbf{a}$ and $\mathbf{b}$ separately.

The first match is given by

$$\frac{\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : b + ab}}{\vdash \text{Right } \text{Right}(a, b) : a + (b + ab)} \Rightarrow \vdash [\text{Right } \text{Right}(a, b)] : (a + (b + ab))^*. \quad (4.10)$$

Thus,

$$v_2 = [\text{Right } \text{Right}(a, b)], \quad |v_2| = \mathbf{ab}. \quad (4.11)$$

The second match uses two iterations, one for each character:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + (b + ab)} \quad \frac{\frac{\vdash b : b}{\vdash \text{Left } b : b + ab}}{\vdash \text{Right } \text{Left } b : a + (b + ab)} \Rightarrow \vdash [\text{Left } a, \text{Right } \text{Left } b] : (a + (b + ab))^*. \quad (4.12)$$

Thus,

$$v'_2 = [\text{Left } a, \text{Right } \text{Left } b], \quad |v'_2| = \mathbf{ab}. \quad (4.13)$$

Hence $v_2 \neq v'_2$ are two distinct parse trees for the same expression and word. Here the ambiguity comes from the number of iterations of the Kleene star.

Ambiguous example 3: duplicate alternatives. A different form of ambiguity arises when an alternation contains the same alternative twice. Consider $r_3 = (a + a)^*$ against $w = \mathbf{aa}$. Each iteration can choose either copy of a , so each occurrence of $\mathbf{a}$ gives two choices:

$$\begin{aligned} v_3^{LL} &= [\text{Left } a, \text{Left } a], & v_3^{LR} &= [\text{Left } a, \text{Right } a], \\ v_3^{RL} &= [\text{Right } a, \text{Left } a], & v_3^{RR} &= [\text{Right } a, \text{Right } a]. \end{aligned} \quad (4.14)$$

Each is derivable using $\top_+^L$ or $\top_+^R$ for each iteration and $\top_*$ to form the list, and all four flatten to the same word:

$$|v_3^{LL}| = |v_3^{LR}| = |v_3^{RL}| = |v_3^{RR}| = \mathbf{aa}. \quad (4.15)$$

Thus the four parse trees differ only in which syntactically distinct copy of a is selected. The ambiguity is therefore caused by duplicate alternatives rather than by different ways of partitioning the input.

Ambiguous example 4: nested repetition. A further source of ambiguity is nested repetition. Consider $r_4 = (a^*)^*$ against $w = \mathbf{aa}$. The word can be matched by a single outer iteration whose inner repetition consumes both characters, or by two outer iterations, each containing one inner iteration.

The first match uses one outer iteration and two inner iterations:

$$\vdash [[a, a]] : (a^*)^*, \quad (4.16)$$

with parse tree

$$v_4 = [[a, a]], \quad |v_4| = \mathbf{aa}. \quad (4.17)$$

The second uses two outer iterations, each containing one inner iteration:

$$\vdash [[a], [a]] : (a^*)^*, \quad (4.18)$$

with parse tree

$$v'_4 = [[a], [a]], \quad |v'_4| = \mathbf{aa}. \quad (4.19)$$

Thus $v_4 \neq v'_4$ are two distinct parse trees for the same expression and word. The ambiguity here arises from the interaction of nested repetitions: the same input can be partitioned differently between the iterations of the inner and outer Kleene stars.

4.3 Disambiguation Policies

The parse-tree judgment identifies all valid witnesses for a match, but does not determine which witness a parser should return. The last two examples also show that not every ambiguity needs to be resolved by the parser itself. Duplicate alternatives, as in r_3 , can be removed by simplifying the expression, while nested repetition, as in r_4 , can likewise be rewritten without changing the language. However, the first two examples are different. Their parse trees represent genuinely different ways of matching the same input, so simplifying the expression does not by itself determine which match should be selected.

This raises the question of how a parser should choose between such valid matches. A *disambiguation policy* to make this choice. Given a regular expression r and word w , it selects one parse tree v satisfying

$$\vdash v : r \quad \text{and} \quad |v| = w. \quad (4.20)$$

The two policies considered here make this choice according to different principles. Greedy matching follows the structural priority of the expression, whereas POSIX matching gives priority to longer leftmost submatches. Their difference becomes clear when they are applied to the same ambiguous expression.

4.3.1 Greedy Leftmost Matching

The policy associated with conventional backtracking matchers is *Greedy*: alternatives are considered from left to right, and the first one leading to an overall match is selected; within a Kleene star, another iteration is preferred whenever it can still lead to an overall match [Frisch and Cardelli, 2004].

Both preferences are structural, because they are decided by the position of a subexpression in r , and never by how much input a candidate consumes.

Applied to r_1 . For $r_1 = (a + ab)(b + \varepsilon)$ and $w = \mathbf{ab}$, Greedy selects the left alternative of $a + ab$, consuming $\mathbf{a}$, followed by the left alternative of $b + \varepsilon$, consuming $\mathbf{b}$:

$$v_1^G = (\textit{Left } a, \textit{Left } b). \quad (4.21)$$

Applied to r_2 . For $r_2 = (a + (b + ab))^*$ and $w = \mathbf{ab}$, Greedy first consumes $\mathbf{a}$ with the left alternative, then continues the star and matches $\mathbf{b}$ with its second alternative:

$$v_2^G = [\textit{Left } a, \textit{Right Left } b]. \quad (4.22)$$

Applied to r_3 . For $r_3 = (a + a)^*$ and $w = \mathbf{aa}$, both alternatives consume one character, so the left alternative is selected at each iteration:

$$v_3^G = [\textit{Left } a, \textit{Left } a]. \quad (4.23)$$

Applied to r_4 . For $r_4 = (a^*)^*$ and $w = \mathbf{aa}$, the preference for continuing a repetition applies first to the inner star, since that is the star the first character is consumed by. Having consumed $\mathbf{a}$, the inner star takes a second iteration instead of closing its list, so the first outer iteration consumes both characters:

$$v_4^G = [[a, a]]. \quad (4.24)$$

The alternative $[[a], [a]]$ closes the inner star after one character and opens a second outer iteration. Greedy reaches the outer star only with the input the inner star has declined, so this tree is not selected.

Procedural versus declarative view. The description above gives a procedural intuition for Greedy matching. Alternatives are considered from left to right, and repetition continues when doing so can still lead to a complete match. The same policy can be stated declaratively as an ordering on the parse trees admitted by r and w , with the selected tree being the maximal one [Frisch and Cardelli, 2004]. This declarative view is useful for comparing Greedy with other disambiguation policies because it makes the preferred parse tree an explicit object of comparison.

These two views describe the same policy but are not the same computationally. Frisch and Cardelli formulate the ordering independently of any particular matching algorithm and show how it can be computed without backtracking.

Written as inference rules, the Greedy ordering $v_1 \succ_r v_2$ is:

$$\begin{aligned}
\text{(C1)} \quad & \frac{v_1 = v'_1 \quad v_2 \succ_{r_2} v'_2}{(v_1, v_2) \succ_{r_1 r_2} (v'_1, v'_2)} \\
\text{(C2)} \quad & \frac{v_1 \succ_{r_1} v'_1}{(v_1, v_2) \succ_{r_1 r_2} (v'_1, v'_2)} \\
\text{(G1)} \quad & \frac{}{\text{Left } v_1 \succ_{r_1+r_2} \text{Right } v_2} \\
\text{(A3)} \quad & \frac{v_2 \succ_{r_2} v'_2}{\text{Right } v_2 \succ_{r_1+r_2} \text{Right } v'_2} \\
\text{(A4)} \quad & \frac{v_1 \succ_{r_1} v'_1}{\text{Left } v_1 \succ_{r_1+r_2} \text{Left } v'_1} \\
\text{(G2)} \quad & \frac{}{v : vS \succ_{r^*} []} \\
\text{(K3)} \quad & \frac{v_1 \succ_r v_2}{v_1 : vS_1 \succ_{r^*} v_2 : vS_2} \\
\text{(K4)} \quad & \frac{v_1 = v_2 \quad vS_1 \succ_{r^*} vS_2}{v_1 : vS_1 \succ_{r^*} v_2 : vS_2}
\end{aligned} \tag{4.25}$$

Rules (C1) and (C2) propagate comparisons through concatenation. Rule (C2) gives priority to the first component whenever it differs, and (C1) compares the second component when the first components are equal, so earlier subexpressions have priority over later ones. Rules (G1) and (G2) are the two structural preferences of the procedural description: (G1) prefers the left alternative outright, and (G2) prefers any non-empty list of iterations to stopping. Neither has a premise, so neither consults what the competing trees consume. Rules (A3), (A4), (K3) and (K4) are congruence rules: they carry a comparison into a position where both candidates have already made the same choice, first comparing the heads of two iteration lists and then their tails.

For example, consider the two parse trees

$$v = (\text{Left } a, \text{Left } b), \quad v' = (\text{Right}(a, b), \text{Right}())$$

for $r_1 = (a + ab)(b + \varepsilon)$. Since the first components differ, (C2) reduces the comparison to

$$\text{Left } a \quad \text{versus} \quad \text{Right}(a, b)$$

within $a + ab$. Rule (G1) immediately gives

$$\text{Left } a \succ_{a+ab} \text{Right}(a, b),$$

without comparing the lengths of the two matches. Thus the structural priority of the left alternative determines the result before the second component is considered.

This distinction is relevant for the parser implementations considered later. The partial-derivative parsers do not enumerate all parse trees and then select a maximal one, but compute a kind of Greedy result directly. This also matters when a maximal parse tree does not exist, as the following example demonstrates.

4.3.2 The Infinite Empty-Match Chain Problem

Greedy's preference for continuing a repetition creates a problem when the repeated expression can match the empty word. Consider ε^* against the empty input. The following parse trees are all valid:

$$v_0 = [], \quad v_1 = [()], \quad v_2 = [(), ()], \quad \dots \quad (4.26)$$

Each additional iteration still flattens to ε , and Greedy prefers every additional iteration:

$$v_0 \prec_{\varepsilon^*} v_1 \prec_{\varepsilon^*} v_2 \prec_{\varepsilon^*} \dots \quad (4.27)$$

There is therefore no maximal parse tree. The language itself is not problematic: ε is a valid match. The problem is that the Greedy ordering does not identify a largest witness when empty iterations can be added indefinitely.

A less degenerate example is $(\varepsilon + a)^*$ against **a**. The parse trees

$$\begin{aligned} v_0 &= [Right\ a], \\ v_1 &= [Left\ (), Right\ a], \\ v_2 &= [Left\ (), Left\ (), Right\ a], \\ &\vdots \end{aligned} \quad (4.28)$$

all flatten to **a**, while Greedy again prefers each additional empty iteration.

The same phenomenon occurs whenever a star's body is nullable. For $(a^*)^*$ against **aaa**, for example, the inner star can match ε , so additional empty outer iterations can always be added:

$$[[a, a, a]] \prec_{(a^*)^*} [[a, a, a], []] \prec_{(a^*)^*} [[a, a, a], [], []] \prec_{(a^*)^*} \dots \quad (4.29)$$

Thus the Greedy ordering may contain an infinite ascending chain without a maximal element. Engines that search directly for such a maximal match can encounter the familiar difficulties of nested nullable repetition [Cox, 2007]. The frontier-based Greedy parsers of Chapter 6, by contrast, compute the match without searching for an ordering maximum.

Two premise-free rules of Equation (4.25) produce these chains. (G2) prefers a non-empty iteration list to the empty one without considering what it consumes, so an iteration flattening to ε can always be added again. In the second example, (G1) instead prefers the *Left* $()$ branch of $\varepsilon + a$ regardless of length, so (K3) ranks each tree above the one with one fewer empty iteration. In both cases, the deciding rule has no premise and therefore cannot detect that the added iteration contributes nothing.

This exposes a limitation of the Greedy ordering: on such an expression a maximal parse tree does not exist, so the ordering alone does not specify an answer. Frisch and Cardelli resolve this by restricting the specification to parse trees in which no iteration flattens to ε , and show that such a tree exists whenever the word is matched at all [Frisch and Cardelli, 2004]. The POSIX ordering instead settles these comparisons within the rules themselves, as Section 4.3.3 shows.

4.3.3 POSIX Longest-Leftmost Matching

POSIX uses a different disambiguation policy. Subpatterns are compared according to the length of the substrings they match, with subpatterns that occur earlier in the regular expression taking priority. Section 9.1 of the POSIX Base Definitions states that, consistent with the whole match being the longest of the leftmost matches, each subpattern, from left to right, shall match the longest possible string, and that for this purpose a null string is considered longer than no match at all [IEEE and The Open Group, 2017].

Sulzmann and Lu specify this policy by reusing the Greedy ordering of Equation (4.25) and substituting for its two structural preferences [Sulzmann and Lu, 2014]. The congruence rules (A3), (A4), (K3) and (K4) and the concatenation rules (C1) and (C2) are kept unchanged. In their place, (G1) becomes the length-sensitive pair

$$\begin{aligned} \text{(A1)} \quad & \frac{\text{len}(|v_2|) > \text{len}(|v_1|)}{\text{Right } v_2 \succ_{r_1+r_2} \text{Left } v_1} \\ \text{(A2)} \quad & \frac{\text{len}(|v_1|) \geq \text{len}(|v_2|)}{\text{Left } v_1 \succ_{r_1+r_2} \text{Right } v_2} \end{aligned} \tag{4.30}$$

and (G2) becomes the pair

$$\begin{aligned} \text{(K1)} \quad & \frac{|v : vs| = \varepsilon}{[] \succ_{r^*} v : vs} \\ \text{(K2)} \quad & \frac{|v : vs| \neq \varepsilon}{v : vs \succ_{r^*} []}. \end{aligned} \tag{4.31}$$

Both substitutions replace a premise-free rule with a pair of rules whose premises read a flattening, which is what makes the ordering length-sensitive. Rule (A1) prefers the branch consuming more input, and (A2) breaks an equal-length tie in favor of the left branch, so the left branch still wins whenever length does not decide. Rule (K2) agrees with (G2) whenever the iterations consume something, so the two policies differ on repetition only when a list of iterations flattens to ε , and there (K1) reverses (G2) by preferring the empty list. Between them, (A1) and (K1) close every chain of Section 4.3.2: (K1) settles the comparisons that (G2) drove, and (A1) settles the one that (G1) drove, since an empty iteration consumes less than a non-empty one. No restriction on the admissible parse trees is needed.

The ordering compares only parse trees that flatten to the same word. Sulzmann and Lu make this explicit in the associated non-strict relation,

$$v_1 \succeq_r v_2 \iff v_1 = v_2 \text{ or } (v_1 \succ_r v_2 \text{ and } |v_1| = |v_2|), \tag{4.32}$$

and call v_1 the POSIX parse tree for r when $\vdash v_1 : r$ and $v_1 \succeq_r v_2$ holds for every v_2 with $\vdash v_2 : r$ and $|v_2| = |v_1|$. The length restriction matters because $\succ_r$ by itself relates trees with different flattenings: (A1) compares two branches by length, so it would rank a tree consuming more input above one consuming less even when the shorter tree is the only one matching w . Equation (4.32) confines the comparison to the candidates that are answers to the same parsing problem.

Applied to r_1 . For $r_1 = (a + ab)(b + \varepsilon)$ and $w = \mathbf{ab}$, consider

$$v = (\textit{Right}(a, b), \textit{Right}()), \quad v' = (\textit{Left } a, \textit{Left } b). \quad (4.33)$$

By (C2), the comparison reduces to the first components. The right alternative consumes two characters and the left alternative one, so (A1) gives

$$\textit{Right}(a, b) \succ_{a+ab} \textit{Left } a. \quad (4.34)$$

Thus POSIX selects

$$v_1^P = (\textit{Right}(a, b), \textit{Right}()), \quad (4.35)$$

the opposite of the Greedy result.

Applied to r_2 . For $r_2 = (a + (b + ab))^*$ and $w = \mathbf{ab}$, consider

$$v_2 = [\textit{Right } \textit{Right}(a, b)], \quad v'_2 = [\textit{Left } a, \textit{Right } \textit{Left } b]. \quad (4.36)$$

By (K3), the comparison is determined by the first iterations. The right alternative consumes two characters while the left consumes one, so (A1) gives

$$\textit{Right } \textit{Right}(a, b) \succ_{a+(b+ab)} \textit{Left } a. \quad (4.37)$$

Hence POSIX selects the single two-character iteration,

$$v_2^P = [\textit{Right } \textit{Right}(a, b)], \quad (4.38)$$

whereas Greedy selects two one-character iterations.

Applied to r_3 . For $r_3 = (a + a)^*$ and $w = \mathbf{aa}$, both alternatives consume exactly one character. Thus (A1) does not apply, and (A2) breaks the tie in favor of the left alternative:

$$\textit{Left } a \succ_{a+a} \textit{Right } a. \quad (4.39)$$

Applying (K3)–(K4) to both positions gives

$$v_3^P = [\textit{Left } a, \textit{Left } a]. \quad (4.40)$$

POSIX therefore agrees with Greedy: the alternatives have equal length, so the POSIX tie-break coincides with Greedy's left preference.

Applied to r_4 . For $r_4 = (a^*)^*$ and $w = \mathbf{aa}$, consider

$$[[a, a]] \quad \text{and} \quad [[a], [a]]. \quad (4.41)$$

The first outer iterations are $[a, a]$ and $[a]$, which differ, so (K4) does not apply and the comparison is made by (K3) between these two iterations under $\succ_{a^*}$. Both are non-empty lists whose first elements are the literal a , so (K3) does not apply at this inner level either, and (K4) passes the comparison to the tails $[a]$ and $[]$. Since $[[a]] = \mathbf{a} \neq \varepsilon$, rule (K2) gives

$$[a] \succ_{a^*} [], \quad (4.42)$$

so (K4) yields $[a, a] \succ_{a^*} [a]$, and (K3) lifts this to the outer star:

$$[[a, a]] \succ_{(a^*)^*} [[a], [a]]. \quad (4.43)$$

Thus POSIX selects

$$v_4^P = [[a, a]], \quad (4.44)$$

the same parse tree as Greedy. The agreement has a different source from the one on r_3 . There the two policies coincided because an alternation tied in length, so (A2) reproduced (G1). Here no alternation rule is consulted at all: the comparison uses (K2), (K3) and (K4), and the only step where the two policies could diverge is the appeal to (K2), which agrees with (G2) because $[a]$ does not flatten to ε .

Applied to the infinite-chain examples. The repetition rules also resolve the empty-match chains of Section 4.3.2. For ε^* against ε , (K1) gives

$$[] \succ_{\varepsilon^*} [()], \quad (4.45)$$

and therefore prefers the empty iteration list to every list of empty iterations.

For $(\varepsilon + a)^*$ against $\mathbf{a}$, (K3) reduces the comparison between

$$[Right\ a] \quad \text{and} \quad [Left\ (), Right\ a] \quad (4.46)$$

to their first iterations. Since $1 > 0$, (A1) gives

$$Right\ a \succ_{\varepsilon+a} Left\ (), \quad (4.47)$$

so the tree without the additional empty iteration is preferred.

Likewise, for $(a^*)^*$ against $\mathbf{aaa}$, (K4) reduces

$$[[a, a, a]] \quad \text{and} \quad [[a, a, a], []] \quad (4.48)$$

to their tails, where (K1) gives

$$[[a, a, a]] \succ_{(a^*)^*} [[a, a, a], []]. \quad (4.49)$$

Thus POSIX selects $[[a, a, a]]$ rather than extending the match with empty outer iterations. Unlike the Greedy ordering, which admits the infinite chains described in Section 4.3.2, the POSIX rules explicitly settle these empty-match comparisons.

The contrast between the two policies is therefore clear: Greedy gives structural priority to earlier alternatives and prefers continuing repetition, whereas POSIX prefers longer leftmost submatches and uses the structural order to break ties.

5 Derivative-Based Parser

Chapter 4 fixed what is being computed, the $\geq_r$ -maximal parse tree of Section 4.3.3, without saying how. This chapter presents Sulzmann and Lu’s answer [Sulzmann and Lu, 2014]: a parser built directly on the Brzozowski derivative of Section 3.2, in two forms. The first, direct construction computes a derivative sequence and then explicitly rebuilds a parse tree from it in a second, backward pass. The second, bit-coded construction fuses parse-tree bookkeeping into the derivative step itself, needing only one forward pass. Both are POSIX, computing the identical $\geq_r$ -maximal tree, and differ only in how that tree is produced.

Both are developed here as theory first (Sections 5.2 to 5.4), then as the two Rust implementations that realize them, `deriv_std` (Section 5.5) and `deriv_bc` (Section 5.5.1), and are finally set against each other in Section 5.6. Throughout, the implementation’s own execution traces are used as the primary evidence for what each construction does, so Section 5.1 introduces the tracing facility first. Chapter 6 then repeats the exercise once more, replacing the Brzozowski derivative throughout with Antimirov’s partial derivative from Section 3.3, and, as that chapter shows, changing more than just which derivative operator is used.

5.1 Reading the Implementation’s Traces

Each of the four parsers in this thesis exists in two forms: a plain one and a *traced* one that records every step it takes. The traced form is not a debugging afterthought bolted on afterwards. It computes the same result by the same code path and additionally emits, at each step, the rule it applied and the intermediate value it produced, which makes it possible to check a construction against its defining equations by running it rather than by reading it.

The facility is exposed through `-diag`, which selects one of four verbosity levels:

Level	Adds	Destination
<code>-diag 0</code>	<code>true/false</code> only	<code>stdout</code>
<code>-diag 1</code>	regex, input, tree, error caret	<code>stdout</code>
<code>-diag 2</code>	timing, step count, construction steps	<code>stdout</code>
<code>-diag 3</code>	full derivation trace	<code>reports/report.txt</code>

Level 3 writes to a file rather than the terminal because its output grows with both the input length and the size of the intermediate expressions, and Section 5.4 shows how quickly the latter can grow. `-diag-report` overrides the destination; the environment variables `REGEX_DIAG` and `REGEX_DIAG_REPORT` set the same two values for library use.

How traces are used in this chapter. Rather than reproducing whole reports, the sections that follow quote *excerpts*, each placed next to the construction step it illustrates. A level-3 report is organized into labelled sections, and those sections map onto the algorithm's own phases closely enough that individual fragments stand on their own:

Report section	Construction phase
FORWARD PASS	the derivative sequence
NULLABILITY CHECK	the accept/reject decision
BACKWARD PASS	<i>mkEps</i> and <i>inj</i> , direct construction
INTERNALIZE, MKEEPSBC + DECODE	the bit-coded construction's ends
PARTIAL RECOVERY, ERROR SUMMARY	failure diagnostics

All eight reports quoted from in this chapter and the next are reproduced in full in Appendix C, so that any excerpt can be read back in its original context. They all use the same running example, $r_1 = (a + ab)(b + \varepsilon)$ from Section 4.3, against the matching input "ab" and the non-matching input "b". In the frontend's surface syntax (Section 7.2) that expression is written "(a|ab)(b|)", with ε spelled as an empty right alternative, so the command producing every trace in this chapter is

```
cargo run -- parse "(a|ab)(b|)" "ab" --parser <PARSER> --diag 3
```

Excerpts are shown exactly as the tool writes them, except that the tool's Unicode operators are transliterated into ASCII (∂ as **d**, $\cdot$ as **.**, $\oplus$ as **(+)**, ε as **epsilon**, ϕ as **phi**, and $\checkmark$ as **[OK]**), and long lines are wrapped.

5.2 Core Algorithm

The direct construction follows the same two-phase shape informally sketched for Brzowski's derivative in Section 3.2: consume the input by taking successive derivatives, then decide whether the result is nullable. What is new here is what happens *between* those two phases. Instead of a single Boolean, a parse tree is built. The construction is best understood in two halves, shown in Figure 5.1: a forward pass that is nothing more than the derivative sequence already introduced, and a backward pass that walks that sequence in reverse to assemble a tree one consumed character at a time.

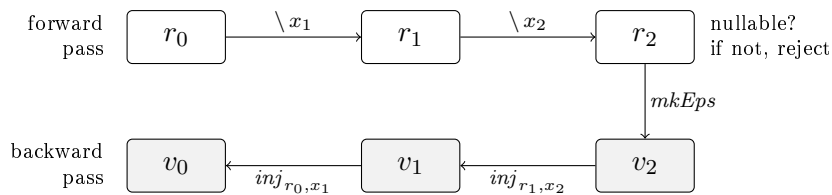


Figure 5.1: The direct construction on a two-character input. The forward pass takes one derivative per character; the backward pass turns the empty-word tree v_2 back into a tree of the original r_0 , undoing one derivative per step. The whole forward sequence must be retained until the backward pass consumes it.

Forward pass. For input $x_1x_2 \cdots x_n$, compute $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ exactly as before, where $r_0 = r$ and $r_i = r_{i-1} \setminus x_i$. If r_n is not nullable, no parse tree exists and parsing fails. This half of the construction is identical to `match_deriv` (Section 3.2) with one difference:

no simplification is applied between steps. `simplify` is safe for a Boolean matcher, since it only ever normalizes r_i to a language-equivalent expression, but the backward pass below needs to walk back through the *exact* sequence of derivative transformations that produced r_n from r_0 , and a simplified r_i may no longer be the expression `inject` expects to see at that step.

The trace makes both halves of that claim visible at once. Each forward step names the derivative rule it applied and prints the resulting expression in full:

```
Step 1: d(r0, 'a')
  rule: d(r1.r2, 'a') [nullable(r1)=false]
        = d(r1, 'a').r2
r1 = Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = true [OK] position 1 matched
```

The rule line records which branch of the $(r_1 r_2) \setminus x$ case split was taken and why, and `r1` is printed unsimplified: the `Eps` inside `Seq(Eps, Lit('b'))` is exactly the kind of node `simplify` would have removed, and it is retained because `inject` will need it.

Backward pass. Once r_n is known to be nullable, the parse tree v_n for the empty remainder is built directly, by the function `mkEps` of Equation (5.1), defined by structural recursion on a nullable expression:

$$\begin{aligned}
mkEps_\varepsilon &= () & (E_\varepsilon) \\
mkEps_{r^*} &= [] & (E_*) \\
mkEps_{r_1 r_2} &= (mkEps_{r_1}, mkEps_{r_2}) & (E_{\text{seq}}) \\
mkEps_{r_1 + r_2} &= \begin{cases} \text{Left } mkEps_{r_1} & \varepsilon \in L(r_1) & (E_+^L) \\ \text{Right } mkEps_{r_2} & \text{otherwise} & (E_+^R) \end{cases} & (5.1)
\end{aligned}$$

The rule names continue the scheme of Section 3.2, and derivations below cite them with the binding of r_1 and r_2 stated, since which subexpression each stands for is what decides the case taken. `mkEps` has no case for a literal x or for $\emptyset$, since neither is ever nullable. The function is only ever called on an expression already known to satisfy $n(r)$. Its only interesting case is alternation, and exactly one of rules (A1)/(A2) of Section 4.3.3 is relevant there: both sides of an empty match have length 0, so (A2)'s tie-break, "prefer left when lengths are equal", degenerates at the empty word to "prefer left whenever left is even possible", which is exactly the $\varepsilon \in L(r_1)$ guard above.

That guard is the single most consequential line in the whole backward pass, and the trace prints it firing:

```
mkEps(r2):
  mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))
v2 = Left (Right Right (), Right ())
```

Matching `r2` against `mkEpsr1+r2` binds r_1 to its long `Seq(...)` disjunct and r_2 to `Alt(Eps, Phi)`, and at this point *both* are nullable. E_+^L and E_+^R are therefore not both applicable: the case split tests only $\varepsilon \in L(r_1)$, so E_+^L fires and r_2 is never examined. `mkEps` does not compare the two; it takes the left one because it is nullable at all. Everything the backward pass produces downstream follows from that choice, and Sulzmann and Lu prove that making it this way is the POSIX-correct one [Sulzmann and Lu, 2014].

With v_n in hand, the remaining task is to turn it back into a parse tree of the *original* r_0 , one derivative step in reverse at a time. This is the job of *inj*, an injection function that takes a parse tree of a derivative $r \setminus x$ and re-inserts the consumed character x to produce a parse tree of r itself, Equation (5.2):

$$\begin{aligned}
inj_{r^* \setminus x}(v, vs) &= (inj_{r \setminus x} v) : vs & (I_*) \\
inj_{(r_1 r_2) \setminus x}(v) &= \begin{cases} (inj_{r_1 \setminus x} v_1, v_2) & v = (v_1, v_2), \text{ not nullable}(r_1) & (I_{\text{seq}}) \\ (inj_{r_1 \setminus x} v_1, v_2) & v = \text{Left}(v_1, v_2), \text{ nullable}(r_1) & (I_{\text{seq}}^L) \\ (mkEps_{r_1}, inj_{r_2 \setminus x} v_2) & v = \text{Right } v_2, \text{ nullable}(r_1) & (I_{\text{seq}}^R) \end{cases} & (5.2) \\
inj_{(r_1 + r_2) \setminus x}(v) &= \begin{cases} \text{Left } (inj_{r_1 \setminus x} v_1) & v = \text{Left } v_1 & (I_+^L) \\ \text{Right } (inj_{r_2 \setminus x} v_2) & v = \text{Right } v_2 & (I_+^R) \end{cases} \\
inj_{x \setminus x}() &= x & (I_{\text{lit}})
\end{aligned}$$

The three-way $(r_1 r_2)$ case is worth reading against the derivative rule it inverts (Section 3.2): $(r_1 r_2) \setminus x$ produced either $(r_1 \setminus x) r_2$ alone (when r_1 is not nullable) or that same term joined with $r_2 \setminus x$ via $+$ (when it is), tagged *Left/Right* respectively by the *Alt* this new $+$ introduces. *inj* undoes exactly that tagging. A bare pair means r_1 was not nullable and the derivative went straight into it; a *Left*-tagged pair means it was nullable but the derivative still chose to continue matching r_1 ; a *Right*-tagged value means the derivative instead jumped straight into r_2 , so r_1 's own contribution to the reconstructed pair has to be synthesized fresh via $mkEps_{r_1}$, since no part of it was consumed by x at all.

Parsing is then this two-phase process packaged into one function, applying *mkEps* once at the end of the forward pass and *inj* once per character on the way back:

$$\begin{aligned}
parse(r, \varepsilon) &= mkEps_r & (\text{if } \varepsilon \in L(r)) \\
parse(r, xw) &= inj_{r \setminus x}(parse(r \setminus x, w)) & (5.3)
\end{aligned}$$

The worked example, one step at a time. Return to $r_1 = (a + ab)(b + \varepsilon)$ against **ab**, whose two candidate parse trees were already established in Section 4.3. The forward pass computes

$$\begin{aligned}
r_1 &\xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) & D_{\text{seq}}, r = a + ab, s = b + \varepsilon, n(r) = \text{false} \\
&\xrightarrow{b} (\phi + (\phi b + \varepsilon))(b + \varepsilon) + (\varepsilon + \phi) & D_{\text{seq}}^n, r = \varepsilon + \varepsilon b, n(r) = \text{true}
\end{aligned} \tag{5.4}$$

The two steps take different cases of the same rule, and the reason is the binding: after one character the left operand has become $\varepsilon + \varepsilon b$, which is nullable by N_+ and N_ε , where the original $a + ab$ was not. That is what adds the second summand $(\varepsilon + \phi)$, and it is that summand which makes the whole expression nullable. The trace's second forward step confirms the rule taken and shows the expression that the hand derivation abbreviates:

```

Step 2: d(r1, 'b')
  rule: d(r1.r2, 'b') [nullable(r1)=true]
        = d(r1, 'b').r2 + d(r2, 'b')
r2 = Alt(Seq(Alt(Phi, Alt(Seq(Phi, Lit('b'))), Eps)), Alt(Lit('b'),
  Eps)), Alt(Eps, Phi))
nullable(r2) = true [OK] position 2 matched

```

This is the nullable- r_1 branch, the one that introduces a fresh *Alt*, and it is that new *Alt* whose two branches *mkEps* then chooses between. From v_2 the backward pass runs two injections, each printed with the expression it injects into and the value it produces:

```
inject(r1, 'b', v2):
  -> (Right (), b), Right ()
v1 = (Right (), b), Right ()

inject(r0, 'a', v1):
  -> (Right (a, b), Right ())
v0 = (Right (a, b), Right ())
```

Reading the two lines together shows *inj* doing its characteristic work. At the first step the tree still contains $()$, an empty-word placeholder where the literal **a** has not yet been reinserted; at the second, the *Lit* case replaces it with **a**, and the result is a complete tree of r_1 itself. The answer, $(\text{Right}(a, b), \text{Right}())$, selects the *longer* *ab* branch, matching the POSIX result claimed in Section 4.3.3 rather than Greedy's $(\text{Left } a, \text{Left } b)$.

The report's header quantifies the cost of the two-pass shape in one line:

Expressions stored: 3 ($r_0 \rightarrow r_1 \rightarrow r_2$)

one expression per input character plus the original, all of them live until the backward pass has consumed them. That is exactly the requirement Figure 5.1 draws, and exactly the one Section 5.3 removes.

5.3 Bit-Coded Optimization (Theory)

The direct construction of Section 5.2 must retain the whole forward-pass sequence, every r_i from r_0 to r_n , or in an implementation an equivalent call stack, so that *inj* has each $r_{i-1} \setminus x_i$ available to reconstruct against on the way back: the backward pass cannot begin until the forward pass has fully finished. Sulzmann and Lu's bit-coded construction (their Section 3.2, "Incremental Bit-Coded Forward Parse Tree Construction") [Sulzmann and Lu, 2014] removes this requirement by incrementally building the parse tree *during* the forward pass itself, so that by the time the last character is consumed, the winning tree's information is already sitting in the final expression, with no second pass required. Figure 5.2 gives the resulting shape, which is Figure 5.1 with the return journey removed.

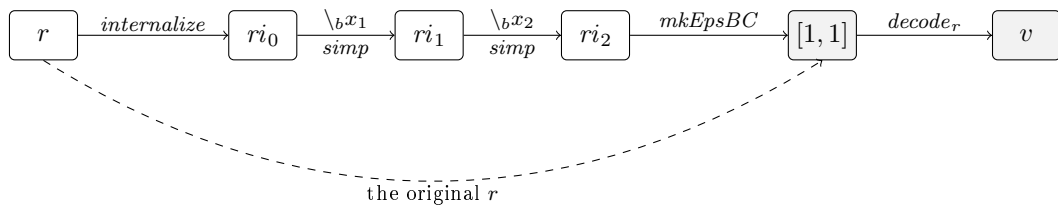


Figure 5.2: The bit-coded construction on the same two-character input. Every arrow points forward: no intermediate ri_i is needed once its successor exists. The dashed arrow marks the one backward dependency that remains, namely that *decode* must be given the original, un-annotated r to know what shape the bits describe.

Bit-code representation. Rather than a `ParseTree` value, a parse tree is represented as a flat sequence of bits: 0 marks a *Left* choice, 1 a *Right* choice, and inside a Kleene star, 0 marks "one more iteration" and 1 marks "stop". A pair of mutually recursive functions $encode_r/decode_r$ convert between a $\vdash v : r$ parse tree and its bit-string representation, structurally straightforward given the tagging already present in v itself.

The star convention is worth spelling out on a concrete case, since it is the one place where a bit does not correspond to an alternation. Under $r = a^*$, the word `aaa` encodes as $[0, 0, 0, 1]$: three 0s, one per iteration taken, then a final 1 ending the star. The empty word under the same r encodes as $[1]$ alone, a star that stops immediately. So the bit-string length grows with the number of iterations rather than with the size of r , and reading it back requires knowing r , which is what $decode_r$ is given.

Annotated regular expressions. The construction's core idea is to annotate the derivative operation's own *input*, not just its output, with partial bit-code information, so that by the time a sub-expression becomes ε its accumulated annotation already *is* the relevant slice of the final bit-string. An annotated regular expression ri extends plain r with a bit-list bs attached at every node except $\emptyset$, which denotes no language and so has no parse tree to annotate:

$$ri ::= \emptyset \mid (bs@\varepsilon) \mid (bs@x) \mid (bs@ri_1 \oplus ri_2) \mid (bs@ri_1 ri_2) \mid (bs@ri^*) \quad (5.5)$$

Two operations connect plain r to this annotated world. *internalize* converts r into ri with every bit-list initially empty, except at *Alt*: the left operand of $r_1 + r_2$ is tagged with a leading 0 and the right with a leading 1 at construction time, so that which alternative a sub-tree came from can always be recovered from its own annotation without re-inspecting the surrounding structure. $fuse(bs, ri)$ prepends a bit-list onto whatever annotation ri 's own top-most node already carries, and is the one primitive every other operation in this construction is built from.

The trace's `INTERNALIZE` section shows the result for r_1 , with the alternation tags already in place before any input is read:

```
ri0 = []@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
      []@([false]@'b' (+) [true]@epsilon))
```

Reading the four tags off: `[false]` on a and `[true]` on ab record the two alternatives of $a + ab$, and the same pair appears again on b and ε . Every other bit-list is empty. Those four tags are the only bits the construction will ever emit for this expression; everything the derivative does from here is a matter of moving them, never of inventing new ones.

The refined derivative. The ordinary derivative operation is lifted to annotated expressions, written $ri \backslash_b l$, following the same case structure as Equation (3.4)'s $r \backslash l$ but threading bits through instead of discarding the annotation:

$$\begin{aligned} (bs@ri_1 \oplus ri_2) \backslash_b l &= (bs@(ri_1 \backslash_b l) \oplus (ri_2 \backslash_b l)) & (B_+) \\ (bs@ri_1 ri_2) \backslash_b l &= \begin{cases} (bs@(ri_1 \backslash_b l) ri_2) \oplus fuse(mkEpsBC_{ri_1}, ri_2 \backslash_b l) & \varepsilon \in L(ri_1) \\ (bs@(ri_1 \backslash_b l) ri_2) & \text{otherwise} \end{cases} & \begin{matrix} (B_{seq}^n) \\ (B_{seq}) \end{matrix} \\ (bs@ri^*) \backslash_b l &= (bs@fuse([0], ri \backslash_b l) ([]@ri^*)) & (B_*) \end{aligned} \quad (5.6)$$

The concatenation case mirrors $(r_1 r_2) \backslash x$'s two-way split exactly, with one addition: on the branch where r_1 's derivative is abandoned in favor of jumping into r_2 , the bits r_1 would have contributed to an empty match ($mkEpsBC_{ri_1}$, below) are fused directly onto r_2 's derivative rather than discarded. This is the incremental step that replaces the direct construction's *later* call to $mkEps_{r_1}$ inside *inj* (Section 5.2) with work done immediately, during the forward pass. The star case attaches a leading 0 bit ("one more iteration") each time the body is derived once more; the outer *bs* stays where it is, on the new alternation or concatenation node the derivative rule produces, exactly as it does for plain *Alt*.

mkEpsBC. Mirroring Equation (5.1)'s $mkEps$ but returning accumulated bits instead of a tree:

$$\begin{aligned}
mkEpsBC_{(bs@_\varepsilon)} &= bs & (EB_\varepsilon) \\
mkEpsBC_{(bs@ri_1 \oplus ri_2)} &= bs \mathrel{++} \begin{cases} mkEpsBC_{ri_1} & \varepsilon \in L(ri_1) \\ mkEpsBC_{ri_2} & \text{otherwise} \end{cases} & \begin{array}{l} (EB_+^L) \\ (EB_+^R) \end{array} & (5.7) \\
mkEpsBC_{(bs@ri_1 ri_2)} &= bs \mathrel{++} mkEpsBC_{ri_1} \mathrel{++} mkEpsBC_{ri_2} & (EB_{seq}) \\
mkEpsBC_{(bs@ri^*)} &= bs \mathrel{++} [1] & (EB_*)
\end{aligned}$$

Once the forward pass ends on a nullable annotated expression, $mkEpsBC$ extracts its complete bit-string directly, and $decode_r$ turns that bit-string into an ordinary parse tree. $decode$ uses the *original*, un-annotated r , because it needs to know the shape it is decoding bits *for*, which the annotated ri alone no longer determines once its own annotations have been consumed. That is the dashed dependency in Figure 5.2, and the only thing the construction still has to remember from before the forward pass began.

The worked example, one step at a time. Run Section 5.2's r_1 against *ab* through this construction instead, to see the same POSIX answer assembled incrementally rather than reconstructed afterward. Starting from the ri_0 quoted above, deriving with respect to *a* takes the Seq case's second branch, since the left operand $[]@(0@a \oplus 1@ab)$ is not nullable:

```

Step 1 (char: 'a', position 1)
  rule: (bs@ri1.ri2) \ 'a' [nullable_bc(ri1)=false]
        = bs@(ri1\'a'.ri2)
  simp -> []@([]@([false]@epsilon (+) [true]@'b')) . []@([false]@'b'
        (+) [true]@epsilon))

```

Comparing this against ri_0 , the literal 'a' has become ϵ and $[]@'a' . []@'b'$ has become 'b', while both alternation tags stay exactly where they were. Deriving one character has consumed structure and moved no bits.

ri_1 's left operand is now nullable, via $[false]@epsilon$, so the second character takes the other branch of the Seq case, and this is where bits finally move:

```

Step 2 (char: 'b', position 2)
  rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=true]
        = bs@((ri1\'b'.ri2) (+) (fuse (mkEpsBC ri1) (ri2\'b')))
  simp -> []@([true]@([false]@'b' (+) [true]@epsilon) (+) [false,
        false]@epsilon)

```

Two things happened in that one step. The rule's `fuse` (`mkEpsBC ri1`) produced the `[false, false]` prefix on the right disjunct: the first `false` is ri_1 's own empty-match bit, the second is $(b + \varepsilon)$'s left-alternative tag, fused in front of it. And a `[true]` appeared on the left disjunct, tagging "the derivative continued into the left operand" as the *Right* alternative of the freshly introduced node. Where *inj* discovers that tagging only on the way back, $\backslash_b$ must commit to it immediately, as part of the forward pass.

Both disjuncts of ri_2 are nullable, so *mkEpsBC* faces the same choice *mkEps* faced in Section 5.2, and resolves it the same way, by preferring the left disjunct whenever it is nullable at all:

$$\begin{aligned} mkEpsBC_{ri_2} &= [] ++ mkEpsBC_{1@([]@0@b\oplus1@\varepsilon)} & EB_+^L, bs = [] \\ &= [1] ++ ([] ++ mkEpsBC_{1@\varepsilon}) & EB_+^R, bs = [1] \\ &= [1] ++ [1] = [1, 1] & EB_\varepsilon, bs = [1] \end{aligned} \quad (5.8)$$

The outer step takes EB_+^L and the inner one EB_+^R , for the same reason in both cases: the rule tests only whether its own ri_1 is nullable. At the outer node ri_1 is the $1@(\dots)$ disjunct, which is; at the inner node ri_1 is $0@b$, a bare literal, which is not, so the right branch is taken and contributes the second 1. which the trace confirms, together with the decode that follows:

`mkEpsBC(ri2) = [11]`

```
decode((a|ab)(b|), [11]):
  = (Right (a, b), Right ())
```

Both constructions therefore encode "prefer continuing the match already in progress over falling back to the shorter alternative", just applied at different points: in *inj*'s backward pass there, and directly inside *mkEpsBC*'s forward-accumulated bits here. The two bits select *Right* at both alternation points of r_1 , which is the tree Section 5.2's worked example reached via its explicit backward pass, read this time straight off the forward pass alone.

On simplification between steps. Section 5.2 noted that the plain construction's forward pass must *not* simplify between steps, since the backward pass needs to walk back through the exact sequence of derivatives that produced it. The bit-coded construction has no such restriction, and both entry points in `src/parsers/deriv_bc/parse.rs` do call `simp` after every step, which is why every trace excerpt above shows a `simp ->` line rather than a raw derivative. This is safe here for the same reason `fuse` is: the information a backward pass would otherwise need to walk back for is already carried *on* the annotated expression itself, as bits attached to its nodes, so simplifying ri_i down to a smaller but bit-equivalent expression loses nothing *mkEpsBC* needs later. In the plain construction, by contrast, simplifying r_i would destroy the very structure *inject* needs to reconstruct against.

5.4 Simplification Rules

Section 3.2's worked example already showed plain derivatives growing an extra $+$ at every nullable-prefix step with nothing to bound that growth, and the bit-coded construction inherits the same problem. [Sulzmann and Lu \[2014\]](#) give the concrete case

$$\begin{aligned} a^* &\xrightarrow{a} \varepsilon a^* \xrightarrow{a} \phi a^* + \varepsilon a^* \\ &\xrightarrow{a} (\phi a^* + \varepsilon a^*) + (\phi a^* + \varepsilon a^*) \xrightarrow{a} \dots \end{aligned} \quad (5.9)$$

where every step after the first is language-equivalent to εa^* but syntactically doubles in size. Left unchecked, this defeats the whole point of the bit-coded construction's incremental, single-pass promise: an unboundedly growing ri makes each subsequent $\setminus_b$ step correspondingly more expensive.

A simplification function *simp*, applied after every derivative step, keeps the annotated expression's size bounded. Its rules are Phi-propagation through *Seq* and *Alt*, epsilon-elimination ($((bs@(bs'@\varepsilon) ri) \rightarrow fuse(bs ++ bs', ri)$, folding the eliminated node's own bits forward rather than discarding them), and flattening and deduplicating nested alternations, keeping the first occurrence of a syntactic duplicate. All of these are instances of the familiar algebraic identities from Section 3.2 ($\emptyset \cdot r = \emptyset$, $\varepsilon \cdot r = r$, $\emptyset + r = r$, $r + r = r$), applied here to the annotated grammar and combined with an explicit *isPhi* test that recognizes "empty language" expressions built up from these rules rather than only the literal $\emptyset$ constructor.

***simp* on the paper's own example.** Applied to a^* against *aaa*, *simp* collapses every step back down to a bit-annotated form of a^* itself. The trace shows one *simp* result per character, each a single star node whose accumulated bit-prefix has grown by one 0:

```
Step 1 ... simp -> [false]@([@'a')*
Step 2 ... simp -> [false, false]@([@'a')*
Step 3 ... simp -> [false, false, false]@([@'a')*
```

```
mkEpsBC(ri3) = [0001]
decode(a*, [0001]):
  = [a, a, a]
```

The expression stays at two nodes throughout while the bit-prefix grows linearly, and the three 0s plus *mkEpsBC*'s closing 1 are exactly the $[0, 0, 0, 1]$ encoding of "three iterations, then stop" from Section 5.3. This is the clearest single illustration of what the bit-coded construction is for: all of the parse-tree information is in the prefix, and none of it is in the expression.

Where *simp* does not bound the growth. This result does not generalize to every expression, and it is worth being precise about that rather than letting the a^* case stand for the general one. Figure 5.3 measures node counts per derivative step for two patterns, each with and without *simp*.

The two simplified series behave completely differently. For a^* the line is flat. For $(a + (b + ab))^*$ it is straight on a logarithmic axis, which is to say exponential. Every two characters take the size s to $2s + 1$ exactly, so $s + 1$ doubles: from 35 at step 4 the series runs 71, 143, 287, 575, 1151, 2303 at step 16, and 36 863 at step 24. The growth is therefore $\Theta(2^{n/2})$ in the input length, not the bounded size the a^* case suggests. The figures are unchanged if the three-way alternation is grouped the other way, as $((a + b) + ab)^*$, which is what the CLI spelling " $(a|b|ab)^*$ " parses to (Section 2.1); the growth depends on the pattern's shape rather than on that grouping. Applying *simp* repeatedly to a genuine fixpoint changes nothing either: a single pass already *is* the fixpoint here, producing byte-identical expressions at every step, so this is not a case of the implementation stopping too early.

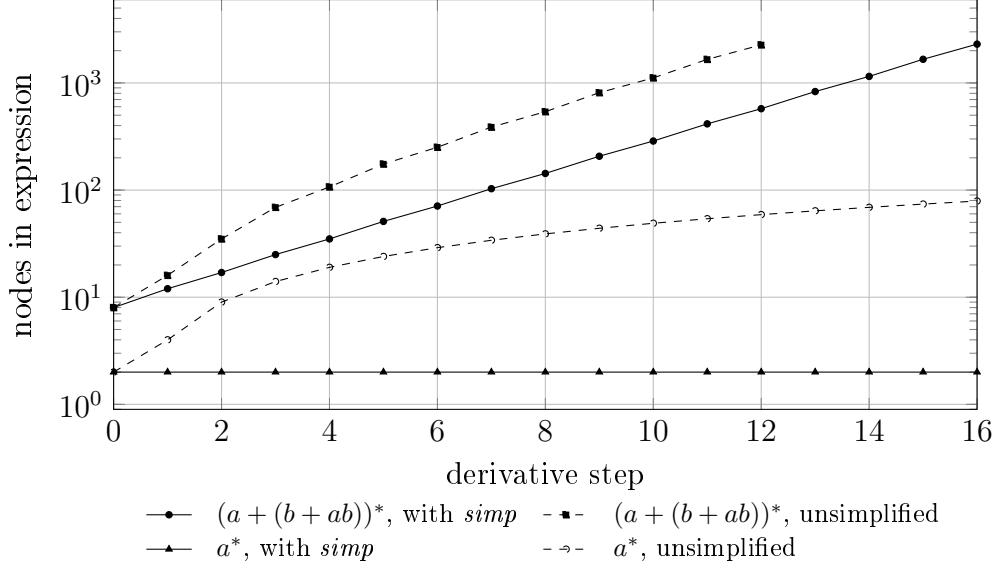


Figure 5.3: Expression size per derivative step, logarithmic vertical scale. For a^* , *simp* holds the expression at two nodes while the unsimplified derivative grows linearly. For $(a + (b + ab))^*$ against $(ab)^n$, *simp* reduces the expression substantially at every step and still leaves it growing exponentially: a straight line on this scale is exponential growth. Regenerate with `cargo run --release --example thesis_figures -- growth; tests/test_thesis_figures.rs` pins these values.

The mechanism is specific and easy to state. *simp*'s deduplication rule discards an alternation branch only when it is *syntactically identical* to an earlier one, and an annotated branch carries its bit-list as part of its syntax. Counting the top-level alternation branches of $\text{simp}(ri_i)$ at each step, and counting them again after erasing every annotation, separates the two:

Step	2	4	6	8	10	12
Branches	2	4	8	16	32	64
Distinct branches	2	4	8	16	32	64
Distinct ignoring bits	1	1	1	1	1	1

At every step, all of the branches are structurally the same expression, differing only in the bits recording which route through the alternation reached it, so the last row stays at 1 while the first two double. The $r + r = r$ rule never fires, because on annotated expressions the two sides are not equal.

The bits are not redundant: each branch's annotation is a different candidate parse tree, and *mkEpsBC*'s left-preference is what eventually selects among them, so dropping duplicates on shape alone would change which tree the parser returns rather than merely shrinking the expression. That is why the rule is stated on full syntactic equality in the first place, and why the growth here is a property of the construction rather than a defect in this transcription of it.

Sulzmann and Lu claim, but do not fully prove, that the resulting construction runs in time linear in the input, on the grounds that each of $\backslash_b$, *simp*, *fuse*, *mkEpsBC*, and *isPhi* only ever recurses over sub-calls bounded by the now-simplified expression's size, and they note explicitly that detailed space-complexity estimates are left as future work [Sulzmann

and Lu, 2014]. Figure 5.3 is one concrete reason that gap matters: the argument’s premise, that *simp* keeps the expression small, holds for a^* and fails for $(a + (b + ab))^*$, and the linear-time claim rests on it. Chapter 8 measures the resulting wall-clock behaviour on both kinds of pattern, and Section 8.5.4 returns to this family specifically.

5.5 Rust Implementation

Both parsers of this chapter are transcriptions of the equations above. Each definition becomes one Rust function over the `Regex` or `ARegex` enum, the case split of the equation becomes the arms of a `match`, and the recursion becomes recursion; the mapping is mechanical enough that reading the code against the equation adds nothing the equation does not already say. Appendix ?? gives the file-by-file correspondence. What is worth setting out here is the short list of places where Rust forces a decision the specification does not make.

Partial functions become panics. $mkEps_r$ is defined only for nullable r : Equation (5.1) has no clause for ϕ or for a literal, because neither is ever reached. Rust has no partial functions, so the transcription has to say something, and it says this:

```
Regex::Lit(c) => panic!("mk_eps called on Lit('{}')", c),
Regex::Phi    => panic!("mk_eps called on Phi"),
```

The panic is an assertion about the caller, not error handling. Every call site establishes nullability before calling, so reaching either arm would mean the invariant the backward pass rests on has already been broken. `inject` makes the same move at a finer grain: a bare `Pair` arriving where a nullable r_1 requires a tagged `Left` or `Right` can only mean the tree handed in did not come from that expression’s own derivative sequence. The alternative in both cases is to invent a plausible value and return it, and a wrong parse tree that looks well formed is precisely what the agreement tests of Chapter 8 could not detect: they would be comparing two parsers that had each guessed.

Recursion or an explicit vector. The two entry points of `deriv_std` compute the same tree by different means. The recursive one descends to the end of the input and calls `inject` as the stack unwinds, so the call stack *is* the derivative sequence of Figure 5.1’s top row and nothing else stores it. The loop one builds that sequence as an explicit `Vec<Regex>` going forward, then walks it backward. The algorithm is identical; the two differ only in where the top row is kept while the bottom row consumes it. That is a resource decision, and in Rust it is a consequential one, because a thread’s stack is a fixed allocation while the heap is not: the recursive entry point overflows on input the loop entry point handles without difficulty. Section 8.5.1 measures where the difference begins.

Trees and their flattening. `ParseTree` (Section 4.2) is the one representation every parser in this thesis produces, which is what lets the agreement tests compare outputs directly. `flatten` is kept separate from tree construction so that it can serve as an independent check: it discards every tag and pair boundary and returns the concatenated literals, so a successful parse of w must flatten back to w whatever the disambiguation policy was. Every diagnostic report prints the two next to each other for that reason.

5.5.1 The Bit-Coded Parser

One extra field, and the traits that come with it. `ARegex` is `Regex` with a `Vec<bool>` added to every variant but ϕ , which has no annotation slot. This is the only point in the implementation where a plain `Regex` will not do, since the whole purpose of the construction is to thread partial parse-tree information through the expression as it is derived, and that needs somewhere on every node to put it.

The two types then diverge in the traits they derive, and Rust makes that divergence explicit rather than incidental. `Regex` derives `Hash` because Chapter 6’s deduplication uses it as a `HashSet` key; `ARegex` does not, because `deriv_bc` never hashes an annotated expression. Its own deduplication, inside `simp`, compares alternation branches pairwise instead. A language deriving a fixed bundle of instances at once would leave that difference implicit; here it is a visible fact about which operations each type is expected to support.

Ownership makes the common case free. `fuse(bs, ri)` prepends a bit-prefix to an expression’s top-level annotation, and it opens by handing the expression straight back when there is nothing to prepend:

```
pub fn fuse(bs: &[bool], ri: ARegex) -> ARegex {
    if bs.is_empty() { return ri; }
    // ... otherwise rebuild the node with bs in front of its prefix
}
```

Because `fuse` takes `ri` by value, that early return is a move and not a copy: the expression is handed on untouched. `fuse` runs on every `Seq` and `Star` step of every derivative and the overwhelming majority of those calls carry an empty prefix, so the check converts the common case from a node rebuild into nothing at all.

Decoding, and where a bad bit-string is caught. `decode` turns the original plain `Regex` plus a flat `Vec<bool>` back into a `ParseTree`, consuming one bit per alternation and one per star iteration. It takes the un-annotated expression, not the derived one, which is the dashed dependency in Figure 5.2 and the reason it is shared with Chapter 6’s `pderiv_bc`: that parser arrives at a `(Regex, bits)` pair by an entirely different route and needs the identical final step. `decode` asserts that no bits remain once the expression has been walked, and panics on an unexpected end of bits, which is the same invariant-assertion discipline as `mk_eps` and `inject`, applied at the other end of the construction.

5.6 Comparing the Two Constructions

Sections 5.2 and 5.3 have now run the same expression against the same input through both constructions, quoting each trace at the point where it illustrated a rule. Setting the two reports against each other as wholes makes the structural difference visible in one view. The full reports are Appendix C.1 and Appendix C.2.

	<code>deriv_std</code>	<code>deriv_bc</code>
Report sections	forward, nullability, backward	internalize, forward, nullability, decode
Named intermediates	<code>r0 -> r1 -> r2</code>	<code>ri0 -> ri1 -> ri2</code>
Retained at the end	all three	the last one only
Tree-building calls	1 <code>mkEps</code> + 2 <code>inject</code>	1 <code>mkEpsBC</code> + 1 <code>decode</code>
Simplification	none, by necessity	<code>simp</code> after every step
Final answer	<code>(Right (a, b), Right ())</code>	

The number of tree-building calls is the row that does not scale with the input. `deriv_std` needs one `inject` per character and so grows with the input length; `deriv_bc` needs exactly one `mkEpsBC` and one `decode` no matter how long the input is. That is the whole content of the optimization, and it is visible in the reports as the difference between a `BACKWARD PASS` section whose length depends on the input and a fixed-size `MKEEPSBC + DECODE` section.

The final bits [11] and the final tree `(Right (a, b), Right ())` encode the same answer in the two representations, and the correspondence is exact: each `Right` in the tree is a 1 in the bit-string, read in the order `decode` walks the expression.

What happens on a failure. Running the same r_1 against `b`, a string not in $L(r_1)$ since every derivation of r_1 must begin by matching `a`, both parsers stop after one derivative step and neither reaches a tree-building phase at all. What each has in hand at that point differs, and the difference is instructive:

```
deriv_std:  r1 = Seq(Alt(Phi, Seq(Phi, Lit('b'))), Alt(Lit('b'), Eps))
            nullable(r1) = false  [FAIL] position 1
```

```
deriv_bc:   simp -> phi
            ri1 = phi
            nullable_bc(ri1) = false  [FAIL]
```

`deriv_std`'s expression denotes the empty language but is not syntactically `Phi`; it is a `Seq` whose left operand has both alternatives dead. `deriv_bc`'s `simp` collapses exactly that expression all the way down to `phi`. This is the single clearest illustration of what Section 5.4's *isPhi* test adds over plain `simplify`: recognizing an empty-language expression that is not the literal `∅` constructor.

Both constructions then report the identical failure, character for character:

```
Failure at position:  1 (1-indexed)
Found:               'b'
Expected:            'a'
  b
  ^
```

The diagnostics layer's expected set is read directly off the first-literal set of the expression at the point of failure, so it does not depend on which construction produced that expression or on how much simplification was applied to it. This is the general pattern for failures across all four parsers in this thesis: the constructions differ in what they build

on a successful match, and agree on what they report when there is nothing to build. The two full failure reports are [Appendix C.5](#) and [Appendix C.6](#).

Agreement between the three POSIX entry points. The two parsers of this chapter, together with `deriv_std`'s loop variant, are three separate routes to the same tree, so their outputs are directly comparable. Running all three over 300 randomly generated expressions against 39 inputs each, 11 700 cases in total, they agree on every one, both on whether the input matched and on the exact `ParseTree` returned when it did. [Section 8.4.2](#) describes the generator and the full test setup; what matters here is that the bit-coded route and the inject route, which share no code beyond `Regex` itself and `nullable`, are not observed to diverge on any of them.

6 Partial-Derivative-Based Parser

Chapter 5 built two parsers around Brzozowski’s derivative. This chapter repeats the exercise with Antimirov’s partial derivative from Section 3.3 in place of $r \backslash x$ throughout. The two derivative operators are both usable as the engine of a parser, but they are not interchangeable as methods of disambiguation: swapping one for the other changes which parse tree comes out. Section 6.1 shows where in the construction that difference is decided, and Section 6.1 shows it on an example small enough to check by inspection.

Neither flops14-extended [Sulzmann and Lu, 2014] nor Sulzmann’s teaching-page Haskell reference [Sulzmann, 2026] defines a POSIX partial-derivative parser at all. The paper’s bit-coded construction (Chapter 5) is Brzozowski-derivative-only, and the teaching page’s own bit-coded partial-derivative parser, `pDerivBC`, is built around a deduplication rule that decides the matter on its own, without any comparison of candidate trees. This chapter builds two parsers on that basis: `pderiv_bc` (Section 6.3), following `pDerivBC` directly, and `pderiv_std` (Section 6.4), this project’s own construction, built to give `pderiv_bc` the same kind of matched non-bit-coded counterpart Chapter 5’s `deriv_bc` has in `deriv_std`. Both are verified to agree with each other exactly, and with the two POSIX parsers on membership alone, in Chapter 8 and again in Section 6.6.

Trace excerpts are used here exactly as in Chapter 5, under the conventions Section 5.1 sets out, and against the same running example. The two parsers’ level-3 reports have their own section structure, since a frontier is not a single expression:

Report section	Construction phase
INITIAL FRONTIER	the one-strand starting state
FORWARD PASS	one block per character: before, rule, after
SELECTION	the first-nullable-strand decision
MKEEPSBC + DECODE	bit-coded only; <code>pderiv_std</code> injects in place

The four full reports are Appendices C.3, C.4, C.7 and C.8.

6.1 Extending Derivatives to Partial Derivatives

Section 3.3 already defined $pd(r, x)$ as a *set* of residual expressions rather than one combined expression, and noted the NFA reading: each element of $pd(r, x)$ is one state of an implicit NFA the original r denotes, alive after reading x . Extending this to parsing means pairing each surviving element of that set with enough information to reconstruct a parse tree once a particular strand turns out to be the winning one at the end of the input. Concretely, that is either an accumulated bit-string per strand (Section 6.2) or, in this project’s own extension, an accumulated tree-injection closure per strand (Section 6.4).

Either way the algorithm shape is the same, and Figure 6.1 draws it: maintain a *frontier* of (**residual**, **extra**) pairs, one per live strand, replace it at each character with the union of *pd* applied to every current strand, each contributing its own possibly-empty set of successor strands, and at the end of the input select the first strand in the frontier whose residual is nullable.

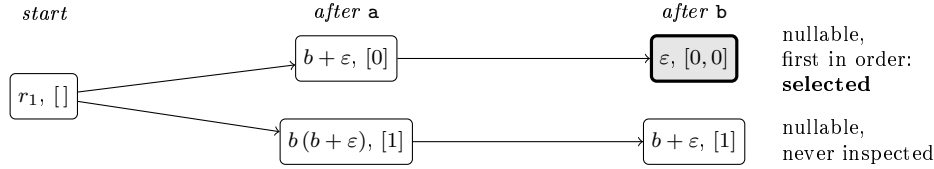


Figure 6.1: The frontier for $r_1 = (a+ab)(b+\varepsilon)$ against "ab", with `pderiv_bc`'s bits shown as the per-strand extra. One character advances every strand independently. At the end both strands are nullable, and selection takes the first in list order without comparing them, which is where the construction settles on one tree rather than the other.

Where the choice is actually made. That last step, "the *first* nullable strand in the frontier", is where this construction's disambiguation policy is actually decided, and it is decided implicitly, as a consequence of how the frontier's growth is kept finite rather than by any explicit ordering rule analogous to Section 4.3.3's (A1) to (A4) and (K1) to (K4).

Antimirov's finiteness theorem (Section 3.3) guarantees the set of *distinct* partial derivatives is bounded, but nothing about the extended, tree-or-bit-carrying construction guarantees strands stay distinct as they accumulate: two different derivation paths can easily reach the identical residual *shape* by two different routes, each carrying its own bits or injection. Kept without pruning, this set would multiply extra copies of an already-explored residual instead of staying within the bounded, syntactic set *pd* itself guarantees. Pruned instead by comparing the accumulated bits or trees for POSIX-style "longest match wins", the way Section 4.3.3's ordering does, the comparison would need to look inside possibly-incomplete, still-growing accumulated state rather than a finished `ParseTree`, which is not what the ordering relation of Section 4.3.3 is defined over.

The Haskell reference's actual choice is `nub2`: deduplicate purely on residual *shape*, keeping whichever strand reached that shape *first*. This sidesteps the problem entirely by discarding the comparison altogether. Once a residual shape has been claimed by some strand, every later strand reaching the identical shape is simply dropped, regardless of what either one's accumulated bits or tree would eventually have produced. "First" here means first in the list-ordered traversal *pd*'s own case structure produces, which explores the left operand of $+$ before the right at every step. The construction is therefore left-biased and commits as soon as something succeeds, which is not the longest-wins-with-left-as-tiebreak rule the derivative parsers follow. It is worth being careful about what that does and does not establish: it says which strand this construction selects, not that the construction implements any particular named disambiguation policy in general.

The trace states the consequence in one line, at the point where the decision is made:

```

SELECTION (first nullable residual, priority/list order)
any nullable residual in final frontier = true -> proceed to
    mkEpsBC + decode

```

There is no comparison here, because there is nothing to compare against: the moment a nullable residual is found, scanning stops. That is the shaded node in Figure 6.1, and the strand below it is never examined even though it is nullable too.

This reading is this thesis’s own account of where the construction settles the question; neither primary source states it in these terms, and neither claims the result coincides with a named policy. What can be checked is narrower and is checked in Chapter 8: the two parsers of this chapter agree with each other on every case tried, they agree with the derivative parsers of Chapter 5 on membership always, and they disagree with them on the tree wherever the expression is ambiguous, including the `paper_r1` and `paper_r2` examples of Section 4.3.

A family of witnesses. The two paper examples are not the only place the divergence is visible, and it is worth having a case where the pattern is simple enough that both answers can be read off by inspection. Take $r = (b + bb)^*$ against a run of `bs`. The derivative parsers prefer the longer alternative at each iteration; the partial-derivative parsers take the left alternative as soon as it works. The two therefore partition the same input differently:

Input	Derivative	Partial derivative
"bb"	[Right (b, b)]	[Left b, Left b]
"bbb"	[Right (b, b), Left b]	[Left b, Left b, Left b]
"bbbbbb"	[Right (b, b) $\times$ 3]	[Left b] $\times$ 6

The derivative parsers use as few iterations as the longest-alternative rule allows, padding with a single `Left b` only when the length is odd. The partial-derivative parsers use one iteration per character throughout, because b is the left alternative and it always matches. Both flatten back to the same input, and both are valid parse trees for it: the expression is ambiguous, and the two constructions resolve the ambiguity differently. These rows are the output of `cargo run - parse "(b|bb)*" "bb" -parser all`, which runs all five entry points on one input and prints the trees side by side.

6.2 Bit-Coded Variant

Sulzmann’s teaching-page Haskell transcript gives a bit-coded partial-derivative construction, `pDerivBC`, that this thesis’s `pderiv_bc` follows directly [Sulzmann, 2026]. It extends `pd` with per-strand bits the same way Section 5.3’s $\backslash_b$ extends the Brzozowski derivative, but, notably, keeps those bits *external* to the regular-expression tree, as a plain list carried alongside a plain, un-annotated r , rather than folding them into an `ARegex`-shaped tree the way $\backslash_b$ does.

Writing D_r for `pDerivBC(x, r)` throughout, to keep the set comprehensions readable, the

base and non-sequence cases are

$$\begin{aligned}
D_\varepsilon &= \{\} & D_\emptyset &= \{\} & (\text{PB}_\varepsilon), (\text{PB}_\emptyset) \\
D_y &= \begin{cases} \{(\varepsilon, [])\} & x = y \\ \{\} & x \neq y \end{cases} & & (\text{PB}_{\text{lit}}) \\
D_{r+s} &= \text{nub2}(\{(r', 0:bs) \mid (r', bs) \in D_r\} \\
&\quad \cup \{(s', 1:bs) \mid (s', bs) \in D_s\}) & & (\text{PB}_+) \\
D_{r^*} &= \text{nub2}(\{(smartC(r', r^*), 0:bs) \mid (r', bs) \in D_r\}) & & (\text{PB}_*)
\end{aligned} \tag{6.1}$$

and the sequence case, Equation (6.2), splits on the nullability of the left factor exactly as Equation (3.4) does:

$$\begin{aligned}
D_{rs} &= \{(smartC(r', s), bs) \mid (r', bs) \in D_r\} & (\text{PB}_{\text{seq}}), \text{ if } n(r) = \text{false} \\
D_{rs} &= \text{nub2}(\{(r's, bs) \mid (r', bs) \in D_r\} \\
&\quad \cup \{(s', mkEpsBC_r ++ bs) \mid (s', bs) \in D_s\}) & (\text{PB}_{\text{seq}}^n), \text{ if } n(r) = \text{true}
\end{aligned} \tag{6.2}$$

The names extend Section 3.3's P scheme, one PB rule per P rule, and the worked example below cites them with the binding of r and s given at each application. The two sequence cases differ in exactly one respect worth keeping in view: PB_{seq} builds its residual with $smartC$, while PB_{seq}^n builds the first of its two strands with a plain $r's$. Here $mkEpsBC_r$ is the plain- r analogue of Section 5.5.1's $mkEpsBC$, identical in structure but walking an un-annotated tree instead of an **ARegex**, and nub2 deduplicates a list of (**residual**, **bits**) pairs purely by comparing the residual, keeping the first occurrence. $smartC$ is the smart concatenation constructor of Section 3.3, collapsing $\varepsilon \cdot s$ to s .

The concatenation case's first strand deliberately keeps r' wrapped in a literal, non-smart-constructed $r's$ rather than $smartC$. This asymmetry is present in the reference itself and is preserved here rather than "corrected", since correctness of this construction is judged against literal agreement with that reference (Chapter 8) and not against an independently-derived ideal form. Section 6.4 returns to the same question from the other side, where the choice between the two constructors is not free at all.

6.3 Rust Implementation: The pderiv_bc Parser

`src/parsers/pderiv_bc/pderiv.rs` transcribes $pDerivBC$ case for case. Each D_r becomes a `Vec<(Regex, Vec<bool>)>` rather than a set. Order matters here, encoding that priority directly in list position, so a `HashSet` would lose exactly the information the construction depends on. `mk_eps_bits` implements $mkEpsBC_r$; `nub2` deduplicates on the residual `Regex` alone, via a `HashSet<Regex>` used only to test membership, keeping the first-seen pair for each distinct residual and discarding the rest. This is the one place in the codebase where `Regex`'s derived `Hash` implementation (Section 2.3) is load-bearing. `pderiv_bc`'s own `Seq` case matches the reference literally, down to the same asymmetric use of plain `Regex::seq` rather than `smart_seq` for its first strand.

`step_frontier` and `select_bits` (`src/parsers/pderiv_bc/parse.rs`) implement the frontier-driving loop of Section 6.1. At each character, `step_frontier` replaces every (**residual**, **bits**) pair in the current frontier with the union of `pderiv_bc` applied to it, concatenating each result's bits onto that strand's already-accumulated prefix. At

the end of the input, `select_bits` scans the frontier in list order and returns the first nullable strand's `bits ++ mk_eps_bits(residual)`. `parse_pderiv_bc` drives this loop to completion and passes the winning bit-string, together with the *original* `Regex` and never the derived residual, to `decode` (Section 5.5.1). That is the same shared decoder Chapter 5's `deriv_bc` uses, since both constructions end up at the identical (`Regex`, `bits`) shape by the time decoding is needed, whatever route each took to get there.

The worked example, one step at a time. Run Section 5.3's $r_1 = (a + ab)(b + \varepsilon)$ against `ab` through `pderiv_bc` instead of `deriv_bc`: same regex, same input, different parser, so that the two constructions can be seen disagreeing on the same worked example rather than only being asserted to. The frontier starts as a single strand, the whole regex with no bits yet, which is the trace's INITIAL FRONTIER section and the leftmost node of Figure 6.1.

Deriving with respect to `a`, the outer node binds $r = a + ab$ and $s = b + \varepsilon$. That r is not nullable, by N_+ over two non-nullable literals and a concatenation, so PB_{seq} applies and no second strand is contributed by s . The strands that do appear come from PB_+ one level down, binding $r = a$ and $s = ab$ and tagging the two alternatives 0 and 1. Where the plain bit-coded construction kept one expression, there are now two strands:

```
Step 1 (char: 'a', position 1)
  rule: pDerivBC('a', r1.r2) [nullable(r1)=false]
        = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
frontier after (2 residual(s)):
  [0] Alt(Lit('b'), Eps)    bits = [0]
  [1] Seq(Lit('b'), Alt(Lit('b'), Eps)) bits = [1]
```

Strand [0] came from a 's own derivative, ε , which `SC` then absorbed against $(b + \varepsilon)$ by its first branch, leaving `Alt(Lit('b'), Eps)` alone; it carries [0] for $(a + ab)$'s left disjunct. Strand [1] came from ab 's derivative, b , which `SC` kept as a concatenation by its second branch since $b \neq \varepsilon$; it carries [1] for the right disjunct. Both bits were emitted by PB_+ one level down, and PB_{seq} shown in the rule line carried them outward unchanged, which is why neither strand's bits grew at this step.

Deriving with respect to `b` steps *both* strands independently, and the trace prints a rule line per strand, which is where the frontier's per-strand character becomes explicit:

```
Step 2 (char: 'b', position 2)
  from [0]:
    rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++
      tag(1, pDerivBC('b',r2)), nub2
    -> (Eps, [0])
  from [1]:
    rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
          = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
    -> (Alt(Lit('b'), Eps), [])
frontier after (2 residual(s)):
  [0] Eps    bits = [00]
  [1] Alt(Lit('b'), Eps) bits = [1]
```

Two different rules fired on the same character, because the two strands are different

expressions and each is matched against the rule set independently. Strand [0] is an `Alt`, so `PB+` applies with $r = b$ and $s = \varepsilon$: the left disjunct contributes ε tagged 0, and the right contributes nothing, since `PB $\varepsilon$` returns the empty set. Strand [1] is a `Seq` binding $r = b$ and $s = b + \varepsilon$, and that r is a bare literal and so not nullable, so `PBseq` applies and derives only the left factor. The accumulated bits differ correspondingly: strand [0] gains a 0 and reaches [00], while strand [1] gains nothing and stays at [1].

Selection then scans this final frontier in list order and takes the *first* nullable strand. Strand [0], (`Eps`, [00]), is checked and accepted before strand [1] is even inspected, regardless of strand [1] also being nullable:

```
selected residual's accumulated bits ++ mkEpsBC(residual) = [00]
```

```
decode((a|ab)(b|), [00]):
  = (Left a, Left b)
```

This is the leftmost-commits rule made mechanical. Strand [0] is exactly the "matched `a`, then `b` separately" reading, the first alternative `pDerivBC` generated back at the first derivative step, and it wins here purely because it appears first in the frontier, not because any length comparison favors it. The decoded tree is the *shorter* reading, matching `a` and then `b` separately, and it is the opposite answer from Section 5.3's (`Right(a, b)`, `Right()`) on the identical regex and input. Nothing about r_1 or "`ab`" changed between the two traces; only the disambiguation policy governing which nullable strand or branch wins did, exactly the distinction Section 6.1 argues is built into `nub2`'s shape-only, first-occurrence-wins deduplication.

Where `nub2` is and is not applied. `nub2` appears inside `pderiv_bc`, once per case that can produce more than one strand, and nowhere else. In particular `step_frontier` concatenates the per-strand results into the next frontier without deduplicating across them, so the guarantee `nub2` provides is local to a single strand's expansion and does not extend to the frontier as a whole. Two strands that were distinct at step i can converge on the same residual at step $i + 1$ and both survive. Section 6.5 measures what that costs.

6.4 Rust Implementation: The `pderiv_std` Parser

`pderiv_std` (`src/parsers/pderiv_std/`) is this project's own construction, with no counterpart in either reference. Its purpose is to stand to `pderiv_bc` as `deriv_std` stands to `deriv_bc`: the same traversal, the same disambiguation policy, and the same answers, but building a `ParseTree` directly instead of a bit-string that has to be decoded afterward. Having both means this chapter's answer is computed twice by routes that share almost no code, which is what makes the agreement check of Section 6.6 worth anything.

What each strand carries. Where `pderiv_bc` pairs each residual with a `Vec<bool>`, `pderiv_std` pairs it with an *injection closure* instead, defined in this module's `inject.rs`:

```
pub(crate) type Inj = Rc<dyn Fn(ParseTree) -> ParseTree>;
```

A strand's `Inj` is a function that, given the parse tree of that strand's *residual*, returns the parse tree of the *original* expression. The frontier starts with one strand carrying

`identity_inj()`, and each step composes the step's own injection inside the strand's accumulated one. Figure 6.2 contrasts the two representations after two steps.



Figure 6.2: What one strand holds after two derivative steps. The bit-coded strand accumulates into a flat buffer; the tree-building strand accumulates a chain of heap-allocated closures, one link per step, each holding the next. Both are applied only once, to the winning strand, at the end of the input.

This is the same idea as Chapter 5's `inject`, with one difference that matters: `deriv_std` applies `inject` once per character during an explicit backward pass, re-deriving the expression sequence it needs as it unwinds, whereas `pderiv_std` never runs a backward pass at all. It builds the composition forward, one layer per step, and applies the finished composition exactly once at the end. In that sense `pderiv_std` is as single-pass as `pderiv_bc` is; the closure chain plays the role the bit-string plays there.

The `Rc` is required rather than incidental. A single step can expand one strand into several, and every one of those successors needs the same parent injection, so the parent has to be shared rather than cloned or moved. `Rc<dyn Fn(...)>` gives shared ownership of a heap-allocated closure at the cost of a reference-count bump per successor. Section 8.5.2 measures what this costs against `pderiv_bc`'s flat `Vec<bool>`; the structural point here is the one Figure 6.2 draws, that a strand's accumulated state is a chain whose length grows with the input where `pderiv_bc`'s is a contiguous buffer.

`pderiv_tree`. `src/parsers/pderiv_std/pderiv_tree.rs` is the (Regex, Inj) analogue of `pderiv_bc`, case for case, with each case's bit-tagging replaced by the tree constructor that bit would have decoded to:

Case	<code>pderiv_bc</code> attaches	<code>pderiv_tree</code> attaches
$\varepsilon, \emptyset$	no strands	no strands
x (matching)	<code>[]</code>	<code> _ Char(x)</code>
$r + s$, left	<code>0:bs</code>	<code> v Left(inj1(v))</code>
$r + s$, right	<code>1:bs</code>	<code> v Right(inj2(v))</code>
rs , continue r	<code>bs</code>	<code> Pair(v1,v2) Pair(inj1(v1), v2)</code>
rs , jump to s	<code>mkEpsBC_r ++ bs</code>	<code> v2 Pair(mk_eps(r), inj2(v2))</code>
r^*	<code>0:bs</code>	<code> Pair(v1,vs) Star(inj1(v1) : vs)</code>

Every row is the same decision, expressed once as a bit and once as a constructor. The correspondence is exact enough that the `Seq` case's "jump to s " strand calls the shared `mk_eps` (Section 5.5) at precisely the point `pderiv_bc` calls `mk_eps_bits`, and for the same reason: the left factor matched empty, so its contribution to the pair has to be synthesized rather than derived. `nub_tree` is the Inj-carrying `nub2`, deduplicating on the residual alone and keeping the first occurrence, so the priority order is preserved identically.

Why the smart constructor cannot be used here. `pderiv_bc` is free to apply *smartC* in its non-nullable `Seq` case and in its `Star` case, so as to collapse $\varepsilon \cdot s$ down to s . `pderiv_tree` is not, and uses plain `Regex::seq` in every position where its sibling uses the smart one. The reason is a direct consequence of what an `Inj` is applied to.

Selection ends by calling `inj(mk_eps(r))` on the winning strand, so each accumulated closure is applied to a tree whose *shape is determined by its own residual*. The `Seq` strands' closures pattern-match on `ParseTree::Pair`, which is what `mk_eps` produces for a `Seq` residual. Collapsing `Seq(Eps, s)` to `s` would make `mk_eps` produce `s`'s own tree shape instead, and the closure expecting a `Pair` would receive something else. The residual and the injection are two halves of one invariant, and a smart constructor rewrites one half without the other.

This is not a speculative concern. Substitute `smart_seq` for `Regex::seq` in the two `Seq` strands of `pderiv_tree`, and the test suite fails immediately, on the simplest possible sequence pattern, with the panic that case's own guard raises:

```
---- parsers::pderiv_std::parse::tests::seq_matches stdout ----
thread '...:seq_matches' panicked at
  src/parsers/pderiv_std/pderiv_tree.rs:71:38:
pderiv_tree: Seq(non-nullable) strand expected Pair, got Char('b')
```

The panics in `pderiv_tree` therefore serve the same purpose as those in `inject` (Section 5.5) and `decode` (Section 5.5.1): they encode a precondition that holds by construction, and they convert a violation into an immediate, located failure rather than a silently wrong parse tree.

This is also why the two parsers' frontiers look different in the traces even though they compute the same answer. After one step on r_1 , `pderiv_bc`'s first residual is `Alt(Lit('b'), Eps)` and `pderiv_std`'s is `Seq(Eps, Alt(Lit('b'), Eps))`: the same language, the same strand, one of them still carrying the `Eps` its sibling discarded. Because `nub_tree` deduplicates on residual syntax, the two parsers can in principle prune at different points, which makes their agreement on every tested case a stronger check than it would be if they shared a residual representation.

Driving the frontier. The driver in `src/parsers/pderiv_std/parse.rs` mirrors its `pderiv_bc` counterpart exactly. `step_frontier` expands every strand and composes injections; `select` scans the final frontier in list order and returns `inj(mk_eps(r))` for the first nullable residual; `parse_pderiv_std` runs the loop, returning `None` as soon as the frontier becomes empty, since a frontier with no strands can never regain one. As with `pderiv_bc`, no deduplication is applied across the flattened frontier, only within each strand's own expansion.

The same worked example, and what the trace cannot show. Running r_1 against `ab` through `pderiv_std` produces the same two-strand fork at step 1, and the same winner at step 2. The one visible difference from `pderiv_bc`'s trace is a recurring annotation:

```
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not
  printable)
-> Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps)) (+ injection
  closure, not printable)
```

This is the one place where this parser’s state is less inspectable than its bit-coded sibling’s. A `Vec<bool>` can be printed; an `Rc<dyn Fn>` cannot. The trace can therefore show which residuals are alive and which rule produced each, but not what tree each strand would build if it won, which is precisely the information `pderiv_bc`’s `bits = [00]` annotations do carry. That limitation is a direct cost of the representation choice drawn in Figure 6.2: a trace can only report state the construction actually keeps, and this one does not keep the bits.

The residuals themselves also differ from `pderiv_bc`’s, in exactly the way the smart-constructor argument above predicts. Where that parser lists `Alt(Lit('b'), Eps)`, this one lists `Seq(Eps, Alt(Lit('b'), Eps))`; where that one lists `Seq(Lit('b'), ...)`, this one lists `Seq(Seq(Eps, Lit('b')), ...)`. The two parsers are not running over the same residuals at all, and still select the same tree.

6.5 Frontier Growth

Both parsers of this chapter inherit Antimirov’s finiteness result for the set of distinct partial derivatives (Section 3.3), and neither inherits a bound on the size of the frontier, for the reason Section 6.1 identified: the frontier holds (`residual`, `extra`) pairs, and only the residual component is subject to *pd*’s bound. This section measures the consequence.

Take $(a + (b + ab))^*$, the `paper_r2` expression, against $(ab)^n$, and record both the number of strands in the frontier after each character and the number of *distinct* residuals among them.

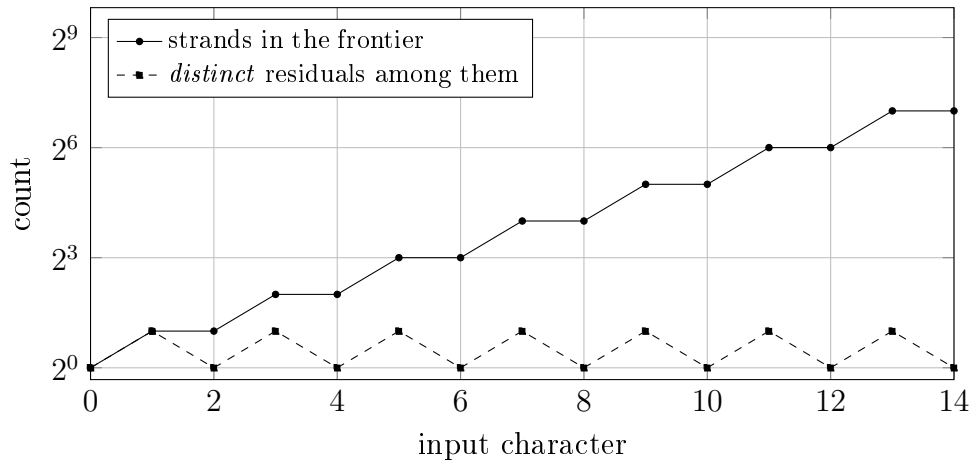


Figure 6.3: Frontier size against distinct residual count for $(a + (b + ab))^*$ on $(ab)^n$, logarithmic vertical scale. The frontier doubles every two characters while the number of distinct residuals never exceeds two: every strand beyond the second is a duplicate carrying different accumulated state. Regenerate with `cargo run --release --example thesis_figures -- frontier; tests/test_thesis_figures.rs` pins these values.

Every strand beyond the second is a duplicate of one already present, differing only in the bits or injection it carries. `nub2` does not remove them because it never sees them together: it runs inside `pderiv_bc`, on the successors of one strand, and the duplicates here arise *between* strands, at the point `step_frontier` concatenates the per-strand results.

This is the partial-derivative counterpart of Section 5.4’s finding for `deriv_bc`, and the two have the same root cause seen from different sides. There, alternation branches that were structurally identical survived deduplication because their bit annotations made them syntactically distinct. Here, residuals that are syntactically identical survive because the deduplication is never applied at the level where they meet. In both cases the bounded object is the set of residual *shapes*, and in both cases what actually grows is the set of shapes paired with accumulated parse-tree state.

Deduplicating across the frontier. The obvious question is whether the duplicates can simply be dropped. Moving `nub2` out of `pderiv_bc` and applying it across the whole flattened frontier, keeping the first occurrence, is a change of one deduplication site, and it preserves the priority order for the same reason `nub2` does inside a single expansion: the frontier is built in priority order, so the first occurrence of a residual is the highest-priority strand that reaches it.

Whether that argument is enough to preserve the selected *tree* is a different question, and this thesis does not prove it. It was tested instead, by running both variants over the 300 generated expressions and 39 inputs of Section 8.4.2, 11 700 cases in total. The two agree on every case, on membership and on the exact tree returned, and the peak frontier over the whole run falls from 170 strands to 4.

The shipped `pderiv_bc` nonetheless keeps `nub2` where the reference puts it. The reason is the one stated in Section 6.2: this parser’s correctness criterion is literal agreement with `pDerivBC`, and a change that is verified on 11 700 cases but not proved is a change to the thing being verified against. Section 8.7 records it as an available optimization rather than an applied one, with the measurement above as the evidence for its value.

6.6 Comparing the Two Policies

Section 4.3 constructed, by hand, the *second* valid derivation of $r_1 = (a+ab)(b+\varepsilon)$ against `ab` (`paper_r1`) that Chapter 5 deliberately left aside: $v' = (\text{Left } a, \text{Left } b)$, matching the *first* alternative of $a + ab$ against `a` alone rather than the second alternative against `ab` in one step. That is this chapter’s result, and the two preceding sections have shown both of its parsers computing it. Figure 6.4 sets the two answers side by side.

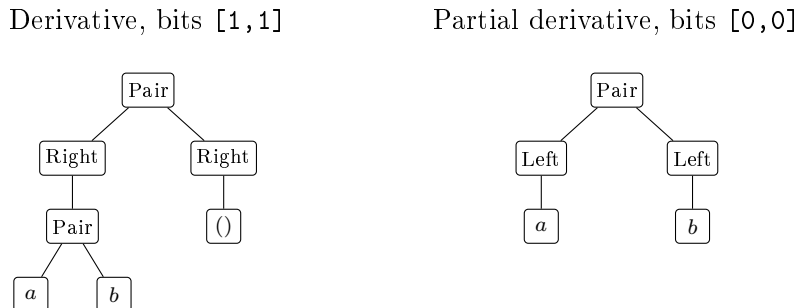


Figure 6.4: The two parse trees for $(a + ab)(b + \varepsilon)$ against `"ab"`. Both flatten to `"ab"` and both are valid. The derivative parsers take the second alternative of $a + ab$, consuming both characters at once and leaving $b + \varepsilon$ to match empty; the partial-derivative parsers take the first, leaving $b + \varepsilon$ to consume the `b`.

The bit-strings underneath make the comparison exact. Four alternation choices exist in r_1 , two of which are reached on any given parse, and the two parsers disagree on both: $[1,1]$ against $[0,0]$. Nothing else differs. The same expression, the same input, the same membership answer, and the entire distinction between the two chapters compressed into two bits.

Failure behaviour is shared. Against b instead of ab , the first partial derivative of r_1 produces no residual at all, since neither alternative of $a + ab$ begins with b :

```
rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
     = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
-> []
frontier after (0 residual(s)):
no nullable residual in this frontier [FAIL]
```

An empty frontier is the partial-derivative counterpart of `deriv_bc` simplifying to `phi` (Section 5.6), and it is detected the same way but earlier: both `parse_pderiv_std` and `parse_pderiv_bc` return as soon as the frontier empties, without consuming the rest of the input, because a frontier with no strands has no way to acquire one. For a long input whose first character already fails, that is the difference between one step and n .

Both parsers then emit the same error summary as Chapter 5's two, character for character:

```
Failure at position: 1 (1-indexed)
Found:               'b'
Expected:            'a'
```

The two chapters' constructions disagree, deliberately, on which tree a successful match produces, but agree exactly on the Boolean question of membership and on the diagnostics reported for a non-match. Neither disambiguation policy nor the choice between bits and closures is ever reached until after the underlying derivative or partial-derivative walk has already found at least one live residual. The full reports are Appendix C.7 and Appendix C.8.

All five entry points at once. The `-parser all` mode runs every parser on one input and checks the three invariants this and the previous chapter claim, without any diagnostics level being selected:

```
$ cargo run -- parse "(a|ab)(b|)" "ab" --parser all
Regex: (a|ab)(b|)
Input: "ab"
```

Parser	Policy	Result
DERIV_REC	POSIX	(Right (a, b), Right ()) -> "ab"
DERIV_LOOP	POSIX	(Right (a, b), Right ()) -> "ab"
DERIV_BC	POSIX	(Right (a, b), Right ()) -> "ab"
PDERIV_STD	GREEDY	(Left a, Left b) -> "ab"
PDERIV_BC	GREEDY	(Left a, Left b) -> "ab"

```
POSIX parsers agree
PDERIV parsers agree
GREEDY agrees with POSIX on membership
```

The three closing lines are the invariants themselves, checked on this input: the three derivative entry points must return identical trees, the two partial-derivative ones must return identical trees, and all five must agree on whether the input matched. Nothing requires the two *policies* to agree, and on this input they do not, which is the entire subject of both chapters. Section [8.4](#) runs the same three checks across the full corpus rather than a single input.

7 External versus Internal Regex

Every example so far in this thesis has been written directly in `Regex` terms, the six-constructor core algebra of Section 2.3 that `deriv` and `pderiv` operate on. No real regular-expression user writes that way. A Snort rule, a SpamAssassin filter, or an ordinary call to `cargo run - match` arrives as an ordinary PCRE-style pattern *string*, `"[a-c]+\d{2}"` rather than `Seq(Star(...), ...)`. `src/frontend/` is the module that closes this gap: it parses that string into its own surface-syntax AST, then lowers that AST into the `Regex` every parser in Chapters 5 and 6 already understands.

It is modeled throughout on the `ExtPattern`, `Parse` and `Translate` modules of `Text.Regex.PDeriv`, from `luzhuomi/regex-pderiv`, Kenny Lu's own reference implementation of Sulzmann and Lu [2012]'s partial-derivative submatching. That is a second, independent reference alongside Sulzmann and Lu [2014]'s own teaching-page material, used here specifically because the `flops14`-extended paper itself has nothing to say about surface syntax: POSIX parsing is defined directly over the same six-constructor `Regex` algebra this thesis's core inherits, with no PCRE-facing frontend of its own.

Section 8.3's real-world corpus depends on everything described here, surface-syntax parsing, desugaring, and the substring-search padding scheme, to turn raw Suricata, SpamAssassin and `RegexLib` rule text into `Regex` values in the first place. The chapter closes (Section 7.4) with the price that translation charges: how large a `Regex` an ordinary-looking pattern actually becomes, which is what the four parsers then have to walk once per input character.

7.1 Two ASTs, One Lowering Step

`src/frontend/` is built around a deliberate separation into two abstract syntax trees connected by a single function, rather than parsing pattern text directly into `Regex`. Figure 7.1 gives the whole module in one view: a common path that every entry point follows, and two optional stages that only some of them switch on.

`ExtPat` (`ext_pattern.rs`) is the *external*, surface-syntax AST: marking and non-marking groups, `{m,n}` bounds, `\d/\w/\s` shorthands, `^/$` anchors, and a `greedy`: `bool` flag on every quantifier (`a*` against `a*?`). It holds everything a PCRE-style string can express, kept exactly as written, with nothing desugared yet. `Regex` (Section 2.3) is the *internal*, six-constructor core every parser derives or partial-derives directly: `Phi` | `Eps` | `Lit` | `Seq` | `Alt` | `Star`, with no groups, no bounds, no anchors and no laziness flag. Anything `ExtPat` can say that `Regex` cannot represent is desugared away by `translate` (Section 7.3) or, for `^` and `$` specifically, erased into `Eps` and handled one level up instead (Section 8.3.1).

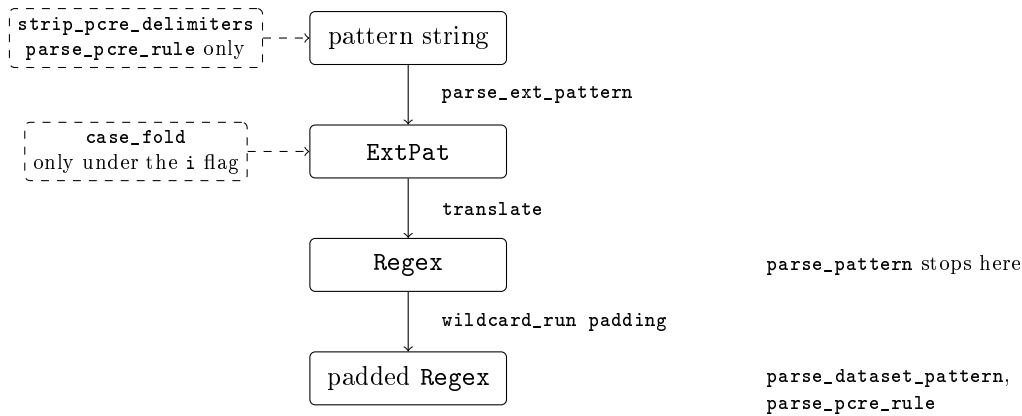


Figure 7.1: The frontend as a pipeline. The solid path is common to all three entry points; the dashed stages apply only where marked. `parse_pattern` stops at `Regex` and means full-string equality, while `parse_dataset_pattern` and `parse_pcre_rule` continue into the padding of Section 8.3.1.

Keeping `ExtPat` as an honest AST in its own right, rather than routing straight from the parser into `Regex` in a single pass, is what makes `translate` independently testable. The unit tests in `translate.rs` construct `ExtPat` values by hand and check their `Regex` output directly, with no dependency on the recursive-descent logic in `parse.rs` at all. It is the same separation-of-concerns argument that Section 5.5 makes for keeping `ParseTree` construction separate from `flatten` and `decode`, applied one layer further out, at the point where a PCRE-style string first becomes structured data at all.

The separation earns its keep a second time at the entry points. `mod.rs` exposes three of them, all sharing one parser and one `translate`, and differing only in what they do to the `ExtPat` in between:

Entry point	Behaviour
<code>parse_pattern</code>	<code>translate</code> alone: full-string match
<code>parse_dataset_pattern</code>	plus anchor-aware padding (Section 8.3.1)
<code>parse_pcre_rule</code>	plus <code>/.../i</code> delimiter stripping and case folding

Every example in Chapters 5 and 6, and every invocation of `cargo run - parse`, goes through `parse_pattern`, which is why `"(a|ab)(b|)"` against `"ab"` means the whole input and not a substring of it. The corpus of Section 8.3 goes through the other two, which is where real rule text is handled.

7.2 Parsing Surface Syntax

`frontend/parse.rs` is an ordinary recursive-descent parser over a `Chars` iterator, structured directly after the grammar of `Text.Regex.PDeriv.Parse`. Its productions correspond one for one, each handling exactly the surface-syntax layer its reference counterpart does:

Reference	This frontend	Layer handled
p_ere	parse_or	alternation
p_branch	parse_concat	concatenation
p_exp	parse_postfixed	postfix quantifiers
p_atom	parse_atom	a single atom
p_group	parse_group	(...) forms
p_charclass	parse_charclass	[...]
p_enum	parse_class_enum	class members
p_bound	parse_bound	{m,n}

The table is a list of names; Figure 7.2 is what those names mean operationally. Each production consumes exactly one syntactic layer and then calls the next one down, so the order of the calls *is* the precedence order, and the single back-edge from `parse_group` is what makes the grammar recursive at all.

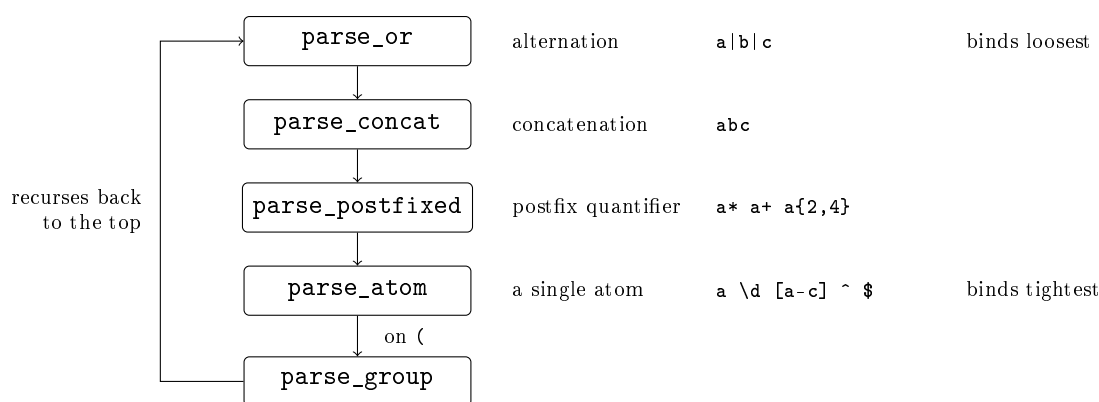


Figure 7.2: The recursive-descent ladder of `frontend/parse.rs`. Each production strips one layer and delegates the rest downward, so precedence falls out of the call order. `parse_group` is the only production that climbs back to the top, which is why parentheses override precedence and why nesting is unbounded.

Four deliberate departures from the reference:

- **Class shorthands are PCRE character classes, not literal escapes.** The reference’s `Translate.hs` reads a bare `EEscape c` as the literal character `c` always; this frontend special-cases `d/D/w/W/s/S` into character classes (Section 7.3’s `alphabet.rs`) instead, since real datasets use them that way throughout.
- **Backreferences and lookahead are rejected outright.** The backreferences `\1` to `\9`, and the four lookahead openers `(?=`, `(?!`, `(?<=` and `(?<!`, all produce a descriptive parse error rather than the reference’s literal-digit or three-digit octal-ASCII escape reading. Both describe languages that are not regular in the formal sense this thesis’s entire construction depends on, so silently mis-parsing them into *some* `Regex` would be a worse outcome than refusing them, and Section 8.3’s extraction pipeline depends on this refusal being loud and categorized rather than silent.
- **Named groups are accepted.** `(?<name>...)` and `(?P<name>...)` parse successfully, and the name is dropped during `translate`. A name carries no meaning once lowered to `Regex`, since this project has no capture-group semantics at all, but

rejecting a construct this common in real Suricata and RegexLib rules purely because the reference parser predates it would reject working patterns for no benefit. `(?P=name)`, a named *backreference*, is rejected along with the numeric ones.

- **\xHH hex escapes are accepted**, another addition absent from the reference, for the same reason.

Two small extensions exist for this thesis’s own sake rather than for the datasets. A literal ε character in the pattern text parses to `ExtPat::Eps`, and a literal $\emptyset$ to `ExtPat::Never`, so that Section 2.1’s own notation for ε and $\emptyset$ can be written directly on the command line. PCRE syntax has no way to spell either, and the alternative used throughout Chapters 5 and 6 is an empty alternative: `(b|)` is $b + \varepsilon$, because `parse_concat` returns `ExtPat::Empty` when a branch is empty rather than failing.

Bounds are parsed speculatively. `parse_bound` is the one place where the parser backtracks. A `{` in a pattern is only sometimes the start of a repetition bound, so `parse_bound` clones the iterator, tries to read a complete `{m}`, `{m,}` or `{m,n}` specification from the clone, and commits the clone back only on success. On failure the `{` is left unconsumed, and since `{` is in the parser’s `SPECIALS` set, the next `parse_atom` call reports it as unexpected.

This has a consequence worth recording, because it makes one check in `translate` unreachable. An inverted bound such as `a{4,2}` is rejected by `try_parse_bound_spec`, which returns `None` when the upper bound is below the lower one, so the error the user sees comes from the *atom* parser:

```
$ cargo run -- parse "a{4,2}" "aa"
Regex parse error: unexpected special character '{' here
```

rather than from `desugar_bound`’s own "upper bound is less than lower bound" message, which no input reaching it through `parse_pattern` can trigger. The message is less specific than the one that exists for the condition, which is worth knowing when reading extraction-failure categories in Section 8.3.

`parse_bound` additionally accepts a lazy-quantifier marker (`a*?`, `a+?`, `a{2,4}?`) and records it as `ExtPat`’s `greedy`: `bool` flag: correctly read, but, as Section 7.3 explains, not honored by `translate`. Grouping, alternation and concatenation otherwise follow the reference’s precedence exactly: alternation binds loosest, then concatenation, then postfix quantifiers, then a single atom, which may be a literal, an escape, a character class, a group or an anchor.

7.3 Translating ExtPat into Regex

`translate` (`frontend/translate.rs`) is a single structural recursion over `ExtPat`, one match arm per constructor, each either a direct one-to-one mapping or an explicit desugaring:

ExtPat	Regex
Empty, Eps	Eps
Never	Phi
Char(c)	Lit(c)
Group/GroupNonMarking(r)	translate(r) (grouping is precedence-only)
Or([r1, ..., rn])	left-folded Alt chain
Concat([r1, ..., rn])	left-folded Seq chain
Opt(r)	Alt(translate(r), Eps)
Plus(r)	Seq(translate(r), Star(translate(r)))
Star(r)	Star(translate(r))
Carat, Dollar	Eps (see Section 8.3.1)
Dot, Any(cs), NoneOf(cs)	sorted, deduplicated Alt of Lit
Escape(c)	class shorthand, or Lit(c)
Bound(r, lo, hi)	unrolled exact/optional repetition, below

The left-folding of `Or` and `Concat` is what makes the associativity convention of Section 2.1 concrete: `abc` becomes `Seq(Seq(a, b), c)`, and it is that tree, not the string, that every derivative in this thesis is taken of.

Character classes are ordinary alternation, not a primitive. `Any`, `NoneOf`, `Dot` and the class-shorthand escapes all route through `alt_of_chars`, which sorts and deduplicates the character set and left-folds it into a plain `Alt` chain of `Lit` nodes. `alphabet.rs` supplies the underlying sets: `alphabet()`, which is 98 characters (printable ASCII plus tab, newline and carriage return), `digit_chars`/`word_chars`/`space_chars` for `\d`/`\w`/`\s`, and `complement` (the alphabet minus a given set) for their negated and uppercase counterparts and for `[^...]`.

Every one of the four parsers only ever needs to understand `Regex`'s original six constructors, at the cost of a `\d` inside a real pattern becoming a ten-way `Alt` that the parser walks one branch at a time on every character. Section 7.4 measures how large that cost actually is.

Two consequences of drawing the alphabet this way are worth stating, since both are departures from PCRE that a rule author would not expect. The alphabet is ASCII only, so a negated class `[^a]` denotes 97 specific characters rather than "every character except `a`", and a non-ASCII input character matches no class at all. And `alphabet()` *includes* newline, so `.` here matches a newline where PCRE's `.` by default does not.

Bounds unroll literally, capped against blowup. `desugar_bound` builds `r` repeated exactly `lo` times, via `repeat_exact`, a left-folded `Seq` chain seeded with `Eps`, then appends either `Star(r)` for an unbounded upper end (`r{2,}`) or `hi - lo` further nested optional copies for a bounded one (`r{2,4}`), each layer of `optional_extra` being `Alt(Eps, Seq(r, ...))`. An exact bound `r{2}` takes a third path, returning `repeat_exact` alone with nothing appended. The `Eps` seed stays in the result rather than being smart-constructed away, so `a{2}` lowers to `Seq(Seq(Eps, Lit('a')), Lit('a'))` rather than `Seq(Lit('a'), Lit('a'))`. That costs one node and one nullable-test per derivative step, and it keeps `repeat_exact` a single unconditional fold with `Eps` as the natural identity for `lo == 0`.

Unlike every other desugaring in this table, this one can make a pattern arbitrarily large from a short piece of source text. `r{1586,}`, which is Section 8.3's own example of a

rejected real-world bound, would otherwise unroll into 1586 nested `Seq` nodes before the `Star` even starts. `MAX_BOUND`, at 1000 repetitions, is this project's own safety limit rather than something carried over from the reference, and `translate` refuses any bound past it with a descriptive error rather than silently building an oversized `Regex`:

```
$ cargo run -- parse "a{1586,}" "aa"
Regex parse error: bound {1586,} exceeds this frontend's cap of
1000 repetitions (literal unrolling would blow up the regex size)
```

Greedy and lazy quantifier syntax is parsed but not honored. Every `Opt`, `Plus`, `Star` and `Bound` match arm above ignores the `greedy` flag that `parse_bound` (Section 7.2) recorded, because `Regex` has no per-operator laziness concept at all. POSIX-versus-Greedy disambiguation in this thesis is a whole-parser policy, `deriv_std/deriv_bc` against `pderiv_std/pderiv_bc` (Section 4.3), rather than a per-quantifier one. `a*` and `a*?` therefore produce identical `Regex` values.

A pattern mixing lazy and greedy quantifiers, expecting the per-operator control PCRE engines provide, will parse successfully but not behave as its author intended. This is a genuine expressiveness gap between the surface syntax this frontend accepts and what `Regex` can distinguish, noted here rather than left to be discovered. The flag is nonetheless carried on `ExtPat` rather than discarded at parse time, so that the information survives to the one layer that could act on it if per-quantifier policy were ever added.

Case folding. `parse_pcre_rule` handles a `/PATTERN/FLAGS` wrapper, the form Suricata and SpamAssassin rules are written in. `strip_pcre_delimiters` splits the body from the flags, passing a pattern through unchanged if it is not `/`-delimited, and the `i` flag triggers `case_fold` on the `ExtPat` before `translate` runs. `case_fold` rewrites each literal letter into a two-way `Any`, so `a` becomes `[aA]`, and leaves digits, punctuation and class shorthands untouched, since those are already case-agnostic. Folding at the `ExtPat` layer rather than after lowering is what keeps this to a single recursion: at the `Regex` layer the same change would have to distinguish literals that came from a class from literals that came from source text, a distinction `translate` has by then already erased.

7.4 The Cost of Translation

Every desugaring in this chapter is exact, and several of them are expensive. Since the object that reaches Chapters 5 and 6 is the lowered `Regex` rather than the pattern string, the size of that object, and not the length of the pattern, is what the parsers' per-character work scales with. Counting nodes in the lowered result for a few patterns makes the translation's real price visible:

The purely structural desugarings stay small, and their shapes are worth seeing written out, since each is exactly what Section 7.3's table prescribes:

Pattern	Lowers to	Nodes
<code>a*</code>	<code>Star(Lit)</code>	2
<code>a?</code>	<code>Alt(Lit, Eps)</code>	3
<code>a+</code>	<code>Seq(Lit, Star(Lit))</code>	4
<code>a{2}</code>	<code>Seq(Seq(Eps, Lit), Lit)</code>	5
<code>a{2,}</code>	exact copies then <code>Star</code>	8
<code>a{2,4}</code>	exact copies then nested options	15

Character classes and search padding are what change the order of magnitude. A `\d` is a 10-way `Alt` chain at 19 nodes, `\w` a 63-way one at 125, `[^a]` a 97-way one at 193, and `.` a 98-way one at 195. An unanchored search for the three-character literal `abc` reaches 399.

That last figure is the one that matters most for Section 8.3. Searching for a three-character literal builds a 399-node expression, because each `wildcard_run()` is a 195-node alternation under a `Star` and two of them are needed. Every derivative step then walks that structure.

Figure 7.3 puts these on one logarithmic axis, together with two patterns closer to what the corpus actually contains. The span is the point: the same six constructors carry expressions differing by more than two orders of magnitude, and nothing about the pattern’s source length predicts where in that range it lands.

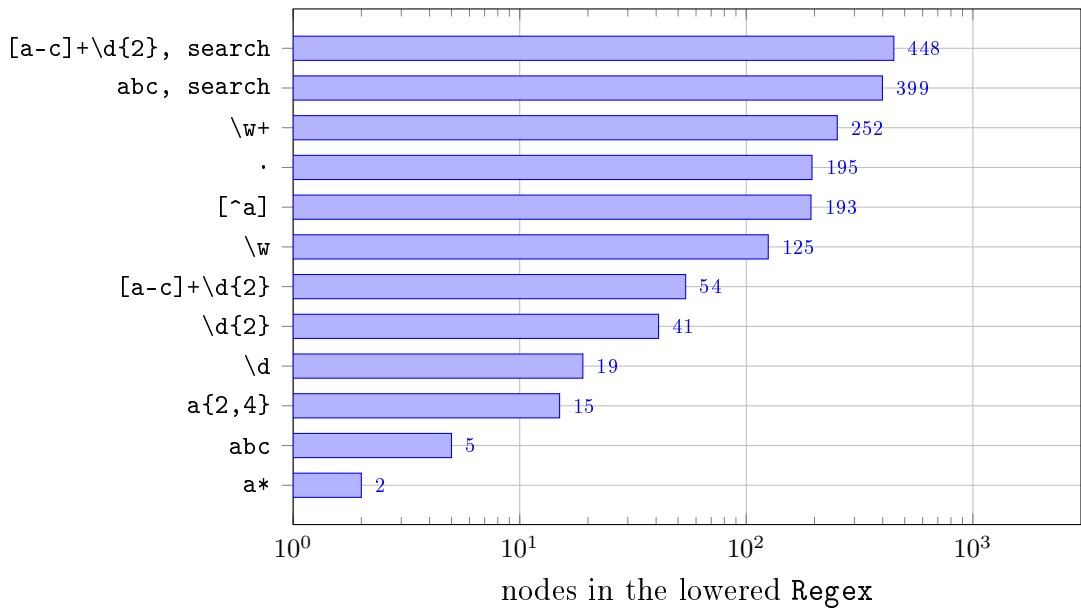


Figure 7.3: Lowered node counts, logarithmic horizontal axis. The three smallest are plain structural desugarings, with no character class involved. Every entry from `\d` upward expands one, and the two `search` entries additionally carry the padding of Figure 8.1. A realistic corpus pattern therefore starts around 450 nodes before any input is read. Regenerate with `cargo run --release --example thesis_figures -- lowered`.

This is the single largest constant factor separating this implementation from an engine with a native character-class representation, and it applies to the corpus patterns uniformly, since essentially all of them are unanchored searches containing at least one class shorthand.

The expansion is also visible in the diagnostics, in a way that is unhelpful rather than

merely large. Because the expected-set reported on a failure is read off the first-literal set of the expression at the point of failure, and a lowered class *is* that set, a single mismatched character against `.` reports all 98 alternatives individually:

```
$ cargo run -- parse "." "ä" --diag 1
Regex:  .
Input:  "ä"
Match:  false
Error:  position 1: found 'ä', expected '\t', '\n', '\r', ' ',
        '!', '"', '#', '$', '%', '&', ... , '}', or '~'
```

(abbreviated here; the tool prints the full list). The message is accurate, and it is exactly what the pattern denotes after lowering, but it no longer resembles the pattern the user wrote. A native class representation would let the diagnostics layer report **expected: any character** instead. It is also an instance of a general limit: once an abstraction has been compiled away, no amount of tracing can recover it, because the information is no longer in the object being traced.

The effect compounds rather than merely adding, which is why the chart climbs faster than the patterns lengthen. `\d{2}` lowers to two copies of the 19-node digit chain, reaching 41 nodes, and `\w+` to a 63-way alternation appearing twice, once for the required copy and once inside the **Star**, reaching 252. Stacking both effects, `[a-c]+\d{2}` is 54 nodes as a full-string pattern and 448 once padded for search, from twelve characters of source text.

Case folding is the one remaining multiplier, and it is comparatively cheap: `case_fold` turns each literal *letter* into a two-way **Alt** and leaves everything else alone, so `/abc/i` lowers to 405 nodes against `/abc/`'s 399, six more for three folded letters. Unlike the class expansion, this one scales with the number of letters in the pattern rather than with the size of the alphabet.

None of this changes which language is denoted, and none of it changes which parse tree any of the four parsers selects. It changes only how much structure each of them traverses per input character, which is exactly the quantity Section 8.5 measures.

8 Evaluation

Chapters 5 and 6 developed four parser combinations – `deriv_std`, `deriv_bc`, `pderiv_std`, `pderiv_bc` – each claimed to compute either the POSIX or the leftmost-first parse tree, by four structurally different constructions. This chapter verifies those claims empirically: Section 8.2 describes how the test suite checks the four parsers, and the three Boolean matchers of Chapter 3, against each other and against the paper’s own worked examples; Section 8.3 describes how a corpus of real-world regular expressions was assembled to test and benchmark against, alongside the synthetic patterns used elsewhere in this thesis; Section 8.4 reports the results of that testing; and Section 8.5 measures where the four parsers’ costs actually diverge from each other, on both the synthetic and real-world workloads.

8.1 Compared Engines

The primary object of comparison in this chapter is the four parser combinations against *each other*: two built on the derivative and two on the partial derivative, one tree-building and one bit-coded within each pair (Chapters 5-6) – since that internal comparison is what this thesis’s own theoretical claims (POSIX-correctness of the derivative parsers, agreement of all four on membership, the tree-versus-bit-coded performance trade-off) are actually about. A second, external comparison against established regular expression engines is motivated in Chapter 1.1 and positions these results relative to the wider field. It is implemented as an optional, feature-gated addition to this codebase (Rust’s `regex` crate as an ordinary optional dependency; Google’s RE2 via a small C++ shim over its own API, since RE2 offers no C API of its own) rather than a default one, specifically so that the default build stays free of any external regular-expression engine dependency – `cargo build/cargo test/cargo bench` never touch either engine unless the `external-engines` feature is explicitly enabled. Section 8.6 reports and discusses the resulting numbers; what follows briefly positions both engines conceptually, since that section refers back to this framing.

8.1.1 Rust’s `regex` Crate

Rust’s `regex` crate compiles a pattern to a Thompson-construction NFA and simulates it without backtracking, guaranteeing linear-time matching in the input length independent of the pattern – the same automaton-based guarantee Chapter 1.1 attributes to finite-automata approaches generally. Its disambiguation policy is leftmost-first, matching Section 4.3.1’s Greedy order rather than POSIX leftmost-longest; it offers no POSIX mode. That makes it the natural external comparison for this thesis’s partial-derivative

parsers (`pderiv_std/pderiv_bc`), which are also leftmost-first, on performance. Whether the two agree on the parse tree as well is a question this thesis does not settle: the partial-derivative construction is left-biased and commits on first success (Section 6.1), which is not by itself a claim that it implements the same policy the crate does.

8.1.2 RE2

Google’s RE2 is likewise automaton-based (Thompson NFA compiled lazily to a DFA on demand), explicitly designed to avoid the exponential backtracking blowup of backreference-supporting engines [Google LLC, 2024, Cox, 2007]. Unlike Rust’s `regex` crate, RE2 supports an explicit POSIX leftmost-longest mode alongside its default Perl-style leftmost-first mode, making it a comparison point for *both* disambiguation policies studied in this thesis, in one engine, depending on which mode is selected.

8.2 Testing Strategy

The test suite is split, following ordinary Rust convention, into unit tests (`#[cfg(test)] mod tests` blocks inside the `src/` files they exercise, testing private helper functions alongside public ones) and integration tests (`tests/`, exercising only the public API each parser or matcher module exposes). `tests/common/mod.rs` factors out what is shared across the integration suites: the two worked examples from the paper, `paper_r1` and `paper_r2` (Section 4.2), and two assertion helpers, `assert_round_trip` (checking `flatten(parse(r,w)) == w`, i.e. that a produced parse tree is actually evidence for the word it was built from) and `assert_parsers_agree` (comparing two `Option<ParseTree>` results with a labeled failure message identifying which two implementations disagreed).

8.3 Dataset Preparation

The synthetic patterns used throughout this thesis – `paper_r1`, `paper_r2`, the pathological (a^*) -style expressions of Section ?? – are deliberately small and hand-constructed, chosen to isolate one specific algorithmic property at a time. They say little about how any of the four parsers behaves on the regular expressions people actually write. Mamouras et al.’s disambiguation-robustness study [Mamouras et al., 2024] motivates closing that gap: it studies exactly the POSIX-versus-Greedy question this thesis studies, via real Snort/-Suricata intrusion-detection signatures, and finds concrete real-world patterns where the two policies visibly diverge. That paper provides no public dataset artifact of its own, so this thesis’s corpus was independently assembled from three families of real-world regular expression sources chosen to cover different "shapes" of real regex use, not three samples of the same thing:

- **Snort/Suricata** network intrusion-detection signatures (`pcre:"..."` rule options, sourced from the Emerging Threats Open ruleset) – literal-heavy, moderate complexity, the closest of the three to "everyday" regex use.
- **SpamAssassin** email spam-filtering rules (`body/header` Perl-regex rules) – content matching over unstructured text, a different shape than network signatures.
- **RegexLib** crowd-sourced general-purpose patterns (validators, extractors) – the

opposite end, where chained lookahead assertions are idiomatic, stress-testing the frontend’s rejection path hardest of the three.

Extraction. Each source embeds its patterns in a different surrounding syntax, so each needs its own extractor (`examples/extract_dataset.rs`): Suricata’s rule lines carry a `pcre:"/PATTERN/FLAGS"`; option; SpamAssassin’s carry a bare `body/header NAME = /PATTERN/FLAGS` without the Suricata’s trailing-quote convention; RegexLib’s sample file lists plain patterns directly, one (possibly multi-line) entry per blank-line-delimited block. All three converge on the same `/PATTERN/FLAGS` PCRE delimiter convention once extracted, parsed by `frontend::parse_pcre_rule` – including the case-insensitive `i` flag, applied by case-folding every literal character in the translated `ExtPat` to an `Any` of both cases (Chapter 7’s frontend) rather than as a runtime matching option, since `Regex` itself has no case-insensitivity concept.

Filtering. Real rules routinely use constructs no derivative-based engine over the six-constructor `Regex` of Section 2.3 can express at all – backreferences and lookahead assertions describe languages that are not regular in the formal sense this thesis’s entire construction depends on. `parse_pcre_rule` rejects both outright with a descriptive error rather than silently mis-parsing them, and `extract_dataset` tallies every rejection by cause (`backreference`, `lookaround`, or `other` – chiefly bound expressions such as `{1586,}` that would unroll past this frontend’s repetition cap, and a handful of unsupported `(?...)` group syntaxes). Re-running this pipeline against the tracked raw sources gives the current triage:

Format	Extracted	Accepted	Backreference	Lookaround	Other
Suricata	231	209 (90.5%)	5	11	6
SpamAssassin	82	69 (84.1%)	1	11	1
RegexLib	499	407 (81.6%)	8	55	29
Total	812	685 (84.4%)	14	77	36

Lookaround dominates the rejections and is concentrated almost entirely in RegexLib, matching that family’s idiomatic use of chained assertions; Suricata and SpamAssassin reject mostly on the same handful of negative-lookahead idioms (guarding against one specific following word). The accepted 685 patterns, deduplicated across all three formats into `data/corpus_patterns.txt` (665 lines after deduplication), are what Section 8.5.5’s dataset benchmark actually runs against.

Unanchored matching. Real rules are *written* as search patterns: a Suricata rule fires on seeing a match anywhere in a packet payload, not on the payload being exactly that pattern. They are therefore parsed through the frontend’s two search entry points, `parse_dataset_pattern` and `parse_pcre_rule`, instead of the full-string-equality `parse_pattern` used everywhere else in this thesis’s own examples. That difference is not cosmetic: real dataset patterns, run unanchored the way they have to be, are measurably bigger than the pattern text suggests and expose scaling behaviour the small synthetic patterns of Section 8.5.4 do not. The mechanism is worth setting out before the measurements that depend on it.

8.3.1 Substring Search: How the Corpus Is Run

No parser in Chapters 5 and 6 answers any question but "does this entire input derive to nullable". There is no "found somewhere inside a longer string" mode anywhere below the frontend, and neither [Sulzmann and Lu \[2014\]](#) nor Sulzmann's teaching-page material defines one: POSIX and Antimirov derivatives are inherently full-match operations. Real rules, though, are *written* as search patterns. A Suricata rule fires on seeing its pattern anywhere in a packet payload, not on the payload being exactly that pattern.

`frontend/mod.rs` therefore implements substring search entirely at this layer, before a `Regex` ever reaches a parser, by padding the translated core with `wildcard_run()`, a `Star` over a 98-way `Alt` of the full alphabet meaning "any run of characters at all", on whichever side lacks an explicit anchor. Figure 8.1 gives the four cases, with the size of each lowered result alongside.

<code>^abc\$</code>	<table><tr><td>abc</td></tr></table>	abc	9 nodes, no padding		
abc					
<code>^abc</code>	<table><tr><td>abc</td><td>Σ^*</td></tr></table>	abc	Σ^*	204 nodes, end padded	
abc	Σ^*				
<code>abc\$</code>	<table><tr><td>Σ^*</td><td>abc</td></tr></table>	Σ^*	abc	204 nodes, start padded	
Σ^*	abc				
<code>abc</code>	<table><tr><td>Σ^*</td><td>abc</td><td>Σ^*</td></tr></table>	Σ^*	abc	Σ^*	399 nodes, both sides padded
Σ^*	abc	Σ^*			

Figure 8.1: Anchor-aware padding under the search entry points. Each Σ^* is one `wildcard_run()`, 196 nodes on its own, so the padding dominates the lowered expression whenever the pattern is short. The fully anchored case still costs 9 rather than 5 nodes, because the erased `^` and `$` leave an `Eps` apiece behind in the concatenation.

`starts_with_carat` and `ends_with_dollar` (`ext_pattern.rs`) decide which sides need padding by walking into `Concat` and `Group` to find a leading `^` or trailing `$` even when nested, before `Carat` and `Dollar` are themselves translated away to `Eps`. Under the search-oriented entry points `parse_dataset_pattern` and `parse_pcre_rule`, as against `parse_pattern`'s plain full-string equality used everywhere else in this thesis's own examples, deciding which side of this padding is skipped is the anchors' only real effect.

Because the anchors are recognized structurally rather than positionally, a `^` that is not at the start, as in `a^b`, is not an anchor at all. It still parses to `Carat` and still translates to `Eps`, so it contributes a node to the lowered expression but nothing to the language:

```
$ cargo run -- parse "a^b" "ab" --diag 1
Regex:  a^b
Input:  "ab"
Match:  true
Tree:   ((a, ()), b)
```

The `()` in the middle of that tree is the erased anchor. PCRE would treat the same pattern as unmatchable outside multiline mode. This is a silent difference rather than an error, and it is the one place in the frontend where a mis-written pattern is neither rejected nor faithfully represented.

Edge case. An unanchored pattern already nullable on empty input, with `"a*"` the simplest example, gets padded on both sides regardless of its own content. The padding's

own empty match, combined with either `Star`, is then enough by itself to match *any* string at all, not just runs of `a`. This is correct substring-search semantics, since "some substring matches" is satisfied by the empty substring that every string contains, and `mod.rs`'s own test `unanchored_nullable_pattern_matches_anything` asserts exactly this behaviour rather than treating it as a defect. It is worth flagging nonetheless, because such a pattern carries no information in a corpus: it matches every input and so contributes nothing to Section 8.3's measurements beyond its own parse cost.

anchors are not a language feature here. Recognising `^` and `$` structurally, and letting them do nothing but suppress one side of the padding, is a decision about the corpus pipeline and not an account of what anchors mean. This thesis's core algebra has six constructors and no notion of a position within an input, so there is nowhere in it for an anchor to live: a derivative consumes a character and knows nothing about where that character sits. Extended matchers that do support anchors get them from a richer algebra. [Varatalu et al. \[2023\]](#) handle intersection, complement and lookarounds on top of derivatives, and treat anchors as expressible through lookarounds rather than as primitives of their own. Adopting that route would mean extending the core algebra and every derivative rule defined over it, which is well beyond the scope of this implementation, but it is the direction in which the gap described above closes, and it is the reason the gap is a scope boundary rather than an oversight.

What this dataset does not (yet) give. The corpus was triaged for "does it parse at all," not for "does POSIX visibly diverge from the partial-derivative answer on this pattern the way `paper_r1`/`paper_r2` do", and unlike Mamouras et al.'s own disambiguation-robustness analysis [[Mamouras et al., 2024](#)], this thesis does not (yet) identify which, if any, of the 685 accepted real-world patterns are themselves disambiguation-non-robust in their sense. Combining this dataset with their static-analysis technique to find such patterns automatically, or constructing inputs that provoke visible divergence between the two constructions on the existing corpus directly, is a natural extension of this evaluation left to future work (Section 8.7).

8.4 Correctness Verification

8.4.1 Paper Examples Reproduced

Both worked examples from Sulzmann and Lu's paper are reproduced as executable tests rather than only as the manual derivations of Sections 4.3 and 4.3.3: `parse_recursive_paper_r1_on_ab` and `parse_recursive_paper_r2_on_ab` (`tests/test_deriv_std.rs`) assert the exact POSIX tree shape derived by hand in Section 4.3.3; `paper_r1_computes_greedy_not_posix` and `paper_r2_computes_greedy_not_posix` (`src/parsers/pderiv_bc/parse.rs`) assert both the expected tree shape of Section 4.3.1 *and* that it disagrees with the POSIX answer on the identical input – encoding the divergence itself, not merely each answer separately, as the property under test. Every one of these four assertions currently passes, confirming this implementation reproduces both documented policies exactly on the paper's own examples.

8.4.2 Cross-Implementation Equivalence

Following the agreement-testing strategy of Section 8.2, the full test suite checks, and at the time of writing passes, agreement at every layer this thesis introduces more than one implementation of the same specification:

- `match_naive = match_deriv = match_pderiv` on membership, across the hand-picked cases of `tests/test_matchers.rs`.
- `parse_recursive` and `parse_loop` produce identical parse trees on every shared test case (Section 5.5), including `recursive_traced_and_loop_traced_agree_on_all_paper_examples`, comparing full derivation traces, not just final answers.
- `deriv_bc` agrees with `deriv_std` (via `parse_recursive`) on every shared test case (`bitcoded_agrees_with_recursive`, `tests/test_deriv_bc.rs`), the empirical counterpart of Section 5’s claim that both compute the identical POSIX tree by different means.
- `pderiv_std` and `pderiv_bc` agree exactly, not just on membership, on every hand-picked and fuzzed case (Section 8.2).
- All four parser combinations agree on membership wherever the two constructions are not known to diverge, and each pair agrees exactly on tree shape within itself: `deriv_std` with `deriv_bc`, and `pderiv_std` with `pderiv_bc`. This is exercised directly by the CLI’s `--parser all` comparison mode (`src/cli/parser.rs`), which reports all three checks side by side for a given regex/input pair on demand.

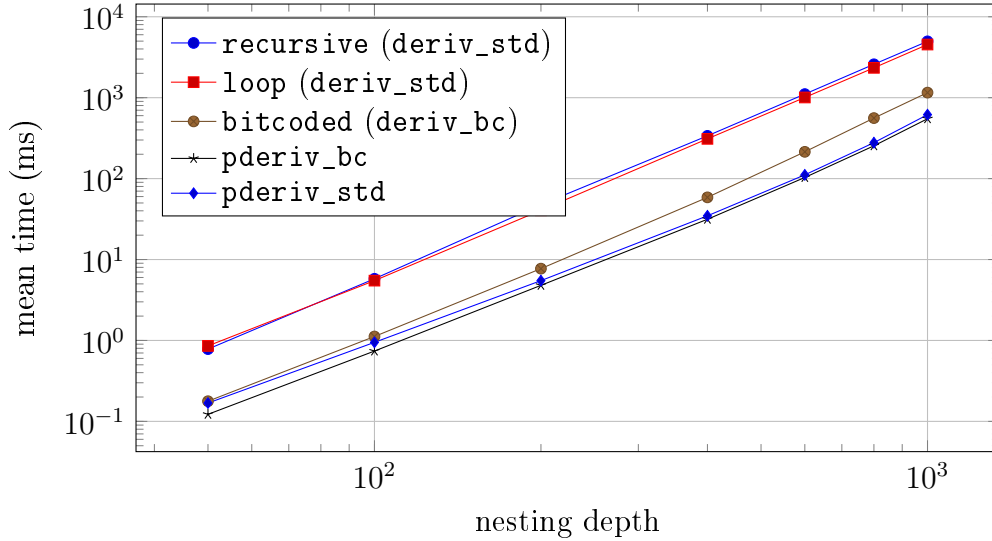
No disagreement has been observed in any of these checks against the current implementation; a regression in any one parser would show up first as a failure here, in the specific pairing that parser participates in, before it could show up as an incorrect answer with no diagnosis of which implementation is at fault.

8.5 Performance Analysis

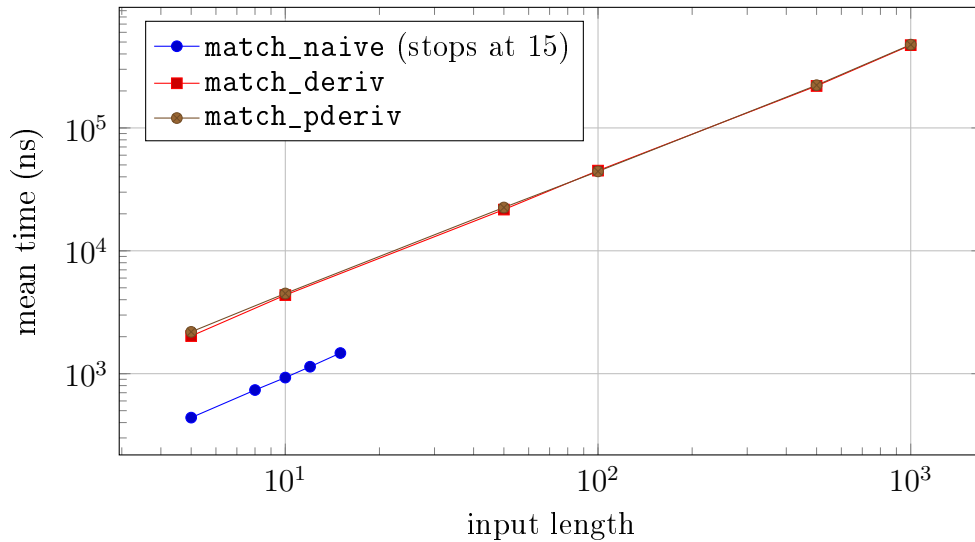
8.5.1 Stack Depth: Recursive vs. Iterative Construction

8.5.2 Heap Allocation: deriv_std vs. deriv_bc

8.5.3 Derivative-Based vs. Partial-Derivative-Based



8.5.4 Pathological Inputs



8.5.5 Benchmark Results

Parser	$n=5$	$n=10$	$n=15$	$n=18$
recursive (deriv_std)	8.0 μ s	110.7 μ s	1.427 ms	6.052 ms
bitcoded (deriv_bc)	11.4 μ s	183.8 μ s	4.175 ms	49.491 ms
pderiv_bc	13.4 μ s	210.6 μ s	2.207 ms	9.236 ms
pderiv_std	22.4 μ s	323.9 μ s	3.374 ms	14.263 ms

Parser	Mean time / sweep
<code>deriv_std_rec</code>	1.311 s
<code>deriv_std_loop</code>	1.091 s
<code>deriv_bc</code>	643.5 ms
<code>pderiv_bc</code>	159.2 ms
<code>pderiv_std</code>	158.6 ms

Parser	Mean time
<code>deriv_std_rec</code>	1.000 s
<code>deriv_bc</code>	81.1 ms

8.6 Comparison with Existing Engines

8.6.1 Performance

Engine	Mean time / sweep	vs. <code>regex</code>
<code>deriv_std_rec</code> (POSIX, recursive)	1.503 s	×1,415,254
<code>deriv_std_loop</code> (POSIX, loop)	1.218 s	×1,146,893
<code>deriv_bc</code> (POSIX, bit-coded)	690.74 ms	×650,417
<code>pderiv_bc</code> (partial derivative, bit-coded)	171.85 ms	×161,815
<code>pderiv_std</code> (partial derivative, tree)	173.28 ms	×163,167
<code>regex</code> (Rust crate)	1.06 μs	×1
<code>re2_perl</code> (leftmost-first)	6.61 μs	×6.2
<code>re2_posix</code> (leftmost-longest)	2.75 μs	×2.6

Engine	Mean time / sweep	vs. <code>regex</code>
<code>match_deriv</code>	273.07 ms	×259,815
<code>match_pderiv</code>	134.95 ms	×128,397
<code>regex</code> (Rust crate)	1.05 μs	×1
<code>re2_perl</code> (leftmost-first)	6.63 μs	×6.3
<code>re2_posix</code> (leftmost-longest)	2.72 μs	×2.6

8.6.2 Standardized Benchmarking via rebar

8.7 Optimization Challenges and Insights

Two optimizations were identified while implementing the four parsers. Neither is applied in the shipped code, and in both cases the reason is the same: the change is worth more than it costs only if correctness is measured against something other than the reference construction. They are recorded here with the measurements that establish their value, since those measurements are the product of this evaluation rather than of the implementation.

Character classes in place of literal alternation. The core algebra of Chapter 2 has six constructors and no character class among them, so the frontend lowers every class to an explicit alternation (Section 7.3). The cost of that decision is visible in the lowered node counts of Section 7.4: a `\d` becomes ten nodes, a `\w` becomes 125, and a wildcard becomes 195. The padding a search pattern receives is one `wildcard_run()` of 196 nodes

per unanchored side, so an unanchored three-letter literal reaches 399 nodes against the 5 its own content needs.

Every one of those nodes is structure the parser walks once per input character. A dedicated class constructor carrying a set of characters would collapse each of them to a single node whose derivative is one membership test, which changes the per-character work from linear in the size of the class to constant. Section 8.5’s dataset measurements are dominated by exactly this effect, since essentially every real-world pattern in the corpus is an unanchored search containing at least one class shorthand.

The optimization is not applied because it is not local. A seventh constructor has to be understood by `nullable`, `deriv`, `simp`, `mkEps`, `inj`, the annotated `ARegex` and its refined derivative, `pDerivBC`, and `decode`, which is every definition in Chapters 5 and 6. The four parsers would then no longer be transcriptions of the constructions they are transcriptions of, which is the property the correctness argument of Section 8.4 rests on. There is a second benefit worth recording alongside the performance one: because the expected set reported on a failure is read off the first-literal set of the expression, a mismatch against a lowered wildcard currently reports all 98 alternatives individually, where a native class could report `expected: any character`.

Frontier-wide deduplication. The partial-derivative parsers apply `nub2` inside a single expansion and never across the flattened frontier, which is what lets the frontier double on the expressions of Section 6.5 while the number of distinct residuals stays bounded. Moving that deduplication to the frontier as a whole is a change of one call site. Measured over the same 11700 cases as Section 8.4.2, it changes no answer, on membership or on the tree returned, and takes the peak frontier from 170 strands to 4.

It is not applied for the reason given in Section 6.2: this parser’s correctness criterion is literal agreement with the reference `pDerivBC`, and a change verified on finitely many cases without a proof is a change to the thing being verified against. The measurement establishes that the optimization is available and what it is worth; adopting it would require replacing agreement with an argument.

8.8 Discussion of Results

9 Related Work and Conclusion

9.1 Related Work

9.2 Conclusion

Bibliography

- Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.
- Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4): 481–494, 1964. doi: 10.1145/321239.321249.
- Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). <https://swtch.com/~rsc/regexp/regexp1.html>, 2007. Accessed: 2026-08-05.
- Alain Frisch and Luca Cardelli. Greedy regular expression matching. In *Automata, Languages and Programming (ICALP 2004)*, volume 3142, pages 618–629. Springer, 2004. doi: 10.1007/978-3-540-27836-8_53.
- Google LLC. RE2: A fast, safe, POSIX-compatible regular expression library. <https://github.com/google/re2>, 2024. Accessed: 2026-08-05.
- IEEE and The Open Group. Posix.1-2017: Base definitions, chapter 9 (regular expressions), 2017. IEEE Std 1003.1-2017.
- Konstantinos Mamouras, Alexis Le Glaunec, Wu Angela Li, and Agnishom Chattopadhyay. Static analysis for checking the disambiguation robustness of regular expressions. *Proceedings of the ACM on Programming Languages*, 8(PLDI):231, 2024. doi: 10.1145/3656461.
- Martin Sulzmann. Programming paradigms: Regular expressions. <https://sulzmann.github.io/ProgrammingParadigms/pp-regular-expressions.html>, 2026. Accessed: 2026-08-08.
- Martin Sulzmann and Kenny Zhuo Ming Lu. Regular expression sub-matching using partial derivatives. In *PPDP*, pages 79–90, 2012. doi: 10.1145/2370776.2370788.
- Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.
- The Rust Project Developers. Using `Box<T>` to point to data on the heap. The Rust Programming Language, 2026a. URL <https://doc.rust-lang.org/book/ch15-01-box.html>. Accessed: 2026-09-11.

- The Rust Project Developers. Derivable traits. The Rust Programming Language, 2026b. URL <https://doc.rust-lang.org/book/appendix-03-derivable-traits.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Enums and pattern matching. The Rust Programming Language, 2026c. URL <https://doc.rust-lang.org/book/ch06-00-enums.html>. Accessed: 2026-09-11.
- The Rust Project Developers. The `match` control flow construct. The Rust Programming Language, 2026d. URL <https://doc.rust-lang.org/stable/book/ch06-02-match.html>. Accessed: 2026-09-11.
- The Rust Project Developers. `std::hash`. Rust Standard Library, 2026e. URL <https://doc.rust-lang.org/std/hash/index.html>. Accessed: 2026-09-11.
- The Rust Project Developers. `HashMap` in `std::collections`. Rust Standard Library, 2026f. URL <https://doc.rust-lang.org/std/collections/struct.HashMap.html>. Accessed: 2026-09-11.
- The Rust Project Developers. `HashSet` in `std::collections`. Rust Standard Library, 2026g. URL <https://doc.rust-lang.org/std/collections/struct.HashSet.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Enumerated types. The Rust Reference, 2026h. URL <https://doc.rust-lang.org/reference/types/enum.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Match expressions. The Rust Reference, 2026i. URL <https://doc.rust-lang.org/reference/expressions/match-expr.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Types. The Rust Reference, 2026j. URL <https://doc.rust-lang.org/reference/types.html>. Accessed: 2026-09-11.
- Ian Erik Varatalu, Margus Veanes, and Juhan-Peep Ernits. Derivative based extended regular expression matching supporting intersection, complement and lookarounds. <https://arxiv.org/abs/2309.14401>, 2023. Accessed: 2026-09-18.

A Code

A.1 Core Types

Regex

src/types/regex.rs

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    /// Empty language:  $L(\Phi) = \{\}$ 
    Phi,

    /// Empty word:  $L(\epsilon) = \{\epsilon\}$ 
    Eps,

    /// Single character:  $L(\text{Lit}(c)) = \{c\}$ 
    Lit(char),

    /// Sequence:  $L(\text{Seq}(r,s)) = \{v++w \mid v \in L(r), w \in L(s)\}$ 
    Seq(Box<Regex>, Box<Regex>),

    /// Alternative:  $L(\text{Alt}(r,s)) = L(r) \cup L(s)$ 
    Alt(Box<Regex>, Box<Regex>),

    /// Kleene star:  $L(\text{Star}(r)) = \{\epsilon\} \cup L(r).L(\text{Star}(r))$ 
    Star(Box<Regex>),
}
```

ARegex

src/types/aregex.rs

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ARegex {
    /// Empty language
    Phi,
    ///  $bs @ \epsilon$ 
    Eps(Vec<bool>),
    ///  $bs @ 1$ 
    Lit(Vec<bool>, char),
    ///  $bs @ (ri1 (+) ri2)$ 
    Alt(Vec<bool>, Box<ARegex>, Box<ARegex>),
    ///  $bs @ (ri1 ri2)$ 
    Seq(Vec<bool>, Box<ARegex>, Box<ARegex>),
    ///  $bs @ ri^*$ 
    Star(Vec<bool>, Box<ARegex>),
}
```

ParseTree

src/types/tree.rs

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseTree {
    /// Empty parse tree (for epsilon)
    Empty,

    /// Literal character
    Char(char),
}
```

```

    /// Pair for concatenation (v1, v2)
    Pair(Box<ParseTree>, Box<ParseTree>),

    /// Left injection for alternative (Left v)
    Left(Box<ParseTree>),

    /// Right injection for alternative (Right v)
    Right(Box<ParseTree>),

    /// List for Kleene star iterations [v1, v2, ..., vn]
    Star(Vec<ParseTree>),
}

```

flatten
src/types/tree.rs

```

/// Flattens parse tree to underlying word (string)
pub fn flatten(v: &ParseTree) -> String {
    match v {
        ParseTree::Empty => String::new(),
        ParseTree::Char(c) => c.to_string(),
        ParseTree::Pair(v1, v2) => format!("{}", flatten(v1), flatten(v2)),
        ParseTree::Left(v) => flatten(v),
        ParseTree::Right(v) => flatten(v),
        ParseTree::Star(vs) => vs.iter().map(flatten).collect(),
    }
}

```

A.2 Shared Algorithms

nullable
src/regex/nullable.rs

```

/// Decides whether epsilon is in L(r)
pub fn nullable(r: &Regex) -> bool {
    match r {
        Regex::Phi => false,
        Regex::Eps => true,
        Regex::Lit(_) => false,
        Regex::Alt(r, s) => nullable(r) || nullable(s),
        Regex::Seq(r, s) => nullable(r) && nullable(s),
        Regex::Star(_) => true,
    }
}

```

deriv
src/regex/deriv.rs

```

/// Computes derivative of r based on character x
pub fn deriv(r: &Regex, x: char) -> Regex {
    match r {
        Regex::Phi => Regex::Phi,
        Regex::Eps => Regex::Phi,
        Regex::Lit(c) => {
            if *c == x { Regex::Eps } else { Regex::Phi }
        }
        Regex::Alt(r1, r2) => {
            Regex::alt(deriv(r1, x), deriv(r2, x))
        }
        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let dr1 = Regex::seq(deriv(r1, x), *r2.clone());
                Regex::alt(dr1, deriv(r2, x))
            } else {
                Regex::seq(deriv(r1, x), *r2.clone())
            }
        }
        Regex::Star(r1) => {
            Regex::seq(deriv(r1, x), Regex::star(*r1.clone()))
        }
    }
}

```

```
}
```

mk_eps
src/regex/mk_eps.rs

```
pub fn mk_eps(r: &Regex) -> ParseTree {
    match r {
        Regex::Eps => ParseTree::Empty,

        Regex::Star(_) => ParseTree::Star(Vec::new()),

        Regex::Seq(r1, r2) => {
            let v1 = mk_eps(r1);
            let v2 = mk_eps(r2);
            ParseTree::Pair(Box::new(v1), Box::new(v2))
        }

        Regex::Alt(r1, r2) => {
            if nullable(r1) {
                ParseTree::Left(Box::new(mk_eps(r1)))
            } else {
                ParseTree::Right(Box::new(mk_eps(r2)))
            }
        }

        Regex::Lit(c) => panic!("mk_eps called on Lit('{ }')", c),
        Regex::Phi => panic!("mk_eps called on Phi"),
    }
}
```

decode
src/regex/decode.rs

```
pub fn decode(r: &Regex, bs: &[bool]) -> ParseTree {
    let (v, rest) = decode_inner(r, bs);
    assert!(
        rest.is_empty(),
        "decode: {} leftover bit(s) after decoding {:?}",
        rest.len(),
        r
    );
    v
}
```

simplify
src/regex/simplify.rs

```
/// Simplify regular expression using algebraic laws
pub fn simplify(r: Regex) -> Regex {
    match r {
        Regex::Seq(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) => Regex::Phi,
                (_, Regex::Phi) => Regex::Phi,
                (Regex::Eps, _) => s,
                (_, Regex::Eps) => r,
                _ => Regex::seq(r, s),
            }
        }
        Regex::Alt(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) => s,
                (_, Regex::Phi) => r,
                _ if r == s => r,
                _ => Regex::alt(r, s),
            }
        }
        Regex::Star(r) => {
            let r = simplify(*r);

```

```

        match r {
            Regex::Eps => Regex::Eps,
            Regex::Phi => Regex::Eps,
            r => Regex::star(r),
        }
    }
    other => other,
}
}

```

A.3 Matchers

match_naive
src/matchers/match_naive.rs

```

pub fn match_naive(input: &str, r: &Regex) -> bool {
    let chars: Vec<char> = input.chars().collect();
    let len = chars.len();
    match_naive_range(&chars, 0, len, r)
}

```

match_deriv
src/matchers/match_deriv.rs

```

pub fn match_deriv(input: &str, r: &Regex) -> bool {
    let mut current = r.clone();
    for c in input.chars() {
        current = simplify(deriv(&current, c));
    }
    nullable(&current)
}

```

match_pderiv
src/matchers/match_pderiv.rs

```

pub fn match_pderiv(input: &str, r: &Regex) -> bool {
    let mut states: HashSet<Regex> = HashSet::new();
    states.insert(r.clone());

    for c in input.chars() {
        let mut next_states: HashSet<Regex> = HashSet::with_capacity(states.len());
        for state in &states {
            next_states.extend(pderiv(state, c));
        }
        if next_states.is_empty() {
            return false;
        }
        states = next_states;
    }

    states.iter().any(nullable)
}

```

A.4 The deriv_std Parser

inject
src/parsers/deriv_std/inject.rs

```

pub fn inject(r: &Regex, l: char, v: ParseTree) -> ParseTree {
    match r {
        Regex::Lit(c) => {
            assert!(*c == l);
            assert!(matches!(v, ParseTree::Empty));
            ParseTree::Char(l)
        }

        Regex::Star(r1) => {
            match v {

```

```

    ParseTree::Pair(v1, vs) => {
        let v1_inj = inject(r1, 1, *v1);
        let mut iterations = vec![v1_inj];
        if let ParseTree::Star(rest) = *vs {
            iterations.extend(rest);
        }
        ParseTree::Star(iterations)
    }
    _ => panic!("inject on Star: expected Pair, got {:?}", v),
}

Regex::Seq(r1, r2) => {
    let r1_nullable = nullable(r1);

    if !r1_nullable {
        match v {
            ParseTree::Pair(v1, v2) => {
                let v1_inj = inject(r1, 1, *v1);
                ParseTree::Pair(Box::new(v1_inj), v2)
            }
            _ => panic!("inject on Seq (non-nullable r1): expected Pair, got {:?}", v),
        }
    } else {
        match v {
            ParseTree::Left(v_pair) => {
                match *v_pair {
                    ParseTree::Pair(v1, v2) => {
                        let v1_inj = inject(r1, 1, *v1);
                        ParseTree::Pair(Box::new(v1_inj), v2)
                    }
                    _ => panic!(
                        "inject on Seq (nullable r1, Left): expected Pair, got {:?}",
                        v_pair
                    ),
                }
            }
            ParseTree::Right(v2) => {
                let v2_inj = inject(r2, 1, *v2);
                let v1_eps = mk_eps(r1);
                ParseTree::Pair(Box::new(v1_eps), Box::new(v2_inj))
            }
            _ => panic!("inject on Seq (nullable r1): expected Left or Right, got {:?}", v),
        }
    }
}

Regex::Alt(r1, r2) => {
    match v {
        ParseTree::Left(v1) => {
            ParseTree::Left(Box::new(inject(r1, 1, *v1)))
        }
        ParseTree::Right(v2) => {
            ParseTree::Right(Box::new(inject(r2, 1, *v2)))
        }
        _ => panic!("inject on Alt: expected Left or Right, got {:?}", v),
    }
}

Regex::Eps => panic!("inject called on Eps"),
Regex::Phi => panic!("inject called on Phi"),
}
}

```

A.5 The deriv_bc Parser

`src/parsers/deriv_bc/internalize.rs`

```

pub fn fuse(bs: &[bool], ri: ARegex) -> ARegex {
    if bs.is_empty() { return ri; }
}

```

```

match ri {
  ARegex::Phi => ARegex::Phi,
  ARegex::Eps(p) => ARegex::Eps(combine(bs, p)),
  ARegex::Lit(p, c) => ARegex::Lit(combine(bs, p), c),
  ARegex::Alt(p, r1, r2) => ARegex::Alt(combine(bs, p), r1, r2),
  ARegex::Seq(p, r1, r2) => ARegex::Seq(combine(bs, p), r1, r2),
  ARegex::Star(p, r) => ARegex::Star(combine(bs, p), r),
}
}

```

deriv_bc
src/parsers/deriv_bc/deriv.rs

```

/// Bit-coded derivative (paper Figure 5)
pub fn deriv_bc(ri: ARegex, l: char) -> ARegex {
  match ri {
    ARegex::Phi => ARegex::Phi,
    ARegex::Eps(_) => ARegex::Phi,
    ARegex::Lit(bs, c) => {
      if c == 1 {
        ARegex::Eps(bs)
      } else {
        ARegex::Phi
      }
    }
    ARegex::Alt(bs, r1, r2) => {
      let d1 = deriv_bc(*r1, l);
      let d2 = deriv_bc(*r2, l);
      ARegex::Alt(bs, Box::new(d1), Box::new(d2))
    }
    ARegex::Seq(bs, r1, r2) => {
      if nullable_bc(&r1) {
        let eps_bits = mk_eps_bc(&r1);
        let d1 = deriv_bc(*r1, l);
        let d2 = deriv_bc(*r2.clone(), l);
        let left_branch = ARegex::Seq(vec![], Box::new(d1), r2);
        let right_branch = fuse(&eps_bits, d2);
        ARegex::Alt(bs, Box::new(left_branch), Box::new(right_branch))
      } else {
        let d1 = deriv_bc(*r1, l);
        ARegex::Seq(bs, Box::new(d1), r2)
      }
    }
    // Empty star prefix: bs = []; Inner exp: r = Box<Lit([], 'a')>
    ARegex::Star(bs, r) => {
      let d = deriv_bc(*r.clone(), l); // ri\'a' = Lit([], 'a')\'a' = Eps([])
      let fused = fuse(&[false], d); // fuse [0] Eps([]) = Eps([false])
      let new_star = ARegex::Star(vec![], r); // []@ri* = []@([]@'a')*
      ARegex::Seq(bs, Box::new(fused), Box::new(new_star))
      // = []@Eps([false]) . []@([]@'a')*
    }
  }
}

```

simp
src/parsers/deriv_bc/simplify.rs

```

/// Simplify an annotated expression to a fixpoint.
pub fn simp(ri: ARegex) -> ARegex {
  simp_once(ri)
}

```

A.6 The pderiv_bc Parser

pderiv_bc
src/parsers/pderiv_bc/pderiv.rs

```

/// Bit-coded partial derivative (pDerivBC): one '(residual, bits)' pair per surviving strand.
pub fn pderiv_bc(r: &Regex, x: char) -> Vec<(Regex, Vec<bool>)> {
  match r {

```

```

Regex::Eps => Vec::new(),
Regex::Phi => Vec::new(),

Regex::Lit(y) => {
  if *y == x {
    vec![(Regex::Eps, Vec::new())]
  } else {
    Vec::new()
  }
}

Regex::Alt(r1, r2) => {
  let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
    .into_iter()
    .map(|(r1_, mut bs)| {
      let mut b = vec![false];
      b.append(&mut bs);
      (r1_, b)
    })
    .collect();
  out.extend(
    pderiv_bc(r2, x)
      .into_iter()
      .map(|(r2_, mut bs)| {
        let mut b = vec![true];
        b.append(&mut bs);
        (r2_, b)
      })
  );
  nub2(out)
}

Regex::Seq(r1, r2) => {
  if nullable(r1) {
    // Strand 1: continue r1, r2 untouched (not smart-constructed,
    // matching the reference literally)
    let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
      .into_iter()
      .map(|(r1_, bs)| (Regex::seq(r1_, *r2.clone()), bs))
      .collect();

    // Strand 2: r1 matched empty, jump straight into r2.
    let eps_bits = mk_eps_bits(r1);
    out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, bs)| {
      let mut b = eps_bits.clone();
      b.extend(bs);
      (r2_, b)
    }));

    nub2(out)
  } else {
    pderiv_bc(r1, x)
      .into_iter()
      .map(|(r1_, bs)| (smart_seq(r1_, r2), bs))
      .collect()
  }
}

Regex::Star(r1) => {
  let out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
    .into_iter()
    .map(|(r1_, mut bs)| {
      let mut b = vec![false];
      b.append(&mut bs);
      (smart_seq(r1_, &Regex::star(*r1.clone())), b)
    })
    .collect();
  nub2(out)
}
}

```

nub2

src/parsers/pderiv_bc/deriv.rs

```
/// Stable dedup on the residual, keeping the first (highest-priority)
fn nub2(pairs: Vec<Regex, Vec<bool>>>) -> Vec<Regex, Vec<bool>> {
    let mut seen: HashSet<Regex> = HashSet::with_capacity(pairs.len());
    let mut out = Vec::with_capacity(pairs.len());
    for (r, bs) in pairs {
        if seen.insert(r.clone()) {
            out.push((r, bs));
        }
    }
    out
}
```

step_frontier.
src/parsers/pderiv_bc/parse.rs

```
/// One step of 'parsePDerivBC2': replace the frontier with union of 'pderiv_bc(x, .)'
pub(super) fn step_frontier(frontier: &[(Regex, Vec<bool>)], x: char) -> Vec<Regex, Vec<bool>> {
    let mut out = Vec::new();
    for (r, bits) in frontier {
        for (r_next, extra) in pderiv_bc(r, x) {
            let mut combined = bits.clone();
            combined.extend(extra);
            out.push((r_next, combined));
        }
    }
    out
}
```

select_bits
src/parsers/pderiv_bc/parse.rs

```
/// 'filter ((r,_) -> nullable r) rs', take head, and append 'mkEpsBC'.
pub(super) fn select_bits(frontier: &[(Regex, Vec<bool>)]) -> Option<Vec<bool>> {
    for (r, bits) in frontier {
        if nullable(r) {
            let mut result = bits.clone();
            result.extend(mk_eps_bits(r));
            return Some(result);
        }
    }
    None
}
```

parse_pderiv_bc
src/parsers/pderiv_bc/parse.rs

```
pub fn parse_pderiv_bc(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier = vec![(r.clone(), Vec::new())];
    for c in input.chars() {
        frontier = step_frontier(&frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    let bits = select_bits(&frontier)?;
    Some(decode(r, &bits))
}
```

A.7 The pderiv_std Parser

Inj
src/parsers/pderiv_std/inject.rs

```
/// Reconstructs a strand's 'ParseTree'; 'Rc' since a fan-out step (e.g. 'Alt') shares one parent injection
pub(crate) type Inj = Rc<dyn Fn(ParseTree) -> ParseTree>;
```

```
pub(crate) fn identity_inj() -> Inj {
    Rc::new(|v| v)
}
```

nub_tree
src/parsers/pderiv_std/inject.rs

```
/// Dedup on the residual alone, keeping the first (highest-priority, GREEDY-ordered)
pub(crate) fn nub_tree(strands: Vec<(Regex, Inj)> -> Vec<(Regex, Inj)> {
    let mut seen: HashSet<Regex> = HashSet::with_capacity(strands.len());
    let mut out = Vec::with_capacity(strands.len());
    for (r, inj) in strands {
        if seen.insert(r.clone()) {
            out.push((r, inj));
        }
    }
    out
}
```

pderiv_tree
src/parsers/pderiv_std/pderiv_tree.rs

```
pub(crate) fn pderiv_tree(r: &Regex, x: char) -> Vec<(Regex, Inj)> {
    match r {
        Regex::Eps | Regex::Phi => vec![],

        Regex::Lit(c) => {
            if *c == x {
                vec![(Regex::Eps, Rc::new(move |_| ParseTree::Char(x)) as Inj)]
            } else {
                vec![]
            }
        }

        Regex::Alt(r1, r2) => {
            let mut out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
                .into_iter()
                .map(|(r1_, inj1)| {
                    let f: Inj = Rc::new(move |v| ParseTree::Left(Box::new(inj1(v))));
                    (r1_, f)
                })
                .collect();
            out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                let f: Inj = Rc::new(move |v| ParseTree::Right(Box::new(inj2(v))));
                (r2_, f)
            }));
            nub_tree(out)
        }

        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let mut out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
                    .into_iter()
                    .map(|(r1_, inj1)| {
                        let r2c = (**r2).clone();
                        let f: Inj = Rc::new(move |v: ParseTree| match v {
                            ParseTree::Pair(v1, v2) => ParseTree::Pair(Box::new(inj1(*v1)), v2),
                            other => panic!(
                                "pderiv_tree: Seq-continue strand expected Pair, got {:?}",
                                other
                            ),
                        },
                    ),
                    (Regex::seq(r1_, r2c), f)
                })
                    .collect();

                let v1_eps = mk_eps(r1);
                out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                    let v1c = v1_eps.clone();
                    let f: Inj = Rc::new(move |v2: ParseTree| {
                        ParseTree::Pair(Box::new(v1c.clone()), Box::new(inj2(v2)))
                    });
                    (r2_, f)
                }));

                nub_tree(out)
            } else {
                pderiv_tree(r1, x)
            }
        }
    }
}
```

```

        .into_iter()
        .map(|(r1_, inj1)| {
            let r2c = (**r2).clone();
            let f: Inj = Rc::new(move |v: ParseTree| match v {
                ParseTree::Pair(v1, v2) => ParseTree::Pair(Box::new(inj1(*v1)), v2),
                other => panic!(
                    "pderiv_tree: Seq(non-nullable) strand expected Pair, got {:?}",
                    other
                ),
            });
            (Regex::seq(r1_, r2c), f)
        })
        .collect()
    }
}

Regex::Star(r1) => {
    let out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
        .into_iter()
        .map(|(r1_, inj1)| {
            let r1c = (**r1).clone();
            let f: Inj = Rc::new(move |v: ParseTree| match v {
                ParseTree::Pair(v1, vs) => match *vs {
                    ParseTree::Star(mut items) => {
                        items.insert(0, inj1(*v1));
                        ParseTree::Star(items)
                    }
                    other => panic!(
                        "pderiv_tree: Star strand expected Star tail, got {:?}",
                        other
                    ),
                },
                other => panic!(
                    "pderiv_tree: Star strand expected Pair, got {:?}",
                    other
                ),
            });
            (Regex::seq(r1_, Regex::star(r1c)), f)
        })
        .collect();
    nub_tree(out)
}
}
}
}

```

parse_pderiv_std
src/parsers/pderiv_std/parse.rs

```

pub fn parse_pderiv_std(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier: Vec<(Regex, Inj)> = vec![(r.clone(), identity_inj())];
    for c in input.chars() {
        frontier = step_frontier(frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    select(&frontier)
}

```

A.8 Frontend

ExtPat
src/frontend/ext_pattern.rs

```

#[derive(Debug, Clone, PartialEq)]
pub enum ExtPat {
    /// the empty string, 'translate's to 'Regex::Eps'.
    Empty,
    Eps,
    /// Literal 'phi' token 'translate's to 'Regex::Phi'.
    Never,
}

```

```

    /// A marking group '( ... )'.
    Group(Box<ExtPat>),
    /// A non-marking group '(?: ... )'.
    GroupNonMarking(Box<ExtPat>),
    /// Alternation 'a|b|c'.
    Or(Vec<ExtPat>),
    /// Concatenation 'ab c'.
    Concat(Vec<ExtPat>),
    /// 'r?' / 'r??' (bool: greedy).
    Opt(Box<ExtPat>, bool),
    /// 'r+' / 'r+?' (bool: greedy).
    Plus(Box<ExtPat>, bool),
    /// 'r*' / 'r*?' (bool: greedy).
    Star(Box<ExtPat>, bool),
    /// 'r{lo,hi}' / 'r{lo,}' / 'r{lo}' (bool: greedy).
    Bound(Box<ExtPat>, u32, Option<u32>, bool),
    /// '^'.
    Carat,
    /// '$'.
    Dollar,
    /// '.'.
    Dot,
    /// '[ ... ]' listed characters (ranges and '\d'/'\w'/'\s'
    /// shorthands already expanded by the parser).
    Any(Vec<char>),
    /// '[^ ... ]'.
    NoneOf(Vec<char>),
    /// '\c' for a 'c' that carries regex meaning to 'translate'
    /// ('d'/'D'/'w'/'W'/'s'/'S') or, for any other 'c', an escaped literal
    /// ('\.', '\(', '\\', ...).
    Escape(char),
    /// An ordinary, unescaped literal character.
    Char(char),
}

```

translate

src/frontend/translate.rs

```

/// Lower an 'ExtPat' into this crate's core 'Regex'.
pub fn translate(ep: &ExtPat) -> Result<Regex, String> {
    match ep {
        ExtPat::Empty => Ok(Regex::Eps),
        ExtPat::Eps => Ok(Regex::Eps),
        ExtPat::Never => Ok(Regex::Phi),
        ExtPat::Group(inner) | ExtPat::GroupNonMarking(inner) => translate(inner),
        ExtPat::Or(alts) => {
            let mut iter = alts.iter();
            let mut acc = translate(iter.next().ok_or("empty alternation")??);
            for a in iter {
                acc = Regex::alt(acc, translate(a)?);
            }
            Ok(acc)
        }
        ExtPat::Concat(parts) => {
            let mut iter = parts.iter();
            let mut acc = match iter.next() {
                Some(p) => translate(p)?,
                None => return Ok(Regex::Eps),
            };
            for p in iter {
                acc = Regex::seq(acc, translate(p)?);
            }
            Ok(acc)
        }
        ExtPat::Opt(inner, _greedy) => Ok(Regex::alt(translate(inner)?, Regex::Eps)),
        ExtPat::Plus(inner, _greedy) => {
            let r = translate(inner)?;
            Ok(Regex::seq(r.clone(), Regex::star(r)))
        }
        ExtPat::Star(inner, _greedy) => Ok(Regex::star(translate(inner)?)),
        ExtPat::Bound(inner, lo, hi, _greedy) => desugar_bound(translate(inner)?, *lo, *hi),
        ExtPat::Carat | ExtPat::Dollar => Ok(Regex::Eps),
        ExtPat::Dot => alt_of_chars(alphabet::alphabet()),
    }
}

```

```
ExtPat::Any(cs) => alt_of_chars(cs.clone()),
ExtPat::NoneOf(cs) => alt_of_chars(alphabet::complement(cs)),
ExtPat::Escape(c) => translate_escape(*c),
ExtPat::Char(c) => Ok(Regex::lit(*c)),
    }
}
```

B Tests

B.1 Shared Fixtures

paper_r1
tests/common/mod.rs

```
pub fn paper_r1() -> Regex {
    Regex::seq(
        Regex::alt(Regex::lit('a'), Regex::seq(Regex::lit('a'), Regex::lit('b'))),
        Regex::alt(Regex::lit('b'), Regex::Eps),
    )
}
```

paper_r2
tests/common/mod.rs

```
/// (a + (b + ab))* - POSIX star-preference example from flops14 p.1
/// Structure: Star(Alt(a, Alt(b, ab))) - nested Alt, ab is Right(Right(...))
pub fn paper_r2() -> Regex {
    Regex::star(Regex::alt(
        Regex::lit('a'),
        Regex::alt(
            Regex::lit('b'),
            Regex::seq(Regex::lit('a'), Regex::lit('b')),
        ),
    ))
}
```

assert_round_trip
tests/common/mod.rs

```
#[allow(dead_code)]
pub fn assert_round_trip(tree: &ParseTree, expected_word: &str) {
    assert_eq!(
        flatten(tree), expected_word,
        "flatten(tree) should equal the matched word"
    );
}
```

assert_parsers_agree
tests/common/mod.rs

```
#[allow(dead_code)]
pub fn assert_parsers_agree(
    label_a: &str,
    tree_a: &Option<ParseTree>,
    label_b: &str,
    tree_b: &Option<ParseTree>,
) {
    assert_eq!(
        tree_a, tree_b,
        "{} and {} disagree:\n {}: {:?}\n {}: {:?}",
        label_a, label_b, label_a, tree_a, label_b, tree_b
    );
}
```

B.2 Paper Examples Reproduced

`parse_recursive_paper_r1_on_ab`
tests/test_deriv_std.rs

```
/// (a + ab)(b + eps) on "ab": A1 picks the longer "ab" branch on the left factor.
#[test]
fn parse_recursive_paper_r1_on_ab() {
    let r = paper_r1();
    let tree = parse_deriv_std_rec("ab", &r).expect("should match");
    assert_round_trip(&tree, "ab");

    assert!(
        matches!(&tree, ParseTree::Pair(_, _)),
        "outer should be Pair (regex is Seq), got {:?}", tree
    );

    if let ParseTree::Pair(left, right) = &tree {
        assert!(
            matches!(left.as_ref(), ParseTree::Right(_)),
            "left factor should be Right (ab branch chosen by A1), got {:?}", left
        );
        assert!(
            matches!(right.as_ref(), ParseTree::Right(_)),
            "right factor should be Right (eps branch, only valid choice), got {:?}", right
        );
    }
}
```

`parse_recursive_paper_r2_on_ab`
tests/test_deriv_std.rs

```
/// (a + b + ab)* on "ab": A1 picks one "ab" iteration over Greedy's two-iteration answer.
#[test]
fn parse_recursive_paper_r2_on_ab() {
    let r = paper_r2();
    let tree = parse_deriv_std_rec("ab", &r).expect("should match");
    assert_round_trip(&tree, "ab");

    if let ParseTree::Star(ref iters) = tree {
        assert_eq!(iters.len(), 1,
            "POSIX should pick 'ab' as one star iteration (A1), got {:?}", iters);

        assert!(
            matches!(&iters[0], ParseTree::Right(inner)
                if matches!(inner.as_ref(), ParseTree::Right(innermost)
                    if matches!(innermost.as_ref(), ParseTree::Pair(_, _)))),
            "iteration should be Right(Right(Pair(a,b))), got {:?}", iters[0]
        );
    } else {
        panic!("expected Star, got {:?}", tree);
    }
}
```

`paper_r1_computes_greedy_not_posix`
src/parsers/pderiv_bc/parse.rs

```
#[test]
fn paper_r1_computes_greedy_not_posix() {
    // (a + ab)(b + eps) on "ab"
    let r = Regex::seq(
        Regex::alt(Regex::lit('a'), Regex::seq(Regex::lit('a'), Regex::lit('b'))),
        Regex::alt(Regex::lit('b'), Regex::Eps),
    );

    let posix = parse_deriv_std_rec("ab", &r);
    let greedy = parse_pderiv_bc("ab", &r);

    assert!(posix.is_some() && greedy.is_some(), "both should match");
    assert_eq!(flatten(posix.as_ref().unwrap()), "ab");
    assert_eq!(flatten(greedy.as_ref().unwrap()), "ab");
}
```

```

// POSIX: the longer "ab" alternative wins (rule A1)
assert_eq!(
  posix,
  Some(ParseTree::Pair(
    Box::new(ParseTree::Right(Box::new(ParseTree::Pair(
      Box::new(ParseTree::Char('a')),
      Box::new(ParseTree::Char('b')),
    )))),
    Box::new(ParseTree::Right(Box::new(ParseTree::Empty))),
  ))
);
// Greedy: the leftmost alternative 'a' wins, committed to immediately
assert_eq!(
  greedy,
  Some(ParseTree::Pair(
    Box::new(ParseTree::Left(Box::new(ParseTree::Char('a')))),
    Box::new(ParseTree::Left(Box::new(ParseTree::Char('b')))),
  ))
);
assert_ne!(posix, greedy, "this is exactly the documented divergence");
}

```

paper_r2_computes_greedy_not_posix

src/parsers/pderiv_bc/parse.rs

```

#[test]
fn paper_r2_computes_greedy_not_posix() {
  // (a + (b + ab))* on "ab"
  let r = Regex::star(Regex::alt(
    Regex::lit('a'),
    Regex::alt(Regex::lit('b'), Regex::seq(Regex::lit('a'), Regex::lit('b'))),
  ));

  let posix = parse_deriv_std_rec("ab", &r);
  let greedy = parse_pderiv_bc("ab", &r);

  assert!(posix.is_some() && greedy.is_some(), "both should match");
  assert_eq!(flatten(posix.as_ref().unwrap()), "ab");
  assert_eq!(flatten(greedy.as_ref().unwrap()), "ab");

  // POSIX: one iteration, the longer "ab" alternative wins (rule A1)
  assert_eq!(
    posix,
    Some(ParseTree::Star(vec! [ParseTree::Right(Box::new(ParseTree::Right(
      Box::new(ParseTree::Pair(
        Box::new(ParseTree::Char('a')),
        Box::new(ParseTree::Char('b')),
      ))
    ))]))
  );
  // Greedy: two iterations, leftmost alternative wins at each step
  assert_eq!(
    greedy,
    Some(ParseTree::Star(vec! [
      ParseTree::Left(Box::new(ParseTree::Char('a'))),
      ParseTree::Right(Box::new(ParseTree::Left(Box::new(ParseTree::Char('b'))))),
    ]))
  );
  assert_ne!(posix, greedy, "this is exactly the documented divergence");
}

```

B.3 Cross-Implementation Equivalence

recursive_traced_and_loop_traced_agree_on_all_paper_examples

tests/test_deriv_std.rs

```

#[test]
fn recursive_traced_and_loop_traced_agree_on_all_paper_examples() {
  use regex_engine::parsers::{parse_recursive_traced, parse_loop_traced};

```

```

let cases: Vec<(&str, Regex)> = vec![
    ("aaa", Regex::star(Regex::lit('a'))),
    ("aab", Regex::star(Regex::lit('a'))),
    ("ab", paper_r1()),
    ("a", paper_r1()),
    ("ab", paper_r2()),
    ("", Regex::star(Regex::lit('a'))),
];

for (input, r) in &cases {
    let (rec_tree, _) = parse_recursive_traced(input, r);
    let (loop_tree, _) = parse_loop_traced(input, r);
    assert_eq!(
        rec_tree, loop_tree,
        "recursive_traced and loop_traced disagree on {:?}", input
    );
}

}

bitcoded_agrees_with_recursive
tests/test_deriv_bc.rs

fn bitcoded_agrees_with_recursive(input: &str, r: &Regex) {
    let rec = parse_deriv_std_rec(input, r);
    let bc = parse_deriv_bc(input, r);
    assert_parsers_agree("recursive", &rec, "bitcoded", &bc);
}

```

B.4 Frontend

unanchored_nullable_pattern_matches_anything
src/frontend/mod.rs

```

#[test]
fn unanchored_nullable_pattern_matches_anything() {
    let r = parse_dataset_pattern("a*").expect("should parse");
    assert!(parse_deriv_std_rec("", &r).is_some());
    assert!(parse_deriv_std_rec("xyz", &r).is_some());
    assert!(parse_deriv_std_rec("zzz999", &r).is_some());
}

```

C Diagnostics

```
cargo run -- parse "(a|ab)(b|)" <INPUT> --parser <PARSER> --diag 3
```

Field	Value
Expression	"(a ab)(b)", that is $(a + ab)(b + \varepsilon)$
Inputs	"ab" (match), "b" (no match)
Parsers	deriv_std_rec, deriv_bc, pderiv_std, pderiv_bc
Output file	reports/report.txt
Omitted	the Timestamp line
Wrapped	long lines
∂	d
.	.
$\oplus$	(+)
ε	epsilon
ϕ	phi
✓	[OK]
Policy	the implementation's internal label, not a claim about disambiguation policy

C.1 deriv_std_rec, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser deriv_std_rec --diag 3
```

```
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Standard (recursive parser)
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
=====
```

```
TIMING
=====
Parse time:          0.012ms
Expressions stored:  3 (r0 -> r1 -> r2)
=====
```

```
FORWARD PASS (Derivatives)
=====
```

```
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
```

```
Step 1: d(r0, 'a')
```

```
rule: d(r1.r2, 'a') [nullable(r1)=false]
      = d(r1, 'a').r2
```

```
r1 = Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps))
```

```
nullable(r1) = true [OK] position 1 matched
```

```
Step 2: d(r1, 'b')
```

```
rule: d(r1.r2, 'b') [nullable(r1)=true]
      = d(r1, 'b').r2 + d(r2, 'b')
```

```
r2 = Alt(Seq(Alt(Phi, Alt(Seq(Phi, Lit('b'))), Eps)), Alt(Lit('b'),
Eps)), Alt(Eps, Phi))
```

```
nullable(r2) = true [OK] position 2 matched
```

```
-----=====
NULLABILITY CHECK
```

```
-----=====
nullable(r2) = true -> proceed to backward pass
```

```
-----=====
BACKWARD PASS (mkEps + inject)
```

```
-----=====
mkEps(r2):
```

```
mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))
```

```
v2 = Left (Right Right (), Right ())
```

```
inject(r1, 'b', v2):
```

```
inject(Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps)), 'b',
Left (Right Right (), Right ()))
```

```
-> (Right ((), b), Right ())
```

```
v1 = (Right ((), b), Right ())
```

```
inject(r0, 'a', v1):
```

```
inject(Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
Eps)), 'a', (Right ((), b), Right ()))
```

```
-> (Right (a, b), Right ())
```

```
v0 = (Right (a, b), Right ())
```

```
-----=====
RESULT
```

```
-----=====
Parse tree:          (Right (a, b), Right ())
```

```
Flattened:           "ab"
```

```
-----=====
END OF REPORT
```

C.2 deriv_bc, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser deriv_bc --diag 3
-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
-----=====
TIMING
-----=====
Parse time:          0.015ms
Derivative steps:    2
-----=====
INTERNALIZE
-----=====
internalize((a|ab)(b|))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([]@[false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([]@[false]@'b' (+) [true]@epsilon))
-----=====
FORWARD PASS (deriv_bc + simp)
-----=====
Step 1 (char: 'a', position 1)
ri0 = []@([]@([]@[false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([]@[false]@'b' (+) [true]@epsilon))
deriv_bc(ri0, 'a'):
  rule: (bs@ri1.ri2) \ 'a' [nullable_bc(ri1)=false]
    = bs@(ri1\'a'.ri2)
simp -> []@([]@([]@[false]@epsilon (+) [true]@'b') . []@([]@[false]@'b' (+)
  [true]@epsilon))
ri1 = []@([]@([]@[false]@epsilon (+) [true]@'b') . []@([]@[false]@'b' (+)
  [true]@epsilon))
  nullable_bc(ri1) = true [OK] bits accumulating

Step 2 (char: 'b', position 2)
ri1 = []@([]@([]@[false]@epsilon (+) [true]@'b') . []@([]@[false]@'b' (+)
  [true]@epsilon))
deriv_bc(ri1, 'b'):
  rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=true]
    = bs@((ri1\'b'.ri2) (+) (fuse (mkEpsBC ri1) (ri2\'b')))
simp -> []@([true]@([]@[false]@'b' (+) [true]@epsilon) (+) [false,
  false]@epsilon)
ri2 = []@([true]@([]@[false]@'b' (+) [true]@epsilon) (+) [false,
```

```

    false]@epsilon)
nullable_bc(ri2) = true   [OK]   bits accumulating

-----
NULLABILITY CHECK
-----
nullable_bc(ri2) = true -> proceed to mkEpsBC + decode
-----
MKEEPSBC + DECODE
-----
mkEpsBC(ri2) = [11]

decode((a|ab)(b|), [11]):
  = (Right (a, b), Right ())
-----
RESULT
-----
Bits:                [11]
Parse tree:          (Right (a, b), Right ())
Flattened:           "ab"
-----
END OF REPORT
-----

```

C.3 pderiv_std, match

```

$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser pderiv_std --diag 3
-----
REGEX ENGINE DEBUG REPORT
-----
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              GREEDY
-----
TIMING
-----
Parse time:          0.017ms
Steps:               2
-----
INITIAL FRONTIER
-----
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
-----
FORWARD PASS (pderiv_tree per residual, frontier flattened)

```

```

-----=====
Step 1 (char: 'a', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  pderiv_tree('a', .) applied to each:
    from [0]:
      rule: pderiv_tree('a', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not printable)
-> Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps)) (+ injection closure,
      not printable)
  frontier after (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  a residual in this frontier is nullable [OK] parse tree available

Step 2 (char: 'b', position 2)
  frontier before (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=true]
= nub_tree $ [(r1'.r2, inj: continue r1) | (r1',_) <-
      pderiv_tree(c,r1)]
++ [(r2', inj: mk_eps(r1) paired with r2's result) | (r2',_) <-
      pderiv_tree(c,r2)]
      -> Eps (+ injection closure, not printable)
    from [1]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not printable)
  frontier after (2 residual(s)):
    [0] Eps
    [1] Seq(Eps, Alt(Lit('b'), Eps))
  a residual in this frontier is nullable [OK] parse tree available

-----=====
SELECTION (first nullable residual, priority/list order)
-----=====
any nullable residual in final frontier = true -> proceed to injection
(already applied by select)
-----=====
RESULT
-----=====
Parse tree:          (Left a, Left b)
Flattened:           "ab"
-----=====

```

END OF REPORT

=====

C.4 pderiv_bc, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \  
--parser pderiv_bc --diag 3
```

=====

REGEX ENGINE DEBUG REPORT

=====

```
Mode:                Bitcoded Partial-Derivative (pDerivBC)  
Regex:               (a|ab)(b|)  
Input:               "ab"  
Result:              MATCH  
Policy:              GREEDY
```

=====

TIMING

=====

```
Parse time:          0.012ms  
Steps:               2
```

=====

INITIAL FRONTIER

=====

```
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),  
  Alt(Lit('b'), Eps))
```

=====

FORWARD PASS (pDerivBC per residual, frontier flattened)

=====

Step 1 (char: 'a', position 1)

frontier before (1 residual(s)):

```
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))  
    bits-so-far = []
```

pDerivBC('a', .) applied to each:

from [0]:

```
rule: pDerivBC('a', r1.r2) [nullable(r1)=false]  
      = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]  
-> (Alt(Lit('b'), Eps), [0])  
-> (Seq(Lit('b'), Alt(Lit('b'), Eps)), [1])
```

frontier after (2 residual(s)):

```
[0] Alt(Lit('b'), Eps)  bits = [0]  
[1] Seq(Lit('b'), Alt(Lit('b'), Eps))  bits = [1]
```

a residual in this frontier is nullable [OK] bits accumulating

Step 2 (char: 'b', position 2)

frontier before (2 residual(s)):

```
[0] Alt(Lit('b'), Eps)  bits-so-far = [0]  
[1] Seq(Lit('b'), Alt(Lit('b'), Eps))  bits-so-far = [1]
```

pDerivBC('b', .) applied to each:

```

    from [0]:
rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++ tag(1,
    pDerivBC('b',r2)), nub2
    -> (Eps, [0])
    from [1]:
    rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
        = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
    -> (Alt(Lit('b'), Eps), [])
frontier after (2 residual(s)):
[0] Eps    bits = [00]
[1] Alt(Lit('b'), Eps)    bits = [1]
a residual in this frontier is nullable [OK] bits accumulating

=====
SELECTION (first nullable residual, priority/list order)
=====
any nullable residual in final frontier = true -> proceed to mkEpsBC +
    decode
=====
MKEEPSBC + DECODE
=====
selected residual's accumulated bits ++ mkEpsBC(residual) = [00]

decode((a|ab)(b|), [00]):
    = (Left a, Left b)
=====
RESULT
=====
Bits:                [00]
Parse tree:          (Left a, Left b)
Flattened:           "ab"
=====
END OF REPORT
=====

```

C.5 deriv_std_rec, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
    --parser deriv_std_rec --diag 3
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Standard (recursive parser)
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              POSIX
=====

```

TIMING

```
=====
Parse time:          0.005ms
Expressions stored:  2 (r0 -> r1)
=====
FORWARD PASS (Derivatives)
=====
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))

Step 1: d(r0, 'b')
  rule: d(r1.r2, 'b') [nullable(r1)=false]
        = d(r1, 'b').r2
r1 = Seq(Alt(Phi, Seq(Phi, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = false [FAIL] position 1

=====
NULLABILITY CHECK
=====
nullable(r1) = false -> no parse tree exists for full input
=====
PARTIAL RECOVERY
=====
No prefix matched.
=====
ERROR SUMMARY
=====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
  b
  ^
=====
END OF REPORT
=====
```

C.6 deriv_bc, no match

```
$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser deriv_bc --diag 3
```

```
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|)
Input:                "b"
Result:              NO MATCH
Policy:              POSIX
=====
```

```

TIMING
=====
Parse time:          0.008ms
Derivative steps:    1
=====
INTERNALIZE
=====
internalize((a|ab)(b|))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
=====
FORWARD PASS (deriv_bc + simp)
=====
Step 1 (char: 'b', position 1)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
deriv_bc(ri0, 'b'):
  rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=false]
    = bs@(ri1\'b'.ri2)
simp -> phi
ri1 = phi
nullable_bc(ri1) = false [FAIL]

=====
NULLABILITY CHECK
=====
nullable_bc(ri1) = false -> no full parse tree exists
=====
PARTIAL RECOVERY
=====
No prefix matched.
=====
ERROR SUMMARY
=====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
  b
  ^
=====
END OF REPORT
=====

```

C.7 pderiv_std, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser pderiv_std --diag 3

```

```

-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              GREEDY
-----=====

TIMING
-----=====
Parse time:          0.007ms
Steps:               1
-----=====

INITIAL FRONTIER
-----=====
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
    Alt(Lit('b'), Eps))
-----=====

FORWARD PASS (pderiv_tree per residual, frontier flattened)
-----=====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
  -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier [FAIL]

-----=====

SELECTION (first nullable residual, priority/list order)
-----=====
any nullable residual in final frontier = false -> no full parse tree
exists
-----=====

PARTIAL RECOVERY
-----=====
No prefix matched.
-----=====

ERROR SUMMARY
-----=====
Failure at position: 1 (1-indexed)
Found:               'b'
Expected:             'a'
  b

```

```

~
-----=====
END OF REPORT
-----=====

```

C.8 pderiv_bc, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser pderiv_bc --diag 3

```

```

-----=====
REGEX ENGINE DEBUG REPORT

```

```

-----=====
Mode:                Bitcoded Partial-Derivative (pDerivBC)
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              GREEDY

```

```

-----=====
TIMING

```

```

-----=====
Parse time:          0.006ms
Steps:               1

```

```

-----=====
INITIAL FRONTIER

```

```

-----=====
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))

```

```

-----=====
FORWARD PASS (pDerivBC per residual, frontier flattened)

```

```

-----=====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
    bits-so-far = []
  pDerivBC('b', .) applied to each:
    from [0]:
      rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
          = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
      -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier [FAIL]

```

```

-----=====
SELECTION (first nullable residual, priority/list order)

```

```

-----=====
any nullable residual in final frontier = false -> no full parse tree
exists

```

PARTIAL RECOVERY

No prefix matched.

ERROR SUMMARY

Failure at position: 1 (1-indexed)
Found: 'b'
Expected: 'a'
b
^

END OF REPORT
